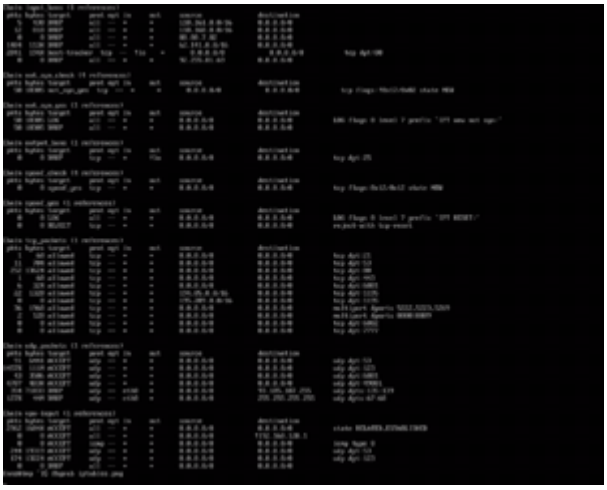
Iptables

Материал из Викиучебника

iptables — утилита командной строки, является стандартным интерфейсом управления работой межсетевого экрана (брандмауэра) **netfilter** для ядер Linux версий 2.4 и 2.6. Для использования утилиты iptables требуются привилегии суперпользователя (root).

Иногда под словом iptables имеется в виду и сам межсетевой экран netfilter.

iptables



Тип	Межсетевой экран
Разработчик	Netfilter Core Team
Написана на	C
ОС	часть ядра Linux
Последняя версия	1.4.21 (2013-Nov-22)
Лицензия	GNU GPL
Сайт	www.netfilter.org (http://www.netfilter.org/)

Содержание

- 1 История
- 2 Архитектура
  - 2.1 Основные понятия
  - 2.2 Принцип работы
  - 2.3 Основные компоненты
    - 2.3.1 netfilter
    - 2.3.2 iptables
    - 2.3.3 conntrack
    - 2.3.4 nfnetlink
    - 2.3.5 ipset

        - 2.3.6 ipvs
    - 2.4 Механизм определения состояний
        - 2.4.1 Критерий состояния соединения
        - 2.4.2 Маркировка соединений
        - 2.4.3 Трансляция сетевых адресов
        - 2.4.4 Использование статистики по соединениям
        - 2.4.5 Ограничение количества соединений
        - 2.4.6 Отслеживание информации о соединениях
- 3 Действия
    - 3.1 Переходы
    - 3.2 Встроенные действия
    - 3.3 Терминальные и нетерминальные действия
- 4 Таблицы
    - 4.1 Таблица mangle
        - 4.1.1 Цепочки
        - 4.1.2 Действия
    - 4.2 Таблица nat
        - 4.2.1 Цепочки
        - 4.2.2 Действия
    - 4.3 Таблица filter
        - 4.3.1 Цепочки
        - 4.3.2 Действия
    - 4.4 Таблица security
        - 4.4.1 Цепочки
        - 4.4.2 Действия
    - 4.5 Таблица raw
        - 4.5.1 Цепочки
        - 4.5.2 Действия
    - 4.6 Таблица rawpost
        - 4.6.1 Цепочки
        - 4.6.2 Действия
- 5 Критерии
    - 5.1 Универсальные критерии
    - 5.2 Критерии, специфичные для протоколов
        - 5.2.1 Протоколы транспортного уровня
            - 5.2.1.1 TCP
            - 5.2.1.2 UDP
            - 5.2.1.3 SCTP
            - 5.2.1.4 DCCP
            - 5.2.1.5 UDP Lite
        - 5.2.2 IPv4
            - 5.2.2.1 ICMP
        - 5.2.3 IPv6
            - 5.2.3.1 ICMPv6
            - 5.2.3.2 MH
        - 5.2.4 IPsec

- 5.2.4.1 AH
- 5.2.4.2 ESP
- 5.3 Критерии состояния соединения
  - 5.3.1 conntrack
  - 5.3.2 state
- 5.4 Дополнительные критерии
  - 5.4.1 Вспомогательные критерии
  - 5.4.2 Критерии маркировки
  - 5.4.3 Лимитирующие критерии
  - 5.4.4 Критерий recent
  - 5.4.5 Критерий u32
  - 5.4.6 Прочие критерии
  - 5.4.7 Критерии из набора xtables-addons
- 6 Программы
  - 6.1 Основные
    - 6.1.1 iptables
      - 6.1.1.1 Работа с цепочками
      - 6.1.1.2 Работа с правилами
      - 6.1.1.3 Вывод информации
    - 6.1.2 iptables-save, iptables-restore и iptables-xml
    - 6.1.3 /etc/init.d/iptables
    - 6.1.4 iptables-apply
    - 6.1.5 ipset
  - 6.2 Вспомогательные
    - 6.2.1 Полезные при конфигурировании фаервола
    - 6.2.2 Полезные при тестировании фаервола
- 7 Фронтенды
  - 7.1 С удобным интерфейсом
  - 7.2 С графическим интерфейсом
  - 7.3 С веб-интерфейсом
  - 7.4 Программные интерфейсы
- 8 Модули ядра
  - 8.1 Модули conntrack и nat
  - 8.2 Модули критериев и действий
- 9 Параметры sysctl/procfs
  - 9.1 Псевдофайлы procfs
    - 9.1.1 Базовые
    - 9.1.2 Относящиеся к критериям и действиям
  - 9.2 Основные параметры sysctl
  - 9.3 Параметры конфигурации интерфейсов
  - 9.4 Параметры, относящиеся к протоколам
    - 9.4.1 TCP
    - 9.4.2 UDP
    - 9.4.3 ICMP
  - 9.5 Параметры conntrack
- 10 Расширения

- 10.1 Userspace-компоненты
  - 10.1.1 NuFW
  - 10.1.2 l7-filter-userspace
  - 10.1.3 iplist
- 10.2 Наборы дополнительных критериев и действий
  - 10.2.1 Patch-o-Matic
  - 10.2.2 Patch-o-Matic-NG
  - 10.2.3 xtables-addons
- 11 Команды
  - 11.1 Команды модификации правил
  - 11.2 Параметры определения правил
- 12 Модули
  - 12.1 Некоторые из встроенных (входящие в стандартный пакет)
    - 12.1.1 state
    - 12.1.2 connlimit
    - 12.1.3 iprange
    - 12.1.4 multiport
    - 12.1.5 mac
- 13 Примечания
- 14 Литература
- 15 Ссылки

История

Изначально разработка netfilter и iptables шла совместно, поэтому в ранней истории этих проектов есть много общего. Подробности см. в статье про netfilter.

Предшественниками iptables были проекты ipchains (применялась для администрирования фаервола ядра Linux версии 2.2) и ipfwadm (аналогично для ядер Linux версий 2.0). Последний был основан на BSD-утилите ipfw.

iptables сохраняет идеологию, ведущую начало от ipfwadm: функционирование фаервола определяется набором правил, каждое из которых состоит из критерия и действия, применяемого к пакетам, подпадающим под этот критерий. В ipchains появилась концепция *цепочек* — независимых списков правил. Были введены отдельные цепочки для фильтрации входящих (INPUT), исходящих (OUTPUT) и транзитных (FORWARD) пакетов. В продолжении этой идеи, в iptables появились *таблицы* — независимые группы цепочек. Каждая таблица решала свою задачу — цепочки таблицы filter отвечали за фильтрацию, цепочки таблицы nat — за преобразование сетевых адресов (NAT), к задачам таблицы mangle относились прочие модификации заголовков пакетов (например, изменение TTL или TOS). Кроме того, была слегка изменена логика работы цепочек: в ipchains все входящие пакеты, включая

транзитные, проходили цепочку INPUT. В iptables через INPUT проходят только пакеты, адресованные самому хосту.

Такое разделение функциональности позволило iptables при обработке отдельных пакетов использовать информацию о соединениях в целом (ранее это было возможно только для NAT). В этом iptables значительно превосходит ipchains, так iptables может отслеживать состояние соединения и перенаправлять, изменять или отфильтровывать пакеты, основываясь не только на данных из их заголовков (источник, получатель) или содержимом пакетов, но и на основании данных о соединении. Такая возможность фаервола называется stateful-фильтрацией, в отличие от реализованной в ipchains примитивной stateless-фильтрации (подробнее о видах фильтрации см. статью о фаерволах). Можно сказать, что iptables анализирует не только передаваемые данные, но и контекст их передачи, в отличие от ipchains, и поэтому может принимать более обоснованные решения о судьбе каждого конкретного пакета. Более подробно о stateful-фильтрации в netfilter/iptables см. w:Netfilter#Механизм определения состояний.

В будущем, разработчики netfilter планируют заменить iptables на nftables — инструмент нового поколения, пока находящийся в ранней стадии разработки[1].

Архитектура

Основные понятия

Ключевыми понятиями iptables являются:

- **Правило** — состоит из *критерия*, *действия* и *счетчика*. Если пакет соответствует критерию, к нему применяется действие, и он учитывается счетчиком. Критерия может и не быть — тогда неявно предполагается критерий «все пакеты». Указывать действие тоже не обязательно — в отсутствие действия правило будет работать только как счетчик.
  - **Критерий** — логическое выражение, анализирующее свойства пакета и/или соединения и определяющее, подпадает ли данный конкретный пакет под действие текущего правила.
  - **Действие** — описание действия, которое нужно проделать с пакетом и/или соединением в том случае, если они подпадают под действие этого правила. О действиях более подробно будет рассказано ниже.
  - **Счетчик** — компонент правила, обеспечивающий учет количества пакетов, которые попали под критерий данного правила. Также счетчик учитывает суммарный объем таких пакетов в байтах.
- **Цепочка** — упорядоченная последовательность правил. Цепочки

можно разделить на *пользовательские* и *базовые*.

- **Базовая цепочка** — цепочка, создаваемая по умолчанию при инициализации таблицы. Каждый пакет, в зависимости от того, предназначен ли он самому хосту, сгенерирован им или является транзитным, должен пройти положенный ему набор базовых цепочек различных таблиц. Схема следования пакетов приведена на рисунке. Кроме того, базовая цепочка отличается от пользовательской наличием «действия по умолчанию» (default policy). Это действие применяется к тем пакетам, которые не были обработаны другими правилами этой цепочки и вызванных из нее цепочек (см. переходы). Имена базовых цепочек всегда записываются в верхнем регистре (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING).
- **Пользовательская цепочка** — цепочка, созданная пользователем. Может использоваться только в пределах своей таблицы. Рекомендуется не использовать для таких цепочек имена в верхнем регистре, чтобы избежать путаницы с базовыми цепочками и встроенными действиями.

- **Таблица** — совокупность *базовых* и *пользовательских* цепочек, объединенных общим функциональным назначением. Имена таблиц (как и модулей критериев) записываются в нижнем регистре, так как в принципе не могут конфликтовать с именами пользовательских цепочек. При вызове команды iptables таблица указывается в формате `-t имя_таблицы`. При отсутствии явного указания, используется таблица filter. Более подробно таблицы будут рассмотрены ниже.

Принцип работы

Все пакеты пропускаются через определенные для них последовательности цепочек (см. рис.). При прохождении пакетом цепочки, к нему последовательно применяются все правила этой цепочки в порядке их следования. Под применением правила понимается: во-первых, проверка пакета на соответствие критерию, и во-вторых, если пакет этому критерию соответствует, применение к нему указанного действия. Под действием может подразумеваться как элементарная операция (встроенное действие, например, ACCEPT, MARK), так и переход в одну из пользовательских цепочек. В свою очередь, действия могут быть как терминальными, то есть прекращающими обработку пакета в рамках данной базовой цепочки (например, ACCEPT, REJECT), так и нетерминальными, то есть не прерывающими процесса обработки пакета (MARK, TOS). Если пакет прошел через всю базовую цепочку и к нему так и не было применено ни одного терминального действия, к нему применяется действие по умолчанию для данной цепочки (обязательно терминальное). Например,

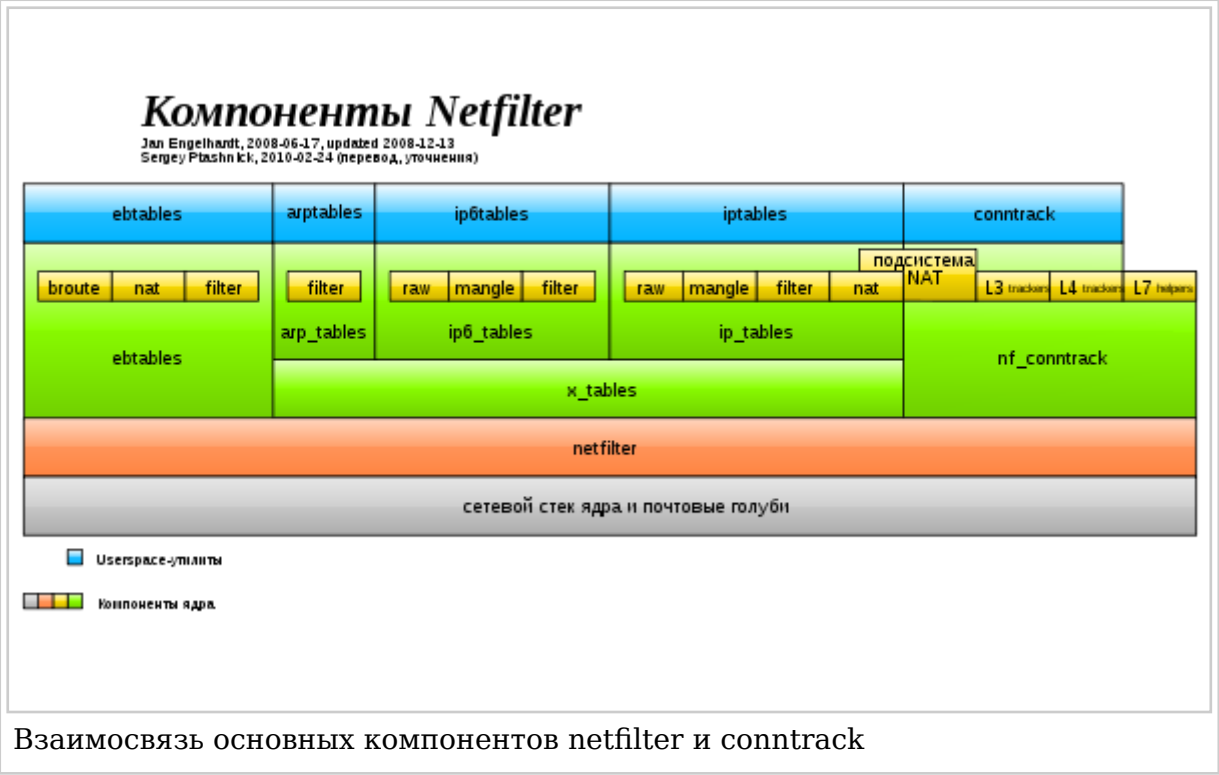
```
iptables -F # Очищаем все цепочки таблицы filter
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Ко всем пакетам, которые
```

```
# соединениям, применяем терминальное действие ACCEPT – пропустить
iptables -P INPUT DROP # В качестве действия по умолчанию устанавливаем DROP – блокирование пакета
iptables -P OUTPUT ACCEPT # Разрешаем все исходящие пакеты
```

Теперь цепочка INPUT таблицы filter содержит единственное правило, которое пропускает все пакеты, относящиеся к уже установленным соединениям. Ко всем остальным входящим пакетам будет применено действие по умолчанию — DROP. Цепочка же OUTPUT вообще не содержит правил, поэтому ко всем исходящим пакетам будет применяться действие по умолчанию ACCEPT. Таким образом хост, настроенный согласно этому примеру и подключенный к Интернету, будет недоступен извне (все попытки установить соединение снаружи блокируются), однако с самого хоста доступ к Интернету будет свободный (исходящие пакеты разрешены, а ответы на них уже относятся к установленным соединениям).

Основные компоненты

netfilter



Взаимосвязь основных компонентов netfilter и conntrack

Компонент ядра Linux, обеспечивающий фильтрацию и модификацию трафика. Собственно, именно он и является фаерволом. В состав netfilter входят следующие модули:

- **ip_tables** — фаервол для протокола IPv4. Обеспечивает фильтрацию пакетов, модификацию их заголовков и трансляцию сетевых адресов.
- **ip6_tables** — фаервол для протокола IPv6. Обеспечивает фильтрацию пакетов и модификацию их заголовков.

- **arp_tables** — фаервол для протоколов ARP и RARP. Обеспечивает фильтрацию и модификацию пакетов.
- **x_tables** — бэкенд для ip_tables, ip6_tables и arp_tables. В этом модуле определены основные операции для работы с фаерволами «таблично-цепочечной» структуры и их компонентами.
- **ebtables** — Ethernet-фаервол (префикс eb от Ethernet Bridge). В отличие от трех перечисленных выше фаерволов, работающих с протоколами сетевого и более высоких уровней, ebtables работает на канальном уровне, выполняя фильтрацию и модификацию ethernet-кадров, проходящих через сетевые мосты, если таковые имеются на хосте.

Принципы работы с ip_tables и ip6_tables практически идентичны, с тем небольшим отличием, что в ip6_tables отсутствует таблица nat и соответствующие действия. В то же время, принципы построения правил в arp_tables и ebtables несколько иные. В частности, arp_tables и ebtables имеют четыре базовых действия — ACCEPT (пропустить), DROP (заблокировать), CONTINUE (продолжить обработку в данной цепочке) и RETURN (прекратить обработку в данной цепочке, вернуть в цепочку уровнем выше). Эти действия могут использоваться отдельно, либо применяться к пакетам после выполнения какого-либо преобразующего действия (например, маркировки). Таким образом, после применения преобразования пакет может быть, скажем, немедленно пропущен фаерволом (ACCEPT), либо будет вынужден проследовать через последующие правила (CONTINUE). Этим arptables и ebtables отличаются от ip_tables и ip6_tables — в последних существует четкое разделение действий на терминальные и нетерминальные, которое нельзя изменить при составлении набора правил, а чтобы, например, пропустить пакет после некоторого нетерминального преобразования, необходимо добавлять дополнительное правило («`iptables ... -j MARK ...; iptables ... -j ACCEPT`» вместо «`ebtables ... -j mark ... --mark-target ACCEPT`»). Также, система conntrack не производит отслеживание соединений для ARP/RARP и Ethernet, поэтому возможности stateful-фильтрации в arp_tables и ebtables отсутствуют (нет критерия conntrack и таблицы raw).

В данной статье рассматриваются только аспекты работы с ip_tables и ip6_tables. В их задачи входит:

- Классификация пакетов на основе различных критериев. В качестве критерия могут выступать, например, IP-адреса источника и/или назначения, состояние соединения (новое/уже установленное), порты источника и/или назначения (для протоколов транспортного уровня, имеющих порты), вспомогательные значения в заголовках пакетов (скажем, длина пакета, TOS, TTL) и т. п. Результаты этой классификации используются при решении других задач из данного списка.
- Фильтрация входящих (INPUT), исходящих (OUTPUT) и транзитных (FORWARD) пакетов, сводящуюся либо к их пропусканию (ACCEPT),

либо к блокированию (DROP или REJECT). Также поддерживаются дополнительные возможности, например, блокирование попыток сканирования портов (DELUDE и CHAOS), эффективное противодействие (D)DoS-атакам (TARPIT).
- Модификация заголовков пакетов и связанной информации. К этому классу задач можно отнести:
  - Трансляцию сетевых адресов (NAT), включая маскарадинг (SNAT или MASQUERADE), «проброс» адресов и портов (DNAT) и даже целых подсетей (NETMAP). Эта задача касается только модуля ip_tables. Модуль ip6_tables не производит трансляции адресов.
  - Изменение различных вспомогательных величин в заголовках пакетов, например, TOS, TTL, TCP MSS.
  - Установку и изменение маркировки пакетов и соединений. Впоследствии эта маркировка может быть использована системой iproute2 для маршрутизации и шейпинга трафика.
- Учет количества обработанных пакетов и их суммарного размера при помощи счетчиков.
- Занесение информации о пакетах в системный журнал (LOG и LOGMARK).
- Передача на обработку userspace-демонам пакетов целиком (NFQUEUE) либо информации о них (NFLOG), что позволяет решать самый широкий круг задач по фильтрации, модификации и учету трафика.
- Внутренние задачи, связанные с обеспечением работы критериев и действий netfilter. Например, сопровождение списков IP-адресов для критериев recent и hashlimit.

iptables

Userspace-утилита, через которую системный администратор может управлять IPv4-фаерволом (ip_tables). К ее задачам относятся:

- Создание и удаление пользовательских цепочек.
- Установка действий по умолчанию для базовых цепочек.
- Добавление и удаление правил.
- Установка и обнуление счетчиков пакетов и байт.
- Просмотр цепочек и правил, а также значений счетчиков.
- Проверка корректности задания параметров, определяющих работу критериев и действий.
- Вывод справки по использованию критериев (`iptables -m` *`критерий`* `-h`) и действий (`iptables -j` *`действие`* `-h`).

Также в комплект поставки вместе с iptables обычно входят вспомогательные утилиты, обеспечивающие сохранение (iptables-save) и последующее восстановление (iptables-restore) состояния фаервола, его инициализацию при запуске системы (init-скрипт) и т. п.

Сегментом фаервола, отвечающим за фильтрацию IPv6-пакетов

(ip6_tables), управляет утилита ip6tables. Так как обычно она поставляется вместе с iptables, имеет похожий синтаксис и выполняет схожие задачи, под термином «iptables» часто подразумевают сразу обе эти утилиты.

Также в рамках данного проекта разрабатываются два набора библиотек:

- **libiptc** — содержит функции, осуществляющие вышеперечисленные операции по управлению цепочками и правилами. Именно с этими функциями и работают утилиты iptables (библиотека libip4tc) и ip6tables (библиотека libip6tc). Приложения, использующие эти библиотеки, могут обращаться к netfilter напрямую, минуя вызов утилит iptables и ip6tables. В качестве примера такого приложения можно назвать Perl-модуль IPTables::libiptc (http://search.cpan.org/~hawk/IPTables-libiptc-0.18/lib/IPTables/libiptc.pm), предоставляющий доступ к функциям этих библиотек из Perl-скриптов.
- **libipq** — содержит функции, позволяющие пользовательским приложениям принимать IP-пакеты на обработку. Отправка со стороны ядра выполняется через модуль ip_queue (ip6_queue для IPv6), что соответствует действию QUEUE. В настоящее время этот механизм объявлен устаревшим (http://netfilter.org/projects/libnetfilter_queue/index.html), и вместо него рекомендуется использовать действие NFQUEUE, работа которого реализуется через модуль ядра nfnetlink_queue и библиотеку libnetfilter_queue.

conntrack

Компонент netfilter, обеспечивающий отслеживание состояния соединений и классификацию пакетов с точки зрения принадлежности к соединениям, что позволяет netfilter осуществлять полноценную stateful-фильтрацию трафика. Как и netfilter, система conntrack является частью ядра Linux. К его задачам относятся:

- Отслеживание состояний отдельных соединений с тем, чтобы классифицировать каждый пакет либо как относящийся к уже установленному соединению, либо как открывающий новое соединение. При этом понятие "состояние соединения" искусственно вводится для протоколов, в которых оно изначально отсутствует (UDP, ICMP). При работе же с протоколами, поддерживающими состояния (например, TCP), conntrack активно использует эту возможность, тесно взаимодействуя с базовой сетевой подсистемой ядра Linux.
- Отслеживание связанных соединений, например, ICMP-ответов на TCP и UDP-пакеты. Более сложный вариант — протоколы, использующие несколько соединений в одной сессии, например, FTP. Для правильной обработки таких протоколов conntrack использует специальные модули (conntrack helpers), которые анализируют трафик и «выхватывают» информацию протокола о новых соединениях (например, порт, на который оно будет открыто), что позволяет обеспечить их корректную фильтрацию, маршрутизацию, шейпинг и

пропускание через NAT.

Более подробно о возможностях системы conntrack и их использовании при работе с iptables/netfilter см. чуть ниже.

В состав conntrack входят следующие компоненты:

- Главный модуль nf_conntrack. Реализует базовую функциональность отслеживания соединений, интерфейсы для работы с модулями расширений, а также механизмы отслеживания протоколов TCP и UDP.
- Трекеры протоколов сетевого уровня: nf_conntrack_ipv4 и nf_conntrack_ipv6. Реализуют операции, специфичные для «своих» протоколов (IPv4 и IPv6 соответственно), а также механизмы отслеживания соответствующих протоколов управляющих сообщений (ICMP и ICMPv6 соответственно).
- Трекеры протоколов транспортного уровня (кроме TCP и UDP, реализованных в главном модуле): nf_conntrack_proto_sctp (SCTP), nf_conntrack_proto_dccp (DCCP), nf_conntrack_proto_gre (GRE), nf_conntrack_proto_udplite (UDP Lite).
- Вспомогательные модули (helpers), обеспечивающие отслеживание протоколов прикладного и сеансового уровней: nf_conntrack_ftp, nf_conntrack_irc, nf_conntrack_pptp, nf_conntrack_netbios_ns и т. п. Требуются далеко не для всех протоколов, а только для тех, отслеживание которых требует знания специфики протокола. Например, неоднократно упоминающийся в этой статье протокол FTP использует в одном сеансе два соединения (управляющее соединение и соединение передачи данных), и чтобы корректно отследить соединение передачи данных, необходимо анализировать трафик управляющего соединения и выделять в нем информацию, касающуюся открытия соединения данных.
- Модуль nf_nat. Реализует базовую функциональность трансляции сетевых адресов и портов с учетом информации о соединениях. Также этот модуль содержит механизмы трансляции для протоколов IPv4, TCP, UDP и ICMP (IPv6 и ICMPv6 не поддерживаются NAT-подсистемой conntrack).
- NAT-трекеры протоколов транспортного уровня: nf_nat_proto_sctp, nf_nat_proto_dccp, nf_nat_proto_gre, nf_nat_proto_udplite.
- Вспомогательные модули NAT для протоколов прикладного и сеансового уровней: nf_nat_ftp, nf_nat_irc, nf_nat_pptp и т. п.
- Модуль взаимодействия с userspace через протокол nfnetlink (см. следующий раздел) nf_conntrack_netlink, которому соответствует userspace-библиотека libnetfilter_conntrack.
- Кроме того, к conntrack можно условно отнести и модули netfilter, обеспечивающие взаимодействие этих двух подсистем: критерий conntrack (xt_conntrack), действия NOTRACK и CT (xt_NOTRACK и xt_CT), практически все действия таблицы nat, а также соответствующие модули (библиотеки iptables/ip6tables):

libxt_conntrack, libxt_NOTRACK, libxt_CT и т. п.

Для взаимодействия с системой conntrack из userspace разработан комплект conntrack-tools (http://conntrack-tools.netfilter.org/), содержащий две утилиты:

- **conntrack** — инструмент, позволяющий системному администратору наблюдать таблицы состояний соединений и взаимодействовать с ними: очищать таблицы целиком, удалять отдельные записи, маркировать соединения вручную (аналог действия CONNMARK), устанавливать тайм-ауты соединений. Поддерживается фильтрация вывода (например, на основании адресов и/или портов источника и/или назначения), а также вывод в различных форматах, включая XML. Кроме того, данная утилита позволяет отслеживать в реальном времени события системы conntrack, например, открытие новых соединений или изменение состояния существующих.
- **conntrackd** — демон, обеспечивающий синхронизацию таблиц состояний с другими хостами, что в сочетании с возможностями демона keepalived позволяет создавать фаерволы на кластерах высокой доступности (High Availability) — при выходе из строя одного из хостов кластера его соединения будут «подхвачены» другими хостами кластера и корректно обработаны с учетом состояний. Также conntrackd можно использовать просто для удаленного сбора статистики по соединениям.

Взаимодействие conntrack с этими программами производится посредством интерфейса nfnetlink, который будет рассмотрен в следующем разделе.

nfnetlink

Интерфейс, позволяющий различным userspace-приложениям взаимодействовать с netfilter и conntrack. Со стороны userspace он обеспечивается набором библиотек libnfnetlink. Ключевые библиотеки из этого набора:

- **libnetfilter_conntrack** (ранее libnfnetlink_conntrack и libctnetlink) — обеспечивает взаимодействие приложений с системой conntrack. В качестве примеров приложений, использующих эту библиотеку, можно упомянуть:
  - Описанные выше программы **conntrack** и **conntrackd** из комплекта conntrack-tools.
  - **iptstate** — в какой-то мере аналог утилиты conntrack, хотя и не имеющий таких богатых возможностей. Предназначена для непрерывного вывода таблицы состояний соединений с периодическим обновлением, в стиле широко известной утилиты top. Поддерживает различные виды сортировки (по адресам/портам источника/назначения, тайм-ауту, счетчикам и т.

п.). Позволяет удалять записи из таблицы состояний.
Не поддерживает работу с таблицей ожидаемых (expected) соединений.

- **libnetfilter_queue** (ранее libnfnetlink_queue) — отвечает за передачу пакетов демонам на предмет анализа. По результатам этого анализа пакет может быть заблокирован или пропущен. При пропускании пакета возможна установка или изменение его маркировки (nfmark). Идеологическим предшественником libnetfilter_queue была ныне устаревшая библиотека libipq. В качестве примеров приложений, использующих libnetfilter_queue, можно упомянуть:
  - **nufw** — фаервол, выполняющий фильтрацию трафика на основе авторизации пользователей. Позволяет устанавливать ограничения на доступ к сетям, находящимся за фаерволом, для отдельных пользователей. Авторизация пользователей на фаерволе обеспечивается отдельным демоном nuauth. Например, в случае, если клиенты используют Linux, авторизация может выполняться совершенно прозрачно, благодаря возможностям PAM (модуль pam_nufw) в момент входа пользователя в систему. Особенно удобны подобные системы в корпоративных сетях, где используются системы централизованного управления аккаунтами пользователей (nufw/nuauth поддерживает LDAP и winbind).
  - **l7-filter-userspace** — демон, позволяющий определить протокол 7-го (прикладного) уровня модели OSI, которому принадлежит полученный пакет, на основании анализа его содержимого при помощи регулярных выражений. Результат анализа возвращает в маркировке пакета. Полезен при фильтрации и шейпинге трафика протоколов, не имеющих фиксированных номеров портов для соединений данных, например, BitTorrent. Впрочем, в настоящее время существуют и альтернативные решения — P2P-протоколы можно выделять при помощи критерия ipp2p (из набора xtables-addons), а вспомогательные соединения в известных системе conntrack протоколах (FTP, IRC, SIP) можно выделять при помощи критерия helper и маркировки соединений.
  - **iplist** — userspace-альтернатива ipset. Как и ipset, позволяет считывать большие списки IP-адресов и проверять пакеты на предмет нахождения адреса их источника/назначения в данном списке. В зависимости от результата проверки, пакет может быть пропущен или заблокирован, либо промаркирован соответствующим образом. Разумеется, ipset, работая на уровне ядра, обеспечивает более высокую скорость проверки, а также поддерживает дополнительные возможности, например, динамическое изменение списков при помощи действия SET (добавление и удаление записей), сохранение дополнительной информации (номера портов, MAC-адреса, тайм-ауты записей). Несмотря на то, что в настоящий момент ipset пока что не принят в ядро, современные дистрибутивы ipset позволяет произвести

сборку и установку необходимых компонентов без необходимости наложения патчей на ядро и iptables, благодаря возможностям подгружаемых модулей (en:LKM) ядра Linux. Поэтому, с учетом вышесказанного, необходимость в iplist в настоящее время сомнительна.

- **libnetfilter_log** (ранее libnfnetlink_log) — предоставляет демонам интерфейс для получения служебной информации о пакетах на предмет регистрации и учета трафика. В настоящее время известен только один проект, использующий данную библиотеку — **ulogd2** ([netfilter] userspace logging daemon, версия 2). Данный проект находится пока в бета-стадии разработки. Надо заметить, что ulogd2 может получать информацию не только через libnetfilter_log (действие NFLOG), но и через классический механизм libipulog (действие ULOG), а также через libnetfilter_conntrack (данный метод позволяет получать информацию не об отдельных пакетах, а о соединениях в целом).

Со стороны ядра взаимодействие обеспечивается базовым модулем nfnetlink и «специализированными» модулями nf_conntrack_netlink, nfnetlink_queue и nfnetlink_log соответственно.

Кроме того, такие программы, как nfnl_osf (передает ядру список сигнатур для критерия детекции операционной системы osf) и ipset начиная с версии 5 (см. ниже) также используют для коммуникаций ядро-userspace интерфейс nfnetlink, но эти задачи являются узкоспециализированными и поэтому реализованы без использования специальных библиотек и модулей ядра.

ipset

Представляет собой набор инструментов, позволяющий работать с большими списками (sets) IP-адресов и/или портов. Поддерживается возможность динамического обновления списков при прохождении пакетов через правила netfilter.

В версии ipset 4 и более ранних поддерживалась работа только с семейством адресов IPv4, однако начиная с пятой версии реализована также и поддержка IPv6.

Стоит отметить, что долгое время проект ipset существовал вне основной ветки развития ядра Linux. В частности, в четвертой версии ipset «ядерные» компоненты приходилось собирать отдельно, используя заголовочные файлы текущего ядра. Однако, после выхода пятой версии, в которой были учтены все предыдущие замечания, началась работа по включению ipset в основную ветку. Она завершилась успехом (http://www.spinics.net/lists/netfilter-devel/msg17027.html): поддержка ipset присутствует в Linux начиная с 2.6.39. В ходе этих работ протокол взаимодействия ядерной и userspace-частей ipset подвергся значительной переработке, и окончательная версия получила (http://git.netfilter.org

/cgi-bin/gitweb.cgi?p=ipset.git;a=commit;h=d6ee5bb4e5346b84efa33858091ba4add2b26de2) номер 6.0. Таким образом, в современных ядрах поддержка ipset присутствует штатно, а пользователи более старых ядер могут воспользоваться реализациями ipset версий 4 и 5, включенными в набор xtables-addons (http://xtables-addons.sourceforge.net/).

ipset состоит из следующих элементов:

- Модуль ядра ip_set, обеспечивающий базовый инструментарий работы со списками записей, который используется модулями расширений.
- Модули расширений, определяющие конкретные структуры записей и методы работы с ними, например, ip_set_hash_ip (тип данных hash:ip), ip_set_bitmap_port (тип данных bitmap:port) и т. д.
- Userspace-утилита ipset, позволяющая выполнять различные операции со списками и записями в них:
  - Создание, удаление, переименование, вывод, очистка списков.
  - Добавление и удаление записей из списков, проверка наличия записи в списке.
  - Вывод справочной информации по работе со списками различного типа.
- Модули расширений для утилиты ipset, соответствующие модулям расширений в ядре, например, ipset_hash_ip (тип данных hash:ip), ipset_bitmap_port (тип данных bitmap:port) и т. д. Каждый такой модуль отвечает за обеспечение интерфейса между «своим» модулем ядра и конечным пользователем, в частности, проверку корректности вводимых и форматирование выводимых данных, а также вывод справочной информации по допустимым командам и параметрам. В ранних версиях ipset (до 4 включительно) такие модули выделялись в виде подгружаемых библиотек, однако в современных версиях ipset они статически включаются в основной бинарник, поэтому разделение существует лишь в исходном коде.
- Модуль netfilter xt_set (критерий set и действие SET), а также соответствующие модули (библиотеки) iptables (libxt_set и libxt_SET). Критерий set позволяет проверять различные параметры пакета (IP/MAC-адреса, TCP/UDP-порты источника и/или получателя) на предмет нахождения или отсутствия в списке. Действие SET позволяет добавлять или удалять записи из списка на основании указанных параметров пакета.

ipset поддерживает следующие типы данных (в скобках приведены названия согласно терминологии устаревшей версии ipset 4):

- **bitmap:ip** (ipmap) — полный перечень IP-адресов.
- **bitmap:ip,mac** (macipmap) — полный перечень IP-адресов, причем вместе с каждым IP-адресом может быть сохранен MAC-адрес.
- **bitmap:port** (portmap) — полный перечень портов.
- **hash:ip** (iphash) — выборочный перечень IP-адресов (либо IP-подсетей

с одинаковой маской).
- **hash:net** (nethash) — выборочный перечень IP-подсетей (блоков IP-адресов). В отличие от hash:ip, в одном списке могут присутствовать подсети с различными масками.
- **hash:ip,port** (ipporthash) — выборочный перечень IP-адресов, причем с каждым адресом может быть сохранен порт. В версии ipset 4 и более ранних существует ограничение: адреса в пределах одного перечня должны принадлежать одному блоку /16 (/255.255.0.0, максимум 65536 адресов). Начиная с версии ipset 5, для каждого номера порта можно также сохранять идентификатор протокола транспортного уровня.
- **hash:ip,port,ip** (ipportiphash) — выборочный перечень IP-адресов, причем с каждым адресом может быть сохранен порт и еще один IP-адрес. В версии ipset 4 и более ранних ограничение на «первичные» адреса (первые в тройке адрес-порт-адрес) такое же, как и для ipporthash. Начиная с версии ipset 5, для каждого номера порта можно также сохранять идентификатор протокола транспортного уровня.
- **hash:ip,port,net** (ipportnethash) — выборочный перечень IP-адресов, причем с каждым адресом может быть сохранен порт и IP-подсеть. В версии ipset 4 и более ранних ограничение на «первичные» адреса такое же, как и у типа ipporthash. Начиная с версии ipset 5, для каждого номера порта можно также сохранять идентификатор протокола транспортного уровня.
- **list:set** (setlist) — список списков. Может содержать списки любого перечисленного здесь типа, кроме list:set. При обращении к такому объекту из netfilter, включенные в него списки рассматриваются как один большой список. Поиск записей (критерий set) производится по всем вложенным спискам соответствующего типа. При добавлении записей (действие SET), вложенные списки поочередно проверяются (согласно порядку их перечисления) на соответствие типа и наличие свободного места, и запись добавляется в первый же подходящий. В то же время, при обращении к такому списку через утилиту ipset, он рассматривается именно как совокупность элементов-списков, что позволяет добавлять, удалять и проверять наличие именно вложенных списков, но не их элементов.
- **hash:net,iface** — выборочный перечень IP-подсетей, причем с каждой подсетью может быть сохранено название сетевого интерфейса. Появился в версии ipset 6.7 (входит в Linux 3.1), ранние версии ipset не имеют подобных возможностей. Существует ограничение: нельзя сохранять более 64 имен интерфейсов с одним и тем же адресом подсети. Этот тип данных удобен при наличии в системе большого количества сетевых интерфейсов (например, сотен VLAN-интерфейсов).

В четвертой версии дополнительно присутствовали два типа списков, упраздненные в более поздних версиях:

- **iptree** — выборочный перечень IP-адресов, причем с каждым адресом может быть сохранено значение тайм-аута в секундах. По истечении тайм-аута адрес удаляется из списка. Чтобы активировать возможность использование тайм-аутов для определенного списка, нужно при его создании задать тайм-аут по умолчанию (опция `--timeout`). Начиная с версии ipset 5, данный тип не поддерживается, при попытке его использования автоматически заменяется на hash:ip. Заметим, что начиная с пятой версии тайм-ауты реализованы для всех поддерживаемых типов данных.
- **iptreemap** — гибридный перечень, позволяющий сохранять адреса, блоки и диапазоны адресов. Начиная с версии ipset 5, данный тип не поддерживается, при попытке его использования автоматически заменяется на hash:ip. Стоит отметить, что в современных версиях ipset при работе с адресами IPv4 можно указывать диапазоны адресов и подсети, которые при обращении к внутренним спискам будут автоматически преобразованы в наборы соответствующих элементов, Например, `ipset create foo hash:ip; ipset add foo 192.168.0.0-192.168.0.3` добавит в список `foo` адреса 192.168.0.0, 192.168.0.1, 192.168.0.2 и 192.168.0.3 (аналогичный эффект может быть достигнут и командой `ipset add foo 192.168.0.0/30`).

Практически для всех типов списков, реализованных в ipset, существует общее ограничение на максимальный размер: 65536 записей.

Большинство перечисленных типов данных можно разделить на две группы: полные перечни (map, bitmap) и выборочные перечни (hash, tree). Разница между этими типами состоит в следующем: если выборочный перечень сохраняет только те элементы, которые в него входят, то полный перечень представляет собой таблицу логических значений (битов) для всех элементов, которые могут входить в данный перечень, и вхождение конкретного элемента в перечень определяется значением соответствующего бита. Полный перечень всегда формируется на базе *одного непрерывного* диапазона значений.

Таким образом, выбор необходимого вам типа данных определяется конкретными условиями задачи, прежде всего, отношением усредненного количества элементов в списке к требуемому диапазону охвата. Например, если вы собираетесь хранить в списке блэк-листы адресов, замеченных в атаках на ваш сервер, целесообразнее использовать тип hash:ip, потому что атакующие вас ботнеты, как бы они ни были велики, все равно занимают незначительную долю адресного пространства IPv4. К тому же, как следует из сказанного в предыдущем абзаце, тип bitmap:ip все равно не может хранить адреса, выходящие за предел одной подсети префикса 16. Но в том случае, если вам, например, нужно задать список хостов из вашей локальной сети, имеющих доступ к каким-либо услугам (службам вашего сервера, выходу в Интернет), и этот доступ должно иметь значительное количество хостов (а не единицы), то целесообразнее будет использовать bitmap:ip.

По поводу хранения адресов подсетей (блоков IP-адресов) можно сказать следующее. Если вам нужно хранить большое количество блоков одного размера из ограниченного диапазона (не более 65536 возможных значений), например, блоки /24 из подсети /8, вы можете использовать тип bitmap:ip, указав маску. Если же вам нужно хранить блоки одного размера из достаточно широкого диапазона, например, /24 из всего адресного пространства IPv4, используйте тип hash:ip, также с указанием маски. При необходимости хранить в одном списке блоки *разного* размера, воспользуйтесь типом hash:net.

Что касается улучшений в версиях 5 и 6 по сравнению с четвертой и более ранними, то, помимо уже упомянутых (поддержка адресов IPv6, поддержка тайм-аутов для всех типов данных, возможность сохранения идентификатора протокола транспортного уровня в типах hash:ip,port, hash:ip,port,ip и hash:ip,port,net), стоит также отметить переход на использование nfnetlink (см. выше) для связи ядра и userspace, а также более простой синтаксис. Например, те действия, которые в ipset 4 выглядели бы так:

```
ipset -N foo macipmap --network 192.168.0.0/16 # Создаем список
ipset -A foo 192.168.1.1,12:34:56:78:9A:BC # Добавляем запись
ipset -T foo 192.168.1.1,12:34:56:78:9A:BC # Проверяем ее наличие в списке
ipset -L foo # Выводим список на экран (точнее, на stdout)
ipset -F foo # Очищаем список
ipset -X foo # Удаляем список
```

в современных версиях ipset будут выглядеть следующим образом:

```
ipset create foo bitmap:ip,mac range 192.168.0.0/16
ipset add foo 192.168.1.1,12:34:56:78:9A:BC
ipset test foo 192.168.1.1,12:34:56:78:9A:BC
ipset list foo
ipset flush foo
ipset destroy foo
```

Кроме того, современные версии ipset поддерживают работу в режиме простой оболочки, принимающей команды со stdin. При этом в них обеспечивается полная обратная совместимость синтаксиса с четвертой версией, то есть синтаксис, корректно работающий для четвертой версии ipset, будет точно так же работать в пятой и шестой.

ipvs

Разработанный в рамках проекта Linux Virtual Server (http://www.linuxvirtualserver.org/) инструментарий для балансировки входящих соединений на транспортном уровне, позволяющий создавать отказоустойчивые (highly available, HA) кластеры с балансировкой

нагрузки. В структуру такого кластера входит один или несколько хостов-балансировщиков (front-end), а также группа хостов-обработчиков (back-end). Хост-балансировщик принимает входящие соединения и перенаправляет их на хосты-обработчики, причем выбор хоста-обработчика осуществляется по специальному алгоритму, обеспечивающему гибкое распределение нагрузки между обработчиками. В кластере также могут присутствовать дополнительные (резервные) балансировщики, которые вступают в игру в случае отказа основного балансировщика.

Как правило, IPVS работает в связке с демоном keepalived, который обеспечивает отказоустойчивость балансировщика (с использованием протокола VRRP) и поддержку в актуальном состоянии списка обработчиков (демон периодически проверяет работоспособность обработчиков и выводит из состава кластера неработоспособные хосты). Также допускается «ручное» управление настройками кластера — через программу ipvsadm.

Как уже упоминалось выше, IPVS обеспечивает балансировку нагрузки на транспортном уровне, т.е. конфигурационной единицей IPVS является *кластерная служба*. Она характеризуется адресом (IPv4 или IPv6), протоколом транспортного уровня (на данный момент IPVS поддерживает протоколы TCP, UDP и SCTP) и номером порта. Обычно для кластерных служб выделяется отдельный адрес, который присваивается активному (на данный момент) балансировщику. В случае его отказа, адрес автоматически передается резервному балансировщику, и кластер продолжает работу. Обратите внимание, что обеспечением отказоустойчивости занимается не IPVS, а keepalived. Впрочем, использование keepalived не является строго обязательным — существуют и альтернативные подходы, например, встраивание IPVS в кластерный стек Linux-HA (http://linux-ha.org/) (см. Heartbeat). В том случае, если отказоустойчивость не критична (требуется только балансировка нагрузки), можно вообще обойтись без дополнительного демона, выполняя всю настройку через ipvsadm.

Для перенаправления пакетов на выбранный обработчик, IPVS может использовать один из следующих методов:

- **Прямая маршрутизация (gatewaying)** — применяется в том случае, если между балансировщиком и обработчиками отсутствуют маршрутизаторы, т.е. они находятся в одном сегменте сети. В этом случае адрес назначения в заголовке пакетов сохраняется неизменным («кластерный» адрес), подменяются лишь MAC-адреса назначения при отправке кадров с этими пакетами.
- **Инкапсуляция IPIP** — модификация предыдущего метода на тот случай, когда балансировщик и обработчики находятся в разных сегментах сети. При помощи инкапсуляции IP-в-IP создаются сетевые туннели, которые объединяют балансировщик и обработчик в один сегмент виртуальной сети.

- **Маскарадинг (NAT)** — наиболее гибкое и универсальное решение: балансировщик подменяет адреса/порты назначения проходящих пакетов на реальные адреса/порты обработчиков (трансляция порт-адрес). При этом обеспечивается как простота настройки (не нужно поднимать дополнительные адреса на обработчиках и туннели между балансировщиком и обработчиком), так и гибкость (целевой порт обработчика может отличаться от порта кластерной службы, что невозможно в рамках других методов). Недостатком данного метода является несколько большее потребление системных ресурсов (трансляция адресов является довольно ресурсоемкой задачей).

В настоящее время существует довольно обширная библиотека алгоритмов балансировки нагрузки между обработчиками:

- **rr (round robin)** — классическая кольцевая схема, когда все обработчики получают новые соединения поочередно.
- **wrr (weighted round robin)** — модификация кольцевой схемы на тот случай, когда одни сервера могут обрабатывать больше соединений, чем другие. Такие сервера включаются в один проход очереди по нескольку раз, и соответственно получают больше соединений.
- **lc (least-connection)** — новое соединение передается тому серверу, который на текущий момент обрабатывает наименьшее число соединений.
- **wlc (weighted least-connection)** — модификация предыдущего метода на случай, когда возможности серверов отличаются. При выборе нового сервера сравниваются не непосредственно количества открытых соединений, а отношения количества соединений к весовому коэффициенту (весу) обработчика (который и характеризует его возможности).
- **lblc (locality-based least-connection)** — формирует привязку обработчиков к кластерному адресу назначения (их может быть несколько). В том случае, если для запрошенного адреса такой привязки еще нет, или привязанный обработчик сильно перегружен — выбирает (по алгоритму lc) и привязывает новый обработчик.
- **lblcr (locality-based least-connection with replication)** — несколько усложненный вариант предыдущего алгоритма. К каждому кластерному адресу привязывается группа обработчиков, которая периодически обновляется (исключаются наиболее нагруженные обработчики, вместо них включаются наименее нагруженные).
- **dh (destination hashing)** — обеспечивает жесткую, неуправляемую привязку обработчиков к кластерным адресам назначения (выбор обработчика производится исключительно на основании элементарных арифметических операций над кластерным адресом и *никак не учитывает* загруженности обработчиков). Может привести к перегрузкам отдельных обработчиков — в этом случае кластер просто перестает обслуживать соответствующий адрес. В такой ситуации нужно либо увеличить лимит соединений на обработчик,

либо выбрать другой алгоритм.
- **sh (source hashing)** — аналогично dh, но обработчики привязываются к адресам клиентов. Как и dh, в случае перегрузки прекращает обслуживание. Отметим, что для формирования *временных* привязок обработчиков к адресам клиентов целесообразнее использовать механизм устойчивых соединений (persistent connections), доступный для всех алгоритмов.
- **sed (shortest expected delay)** — выбирает сервер с наименьшей ожидаемой задержкой ответа. Эта задержка определяется как отношение числа обрабатываемых соединений плюс единица к весу обработчика. Таким образом, этот метод по принципу близок к wlc (отличается лишь прибавлением единицы к числителю сравниваемой величины, что немного меняет влияние веса обработчика на выбор).
- **nq (never queue)** — пытается передать новое соединение обработчику, который в данный момент не обрабатывает других соединений. Если все обработчики уже заняты, использует предыдущий алгоритм (sed).

Для отслеживания соединений к кластерным службам IPVS использует собственную систему трекинга соединений, более примитивную, но менее ресурсоемкую, нежели классический conntrack. При этом поддерживается экспорт информации о кластерных соединениях в conntrack, в целях обеспечения их корректной обработки фаерволом и трансляции адресов (при трансляции адресов используются штатные средства netfilter). Однако, для корректной обработки протоколов, использующих связанные соединения (например, FTP), IPVS использует возможности conntrack. Синхронизация таблицы соединений между основным и дублирующими балансировщиками полностью реализована на уровне ядра (для сравнения, в conntrack эту задачу выполняет userspace-демон conntrackd).

Механизм определения состояний

Важной особенностью iptables/netfilter является механизм определения состояний (connection tracking, nf_conntrack) — специальная подсистема, отслеживающая состояния соединений и позволяющая использовать эту информацию при принятии решений о судьбе отдельных пакетов. Наличие этого механизма делает netfilter полноценным stateful-фаерволом.

Определение состояния соединения порою бывает довольно сложной задачей. Скажем, в случае TCP все относительно просто — контроль состояний реализован средствами самого протокола, установлены специальные процедуры открытия и закрытия соединения. А вот в протоколе UDP для этого специальных процедур не предусмотрено, поэтому UDP-соединением с точки зрения conntrack является поток пакетов, следующих с интервалом, не превышающим заданный тайм-аут (sysctl-параметр `net.netfilter.nf_conntrack_udp_timeout`, по умолчанию 30 секунд), с одного и того же порта одного хоста, на один и тот же порт другого хоста.

Заметим, что классификация пакетов по отношению к соединениям, реализуемая системой conntrack, во многих случаях отличается от официального описания сетевых протоколов. Например, с точки зрения критерия conntrack, TCP-пакет SYN/ACK (отвечающий на SYN) — уже часть существующего сеанса, а по определению TCP такой пакет — всего лишь элемент открытия сеанса.

Существует также понятие «связанных соединений». Например, когда в ответ на UDP-пакет с нашего хоста удаленный хост отвечает ICMP-пакетом icmp-port-unreachable, формально этот ответ является отдельным соединением, так как использует совсем другой протокол. netfilter отслеживает подобные ситуации и присваивает таким соединениям статус «связанных» (RELATED), позволяя корректно пропускать их через фильтры фаервола.

Более сложный вариант связанного соединения — соединение данных в пассивном режиме FTP. FTP-сервер самостоятельно выбирает порт для прослушивания из достаточно большого диапазона, и сообщает номер порта клиенту через управляющее соединение, после чего клиент подключается к этому порту и передает данные. netfilter, а точнее модуль conntrack_ftp, выделяет в потоке данных управляющего соединения нужный номер порта, что позволяет корректно определить новое соединение как связанное и, соответственно, корректно пропустить его через правила фильтрации.

В некоторых случаях целесообразно отключить отслеживание состояния соединений. Например, если ваш сервер находится под (D)DoS-атакой типа флуд, и вам удалось локализовать ее источники, отслеживать соединения с атакующих хостов и тратить для этого ресурсы своей системы явно не имеет смысла. В подобных случаях используется действие NOTRACK, применяемое в таблице raw.

Ниже кратко перечислены возможности, предоставляемые системой отслеживания состояний:

Критерий состояния соединения

При помощи критерия conntrack вы можете классифицировать пакеты на основании их отношения к соединениям. В частности, состояние NEW позволяет выделять только пакеты, открывающие новые соединения, состояние ESTABLISHED — пакеты, принадлежащие к установленным соединениям, состоянию RELATED соответствуют пакеты, открывающие новые соединения, логически связанные с уже установленными (например, соединение данных в пассивном режиме FTP). Состояние INVALID означает, что принадлежность пакета к соединению установить не удалось.

Например, одним простым правилом

```
iptables -I INPUT -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

вы можете обеспечить корректное пропускание всех входящих пакетов, принадлежащих установленным соединениям, и сконцентрироваться только на фильтрации новых соединений.

Заменив в предыдущем правиле `ESTABLISHED` на `ESTABLISHED,RELATED` и подгрузив соответствующие модули ядра, вы автоматически обеспечите корректную фильтрацию протоколов, использующих связанные соединения — FTP, SIP, IRC, H.323 и других. Такое простое (с точки зрения пользователя) решение сложной (с технической точки зрения) проблемы является безусловным достоинством фаервола netfilter и ядра Linux в целом.

Более подробно об использовании этого критерия вы можете почитать ниже.

Кроме критерия conntrack, стоит упомянуть и его идеологического предшественника — критерий state. Изначально для определения состояния соединения использовался именно он, то есть вместо `-m conntrack --ctstate ESTABLISHED,RELATED` использовалось `-m state --state ESTABLISHED,RELATED`. Подобные формулировки до сих пор сохраняются во многих руководствах по iptables. Однако в настоящее время критерий state считается устаревшим, и разработчики iptables рекомендуют (http://jengelh.medozas.de/documents/Perfect_Ruleset.pdf) использовать вместо него критерий conntrack. Также заметим, что критерий conntrack обладает более широкими возможностями, нежели state, и позволяет использовать дополнительную информацию о соединении, в частности, состояние самого соединения (ctstatus), факт применения к нему трансляции адресов, тайм-аут для «повисших» соединений (ctexpire) и т. п.

Маркировка соединений

Этот прием позволяет классифицировать соединение в целом на основании информации об отдельном пакете. Выделив этот пакет, вы применяете к нему действие CONNMARK, и выбранную вами маркировку автоматически приобретают все пакеты в соединении. Впоследствии вы можете, например, модифицировать эти пакеты каким-либо образом, или использовать эту маркировку для маршрутизации или шейпинга пакетов. Таким образом, вы оперируете с соединением как с единым целым. Более того, эта маркировка автоматически копируется и на соединения, связанные с текущим.

Трансляция сетевых адресов

В операциях NAT, производимых с помощью iptables, отслеживание состояний используется автоматически. Вам достаточно указать критерии,

под которые подпадет лишь первый пакет в соединении — и трансляция адресов будет применена ко всем пакетам в этом соединении, а также в связанных с ним соединениях.

Использование статистики по соединениям

Применяя критерий connbytes, вы можете контролировать количество байт или пакетов, переданных по каждому конкретному соединению. В простейшем случае этот механизм может использоваться, скажем, для назначения квот. Более сложный пример — шейпинг пакетов в зависимости от «весовой категории» соединения (ставить пониженный приоритет загрузкам больших файлов).

Этот критерий рассмотрен ниже, в разделе лимитирующие критерии.

Ограничение количества соединений

Используя критерий connlimit, вы можете ограничивать количество одновременно открытых TCP-соединений с каждого хоста или подсети на ваш сервер, что позволяет обеспечить эффективную защиту от DoS-атак или просто некорректно работающего клиентского ПО.

Этот критерий рассмотрен ниже, в разделе лимитирующие критерии.

Отслеживание информации о соединениях

Пользователь (точнее, системный администратор) или его процессы могут непосредственно наблюдать таблицу контроля состояний, считывать статистику по передаче данных, а также модифицировать эту таблицу (например, удалять из нее соединения). Для этого существуют специальные утилиты, такие как conntrack или iptstate. Впрочем, можно читать информацию и напрямую из псевдофайлов `/proc/net/nf_conntrack` и `/proc/net/nf_conntrack_expect`.

Действия

Переходы

Для организации перехода пакета из текущей цепочки в другую (определенную пользователем), просто используйте действие `-j имя_цепочки`. В случае применения в этой цепочке к пакету действия RETURN (см. ниже), пакет вернется в исходную цепочку и продолжит ее прохождение начиная со следующего правила.

Также, существует довольно редко используемое действие «безвозвратного перехода» `-g имя_цепочки`. В таком случае, после прохождения пакетом этой цепочки либо при применении в этой цепочке

к пакету действия RETURN, пакет вернется к месту последнего перехода по `-j`. Если таких переходов не было, к пакету сразу будет применено действие по умолчанию для базовой цепочки.

Заметим, что не стоит пытаться перейти в базовую цепочку (например, `-j OUTPUT`), в текущую цепочку (во избежание зацикливания), а также в цепочку, определенную в другой таблице.

Встроенные действия

Как уже было сказано выше, каждое встроенное действие реализует какую-либо одну операцию, например, ACCEPT пропускает пакет, MARK меняет его маркировку, MASQUERADE обеспечиват маскарадинг соединения. Наиболее общими действиями являются:

- **ACCEPT**, **DROP** и **REJECT** — базовые операции фильтрации. Более подробно они рассмотрены ниже, при описании таблицы filter.
- **RETURN** — обеспечивает возврат из текущей цепочки. В частности, если из цепочки A правилом номер 3 пакет был направлен в цепочку B, то применение к нему в цепочке B действия RETURN приведет к его переходу обратно в цепочку A, и он продолжит ее прохождение со следующего правила (номер 4).

Например, предположим, что нам нужно обеспечить доступ к определенным портам нашего сервера для всех хостов из подсети 10.134.0.64/26, *кроме* 10.134.0.67 и 10.134.0.100.

```
iptables -F # Очищаем все цепочки таблицы filter
iptables -N our_subnet # Создаем специальную цепочку для проверки пакетов из нашей подсети
iptables -A our_subnet -s 10.134.0.67 -j RETURN # Запрещенный хост — выходим
iptables -A our_subnet -s 10.134.0.100 -j RETURN # Запрещенный хост — выходим
# Всем остальным разрешаем доступ к нужным портам
iptables -A our_subnet -p tcp -m multiport --dports 22,53,8080,139,445 -j ACCEPT
iptables -A our_subnet -p udp -m multiport --dports 53,123,137,138 -j ACCEPT
iptables -A our_subnet -p icmp --icmp-type 8 -j ACCEPT
# Разрешаем пакеты по уже установленным соединениям
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# Все пакеты из нашей подсети отправляем на проверку
iptables -A INPUT -s 10.134.0.64/26 -j our_subnet
iptables -P INPUT DROP # Что не разрешено — то запрещено
iptables -P OUTPUT ACCEPT # На выход — можно все
```

Теперь все новые входящие пакеты, отправленные из нашей подсети 10.134.0.64/26, отправляются на проверку в цепочку `our_subnet`. К пакетам с «запрещенных» хостов применяется операция RETURN, и они покидают эту цепочку и впоследствии блокируются действием по умолчанию цепочки INPUT. Пакеты с остальных хостов этой подсети пропускаются в том случае, если они адресованы на порты прокси (8080/tcp), SSH (22/tcp), SMB (139,445/tcp, 137,138/udp), DNS (53/tcp, udp), NTP (123/udp). Также для этих хостов разрешены ICMP-эхо-запросы (пинги). Все остальные

пакеты (включая пакеты не из нашей подсети, пакеты с запрещенных хостов и пакеты на неразрешенные порты) блокируются действием по умолчанию DROP.

Нетрудно заметить, что в нашем простом примере вместо RETURN можно было использовать и DROP. Однако, существует понятие «принципа одного запрета». При организации фильтрующих правил в рамках этого принципа, сначала следует серия разрешающих правил, исключения оформляются в виде RETURN, а все не разрешенные пакеты доходят до конца базовой цепочки и блокируются действием по умолчанию либо последним правилом. Следование этому принципу позволяет достичь гибкости в выборе и смене метода блокирования пакетов. Допустим, вы хотите сменить блокирующее действие DROP на REJECT. Нет ничего проще — просто меняете правило по умолчанию. Так же просто вводятся специфические методы блокировки для TCP, например, REJECT `--reject-with tcp-reset` или DELUDE — достаточно просто добавить это действие (вместе с указанием протокола `-p tcp`) в конец цепочки. Все не-TCP пакеты при этом будут по-прежнему блокироваться действием по умолчанию.

Впрочем, изложенный принцип теряет первоначальный смысл, если в разных случаях нужно обеспечить разные способы блокирования. Рассмотрим чуть более сложный пример: допустим, наш сервер подключен к локальной сети 10.0.0.0/8. В ней есть некий «недоверенный» сегмент, скажем, 10.122.0.0/16, для которого нужно полностью заблокировать доступ к нашему серверу. Но в этом сегменте есть несколько «хороших» хостов (скажем, 10.122.72.11 и 10.122.180.91), которые блокировать не нужно. Поставленную задачу можно решить следующим образом:

```
iptables -F # Очищаем все цепочки таблицы filter
iptables -N check_untrusted # Создаем специальную цепочку для проверки пакетов из нашей подсети
iptables -A check_untrusted -s 10.122.72.11 -j RETURN # Разрешенный хост — выходим
iptables -A check_untrusted -s 10.122.180.91 -j RETURN # Разрешенный хост — выходим
iptables -A check_untrusted -j DROP # Остальных — молча игнорируем
# Разрешаем пакеты по уже установленным соединениям
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# Пакеты из недоверенной подсети проверяем по списку
iptables -A INPUT -s 10.122.0.0/16 -j check_untrusted
# Всем остальным разрешаем доступ к нужным портам
iptables -A INPUT -p tcp -m multiport --dports 22,53,8080,139,445 -j ACCEPT
iptables -A INPUT -p udp -m multiport --dports 53,123,137,138 -j ACCEPT
iptables -A INPUT -p icmp --icmp-type 8 -j ACCEPT
# «Хорошим» хостам, обращающимся на неправильные TCP-порты, вежливо сообщаем об отказе
iptables -A INPUT -p tcp -j REJECT --reject-with tcp-reset
# Для всех остальных протоколов используем стандартный REJECT с icmp-port-unreachable
iptables -P INPUT REJECT
iptables -P OUTPUT ACCEPT # На выход — можно все
```

Заметим, что в данном примере хосты из недоверенного сегмента блокируются «молча» (DROP), в то время как при обращении «хороших» хостов на неправильные порты сервер вежливо сообщает им об отказе (REJECT). Этот принцип также является очень важным при построении фаерволов — чем меньше информации попадает к потенциальному

злоумышленнику, тем лучше, поэтому на опасных направлениях целесообразно использовать «молчаливое» блокирование, в то время как явные сообщения для «своих» упрощают диагностику работы сети и поиск ошибок.

> *Примечание: следующий пример планируется к переносу в еще не написанный раздел статьи (Прочие критерии → mac).*

Например, предположим, что у нас есть объединенная через один свитч подсеть 10.134.0.64/26, к которой наш компьютер подключен через интерфейс eth1. Тогда защиту от спуфинга (подделки адресов отправителя) через проверку MAC-адреса можно обеспечить следующим образом:

```
sysctl net.ipv4.ip_forward=1 # Разрешаем шлюзу передавать транзитный трафик
iptables -F # Очищаем все цепочки таблицы filter
iptables -N check_ours_sp00f # Создаем цепочку, в которой будут проверяться MAC-адреса
iptables -A check_ours_sp00f -s 10.134.0.67 -m mac --mac-source 00:1D:60:2E:ED:A5 -j RETURN
iptables -A check_ours_sp00f -s 10.134.0.68 -m mac --mac-source 00:1D:60:2E:ED:CD -j RETURN
iptables -A check_ours_sp00f -s 10.134.0.69 -m mac --mac-source 00:1D:60:2E:ED:D7 -j RETURN
iptables -A check_ours_sp00f -s 10.134.0.70 -m mac --mac-source 00:1D:60:2E:ED:E0 -j RETURN
# Аналогичным образом проверяем все адреса нашей подсети
# И в конце обязательно добаляем правило для НЕпрошедших проверку
iptables -A check_ours_sp00f -j DROP
iptables -N check_ours # Создаем цепочку, которая будет описывать логику работы с нашей подсетью
iptables -A check_ours ! -i eth1 -j DROP # С других интерфейсов пакеты от них прийти не могут
iptables -A check_ours -j check_ours_sp00f # Прогоняем их через проверку на спуфинг
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Как обычно, пропускаем вс
iptables -A INPUT -s 10.134.0.64/26 -j check_ours # Тех, кто претендует на звание своих, прогоняем
# Прошедшим проверку разрешаем пользоваться нащей проксёй и самбой, а также соединяться по ssh
iptables -A INPUT -s 10.134.0.64/26 -m multiport --dports 22,8080,139,445 -j ACCEPT
iptables -P INPUT DROP # Остальных блокируем
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Как обычно, пропускаем
iptables -A FORWARD -s 10.134.0.64/26 -j check_ours # Проверка
iptables -A FORWARD -s 10.134.0.64/26 -j ACCEPT # Прошедшим проверку разрешаем передавать через на
iptables -P FORWARD DROP # Остальных блокируем
iptables -P OUTPUT ACCEPT # Исходящий трафик разрешаем
```

Все новые входящие пакеты с обратным адресом из подсети 10.134.0.64/26 проходят двойную проверку в цепочке `check_ours`. Первый этап проверки — соответствие интерфейса. Ведь к нашей подсети, по условию задачи, мы подключены только интерфейсом eth1. Если пакет якобы из этой подсети придет с любого другого интерфейса, он будет заблокирован. Второй этап проверки заключается в последовательном чтении списка соответствующих IP- и MAC-адресов (цепочка `check_ours_sp00f`). Каждое правило этой цепочки, кроме последнего, выделяет пакеты от одного конкретного хоста нашей подсети. К пакетам, прошедшим проверку, применяется операция RETURN, выводящая пакет из этой цепочки. Если же пакет не подошел ни по одному из этих правил, он считается не прошедшим проверки и блокируется последним правилом `-j DROP`. Для прошедших же проверку пакетов дальнейшая судьба зависит от того, кому они адресованы. Если они идут через наш хост в другие подсети, то они пропускаются. Если же они адресованы непосредственно нашему хосту, то пропускаются только соединения на некоторые TCP-порты (прокси, SSH,

SMB).

- **LOG** — позволяет записывать информацию о пакетах в журнал ядра (см. syslog).

Например, если в предыдущем примере перед строкой

```
iptables -A INPUT -p tcp -m multiport --dports 22,53,8080,139,445 -j ACCEPT
```

мы добавим строку

```
iptables -A INPUT -p tcp -m multiport --dports 22,53,8080,139,445 -j LOG --log-level INFO --log-pr
```

то для каждого нового соединения к нашему хосту из нашей подсети в системном журнале будет появляться запись примерно такого вида:

```
Jul 16 20:10:40 interdictor kernel: New connection from ours: IN=eth0 OUT= MAC=00:15:17:4c:89:35:00
DST=10.134.0.65 LEN=48 TOS=0x00 PREC=0x00 TTL=112 ID=38914 DF PROTO=TCP SPT=31521 DPT=8080 WINDOW=6
```

Такая запись содержит очень много полезной информации. Начинается она с даты и времени получения пакета. Затем идет имя нашего хоста (interdictor) и источник сообщения (для сообщений фаервола это всегда ядро). Потом идет заданный нами префикс (`New connection from ours:`), после чего следуют данные о самом пакете: входящий интерфейс (определен для цепочек PREROUTING, INPUT и FORWARD), исходящий интерфейс (определен для цепочек FORWARD, OUTPUT и POSTROUTING), далее — сцепленные вместе MAC-адреса источника и назначения (сначала идет адрес назначения, в данном случае это наш интерфейс eth1 с маком 00:15:17:4C:89:35, затем адрес источника, в нашем случае это 00:1D:60:2E:ED:A5, и в конце следует значение EtherType, 08:00 соответствует протоколу IPv4[2]), потом IP-адреса источника (10.134.0.67) и получателя (10.134.0.65, это наш хост), а затем идет различная техническая информация. Например, протокол (TCP), порты источника и назначения (31521 и 8080 соответственно), TOS и TTL, длина пакета (48 байт), наличие флага SYN и т. д.

Указав соответствующие параметры действия LOG, можно дополнить эту информацию номером TCP-последовательности (опция `--log-tcp-sequence`), выводом включенных опций протоколов TCP (опция `--log-tcp-options`) и IP (`--log-ip-options`), а также идентификатором пользователя, процесс которого отправил данный пакет (`--log-uid`, имеет смысл только в цепочках OUTPUT и POSTROUTING).

Параметр `--log-prefix` позволяет задать поясняющую надпись, упрощающую поиск сообщений в системных журналах. Параметр `--log-level` определяет уровень важности лог-сообщения, от которого зависит, в частности, в какой именно из журналов будет записано это сообщение. За более подробными сведениями обратитесь к документации по вашему демону системного лога.

- **LOGMARK** — специальная модификация действия LOG, реализованная в комплекте xtables-addons. Отличается тем, что заносит в лог информацию, специфичную для системы conntrack, в частности, маркировку соединения (connmark aka ctmark), состояния соединения (ctstate и ctstatus) и т. п. Например:

```
Jul 16 20:10:40 interdictor kernel: New connection from ours: iif=1 hook=INPUT nfmark=0x0 secmark=
ct=0xf5436bcc ctmark=0x0 ctstate=NEW ctstatus=
```

`iif` показывает внутренний идентификатор интерфейса, через который прошел пакет (1 в данном случае соответствует eth0, 0 — lo), `hook` — имя цепочки, из которой было вызвано действие LOGMARK, `nfmark` — маркировку пакета (mark), `secmark` — контекст безопасности SELinux данного пакета (secmark), `classify` — класс шейпера (также известный как `skb->priority`, соответствует обычной для tc форме записи *MAJOR*:*MINOR* согласно формуле `skb->priority == MAJOR << 16 | MINOR`), `ctdir` — направление передачи информации с точки зрения conntrack, `ct` — внутренний идентификатор соединения в системе conntrack (точнее говоря, адрес в памяти, по которому расположена структура, хранящая информацию о соединении), `ctmark` — маркировку соединения (connmark), `ctstate` — состояние соединения, `ctstatus` — статус соединения в системе conntrack. Более подробно о параметрах ctdir, ctstate и ctstatus вы можете прочитать ниже, в описании критерия conntrack. Здесь же ограничимся замечаниями, что «`ctstate=NEW ctstatus=`» соответствует первому пакету в соединении, «`ctstate=ESTABLISHED ctstatus=SEEN_REPLY,CONFIRMED`» — второму и нескольким последующим пакетам, «`ctstate=ESTABLISHED ctstatus=SEEN_REPLY,ASSURED,CONFIRMED`» — пакетам в полностью установленном соединении, «`ct=NULL ctmark=NULL ctstate=INVALID ctstatus=NONE`» — пакету, который не удалось отнести к существующим соединениям, «`ct=UNTRACKED ctmark=NULL ctstate=UNTRACKED ctstatus=NONE`» — пакету, для которого была отключена трассировка conntrack (обычно это выполняется действием NOTRACK в таблице raw, см. ниже).

Таким образом, в сочетании с традиционным LOG, действие LOGMARK позволяет заносить в системный журнал исчерпывающую информацию о пакетах и соединениях.

Заметим, что LOGMARK принимает описанные выше опции `--log-prefix` и `--log-level`.

- **ULOG** — позволяет передавать информацию об обработанных пакетах специальным демонам, таким, как ulogd (http://www.netfilter.org /projects/ulogd/). Такой подход позволяет эффективно управлять информацией о трафике, в частности, заносить ее в базы данных, такие как MySQL, PostgreSQL или SQLite. Впоследствии эти данные могут быть проанализированы и визуализированы с помощью таких средств, как NuLog (http://software.inl.fr/trac/wiki/EdenWall/NuLog).
- **NFLOG** — более универсальный вариант ULOG, обеспечивающий передачу информации о пакете не напрямую в netlink-сокет (как это делает ULOG), а специальной подсистеме — logging backend. Например, бэкенд nfnetlink_log обеспечивает передачу данных в netlink-сокет, то есть с ним NFLOG работает аналогично ULOG.
- **NFQUEUE** — во многом похоже на ULOG, но передает специальному демону не информацию о пакете, а сам пакет целиком. Применяется, в частности, для организации работы l7-filter-userspace.

Например, если демон l7-filter был запущен командой

```
l7-filter -f /etc/l7-filter.conf -q 2
```

то передать ему на обработку все входящие и локально сгенерированные пакеты можно командами

```
iptables -t mangle -F # На всякий случай очищаем таблицу mangle
iptables -t mangle -A PREROUTING -j NFQUEUE --queue-num 2
iptables -t mangle -A OUTPUT -j NFQUEUE --queue-num 2
```

Опция `--queue-num` позволяет указать номер очереди пакетов. Он должен быть совпадать с номером, указанным при запуске демона l7-filter (параметр `-q`).

- **QUEUE** — устаревшая версия NFQUEUE. Не имеет параметров, так как работает только с очередью номер 0.

Выше были рассмотрены действия «общего назначения», то есть не привязанные по своей специфике к таблицам. Далее, при рассмотрении отдельных таблиц, будут описываться действия, специфичные для каждой таблицы. Термин «специфичные» надо понимать так: совсем не обязательно, что действие, специфичное для одной таблицы, будет в принципе недопустимо в другой, но в любом случае использование этого действия в других таблицах будет «плохим тоном» — ведь таблицы и действия четко разделены по назначению. Не стоит «забивать гвозди микроскопом».

Порой эта разница бывает довольно тонкой. Например, если действие NFQUEUE передает пакеты демону l7-filter-userspace (который занимается маркировкой пакетов на основе анализа их содержимого), то правильнее будет разместить вызов такого действия в таблице mangle. Если же оно передает пакеты демону nufw (который обеспечивает разрешение или запрещение трафика на основе авторизации пользователя), то логичнее разместить это действие в таблице filter.

Кроме того, существуют и ограничения, наложенные непосредственно в коде iptables. Например, действия таблицы nat жестко защищены от употребления за ее пределами. Другой пример: действие REDIRECT можно применять только в цепочках PREROUTING и OUTPUT таблицы nat, то есть ограничения могут быть не только на таблицы, но на отдельные цепочки. Данная статья ни в коем случае не претендует на полноту изложения, поэтому, перед тем как использовать какое-либо действие, обязательно ознакомьтесь с документацией, прилагающейся к вашей версии iptables.

Терминальные и нетерминальные действия

- *Терминальными* называются действия, которые прерывают прохождение пакета через текущую базовую цепочку. То есть если к пакету в рамках некоторого правила было применено терминальное действие, он уже не проверяется на соответствие всем следующим правилам в этой цепочке (и в тех цепочках, из которых она была вызвана, если это пользовательская цепочка). Терминальными являются все действия, специфичные для таблиц filter и nat. Из приведенных выше в этом разделе — ACCEPT, DROP, REJECT, NFQUEUE, QUEUE.

Например,

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j LOG --log-prefix "IN HTTP: "
```

Второе из приведенных правил (логгирование) никогда не будет срабатывать, так как все пакеты, удовлетворяющие этому критерию (входящие TCP-пакеты на порт 80), будут пропущены по первому из этих правил, и до второго просто не дойдут.

- *Нетерминальными*, соответственно, являются действия, *не* прерывающие процесс прохождения пакета через цепочки. Нетерминальными являются действия, специфичные для таблицы mangle, а из перечисленных выше — LOG, ULOG и NFLOG.

Например,

```
```

```
iptables -A INPUT -p tcp --dport 80 -j LOG --log-prefix "IN HTTP: "
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

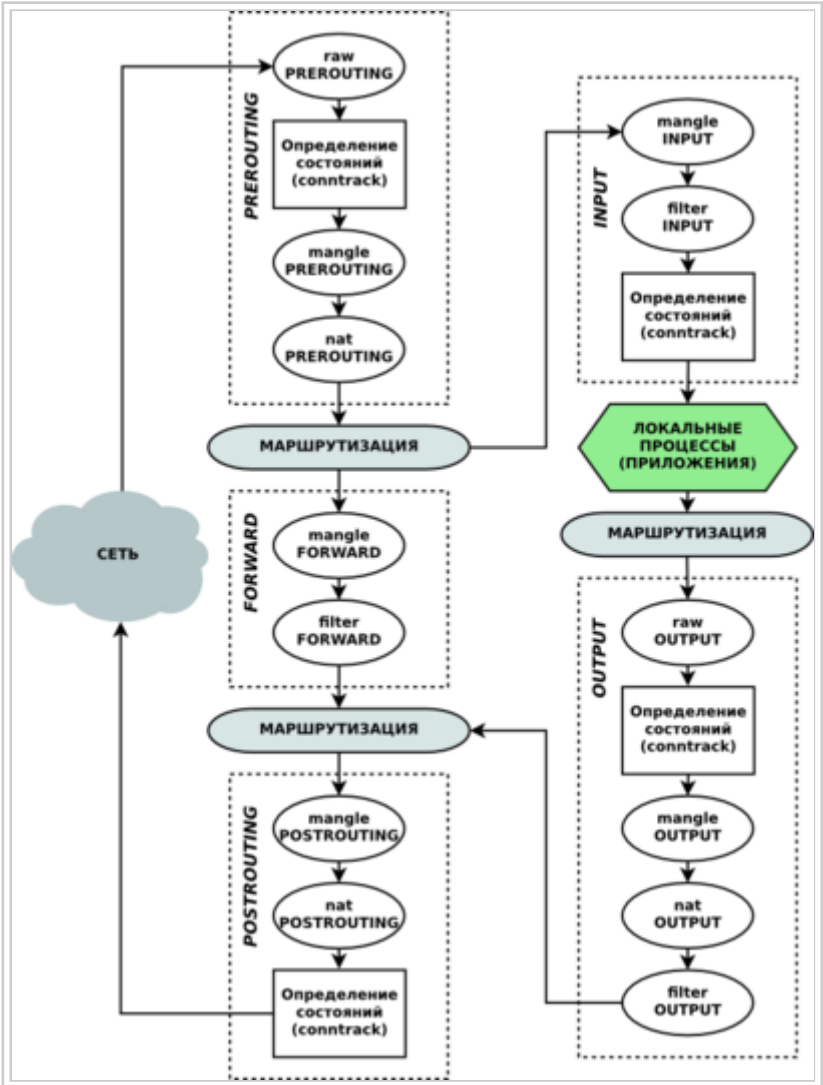
Теперь все входящие на TCP-порт 80 пакеты будут сначала заноситься в лог, и только потом пропускаться.

Действие RETURN с точки зрения такой классификации занимает промежуточное положение — оно может прервать прохождение пакета через отдельную цепочку (или несколько цепочек, если для перехода в них использовалась опция `-g`, а не `-j`), но совсем не обязательно, чтобы это приводило к завершению обработки пакета в рамках текущей базовой цепочки.

Таблицы

Таблица mangle

Данная таблица предназначена для операций по классификации и маркировке пакетов и соединений, а также модификации заголовков пакетов (поля TTL и TOS).



Путь проверки пакета в системе netfilter

Цепочки

Таблица mangle содержит следующие цепочки:

- **PREROUTING** — позволяет модифицировать пакет *до* принятия решения о маршрутизации.
- **INPUT** — позволяет модифицировать пакет, предназначенный самому хосту.
- **FORWARD** — цепочка,

позволяющая модифицировать транзитные пакеты.
- **OUTPUT** — позволяет модифицировать пакеты, исходящие от самого хоста.
- **POSTROUTING** — дает возможность модифицировать все исходящие пакеты, как сгенерированные самим хостом, так и транзитные.

Заметим, что все цепочки таблицы mangle пакеты проходят раньше, чем одноименные цепочки таблиц nat и filter. Это позволяет классифицировать и маркировать пакеты и соединения в цепочках этой таблицы. Впоследствии эти маркировки могут быть использованы в цепочках двух других таблиц при принятии решений о фильтрации и трансляции адресов.

Действия

Допустимыми действиями в этой таблице являются:

- **TOS** — изменяет поле TOS данного пакета. Поддерживаются опции `--set-tos` (установить поле TOS в заданное значение), а также `--and-tos`, `--or-tos` и `--xor-tos`, комбинирующие текущее значение поля TOS с заданным в правиле значением по соответствующему логическому правилу и записывающие результат как новое значения поля TOS.

Кроме того, опция `--set-tos` поддерживает расширенный синтаксис `--set-tos` *`значение/маска`*. При использовании такого синтаксиса в исходном значении TOS пакета зануляются те биты, которые установлены в маске, затем полученное число XOR’ится с указанным в параметрах значением, и полученная величина записывается в поле TOS. Используя синтаксис языка C, это можно записать так: `NEW_TOS = (OLD_TOS & ~`*`маска`*`) ^ `*`значение`*. Заметим, что в случае с IPv6 операция отрицания маски не выполняется (то есть в исходном значении зануляются те биты, которые в маске *не установлены*), хотя это и не отражено в документации.

Также, в случае IPv4 (iptables) в параметре `--set-tos` вы можете указать одно из допустимых символьных обозначений: Minimize-Delay (TOS 16, требование минимальной задержки), Maximize-Throughput (TOS 8, требование максимальной пропускной способности), Maximize-Reliability (TOS 4, максимальная надежность доставки), Minimize-Cost (TOS 2, минимальная стоимость), Normal-Service (TOS 0, специальные требования отсутствуют). В ip6tables использование этих обозначений не запрещено, однако в этом случае более корректным будет использование действия DSCP (см. ниже).

Практический пример использования действия TOS представлен ниже, в описании критерия connbytes.

Отметим, что, в соответствии с современными соглашениями, поле TOS в заголовке IP-пакета разделяется на две части: DSCP (первые шесть бит кода TOS) и ECN (последние два бита TOS). Современные версии netfilter/iptables не поддерживают изменение значений ECN в

IP-заголовках, поэтому фактически действие TOS работает только с полем DSCP, отличаясь от действия DSCP (см. чуть ниже) разве что интерфейсом (фактически, синтаксисом).

- **DSCP** — изменяет поле DSCP (класс DiffServ) в заголовке пакета. Поддерживаются опции `--set-dscp` (позволяет задать значение DSCP числом) и `--set-dscp-class` (позволяет установить заданный класс DiffServ, например, EF или AF13).

- **TTL** — изменяет поле TTL данного пакета (работает только с IPv4, для IPv6 используется действие HL). Поддерживаются опции `--ttl-set` (установить поле TTL в заданное значение), а также `--ttl-inc` и `--ttl-dec` (соответственно увеличить или уменьшить текущее значение поля TTL на заданное значение). Допустимые значения TTL — от 0 до 255. При достижении TTL=0 пакет уничтожается.

В качестве полезного примера использования этого действия можно привести команду

```
iptables -t mangle -I PREROUTING -j TTL --ttl-inc 1
```

делающую наш шлюз невидимым для большинства трассировщиков.

Обратите внимание, что автоматически выполняемая сетевым стеком ядра операция уменьшения TTL на единицу и проверки на равенство нулю выполняется *после* цепочки PREROUTING, но *до* цепочки FORWARD. Таким образом, переместив это правило в цепочку FORWARD, вы обеспечите «невидимость» следующего за вами шлюза.

Другой полезный пример — выравнивание TTL на выходе в Интернет (интерфейс eth0)

```
iptables -t mangle -I POSTROUTING -o eth0 -j TTL --ttl-set 64
```

Некоторые домашние провайдеры, пытаясь бороться с нелегальной перепродажей канала их клиентами, блокируют доступ тем клиентам, на выходе которых замечены пакеты с разными значениями TTL, потому что этот факт косвенно указывает на наличие у клиента внутренней подсети. Предложенное правило, будучи применено на шлюзе такой подсети, делает ее детекцию по TTL крайне затруднительной.

Будьте очень осторожны с правилами, которые увеличивают или, что особенно опасно, напрямую задают значение TTL, так как, при наличии определенных ошибок маршрутизации, это может привести к появлению

«бессмертных» пакетов, которые будут долго циркулировать между одними и теми же узлами, захламляя канал и затрудняя работу сетевого оборудования.

- **HL** — изменяет поле Hop Limit в заголовке IPv6-пакета. Является аналогом IPv4-действия TTL и поддерживает те же операции: `--hl-set` (установить поле HL в заданное значение), а также `--hl-inc` и `--hl-dec` (соответственно увеличить или уменьшить текущее значение поля HL на заданное значение).

- **MARK** — устанавливает или изменяет маркировку пакета. Поддерживает опции `--set-mark`, `--and-mark`, `--or-mark` и `--xor-mark`, аналогичные опциям действия TOS. Важно понимать, что маркировка пакета не хранится в его заголовке или содержимом, и поэтому действует только в пределах одного хоста. Также нужно отличать маркировку пакета от маркировки соединения. Маркировка пакетов может применяться для их дальнейшей обработки фаерволом (критерии mark и connmark), а также для классификации трафика фильтрами шейпинговой подсистемы tc (лексема handle *mark* fw).
- **CONNMARK** — устанавливает или изменяет маркировку соединения. Поддерживает те же опции, что и MARK, а также дополнительные опции `--restore-mark` (копирует маркировку соединения в маркировку пакета) и `--save-mark` (копирует маркировку пакета в маркировку соединения).
- **CLASSIFY** — устанавливает CBQ-класс пакета для его последующей обработки шейпером (опция `--set-class`).
- **TCPMSS** — устанавливает максимальный размер TCP-сегмента. Бывает крайне полезной при использовании VPN-подключения[3] в том случае, если VPN-сервер блокирует ICMP-сообщения destination unreachable/fragmentation needed (тип 3, код 4), тем самым нарушая работу процедуры Path MTU discovery.

С точки зрения клиента, эта проблема выглядит так: пинги проходят нормально, но при попытке открыть какую-либо веб-страницу, браузер «подвисает». При этом с самого шлюза все работает нормально. В этом случае достаточно применить на шлюзе следующую команду

```
iptables -t mangle -I FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu
```

которая обеспечит автоматическую установку размера сегмента в TCP-заголовках SYN- и SYN,ACK-пакетов в соответствии с минимальным из известных нашему шлюзу значений MTU на пути следования пакета. Например, если пакет пришел с интерфейса eth0 (MTU 1500) и уходит через интерфейс ppp0 (MTU 1492), а суммарный размер заголовков сетевого и транспортного уровней составляет 40 байт (20 байт TCP и 20 байт IP), то целесообразно установить MSS равным 1452 байтам.

Помимо возможности автоматического выбора TCP MSS, вы можете задать необходимое значение и вручную, используя параметр `--set-mss` *значение*. Это бывает полезным в том случае, если вам заведомо известно, что дальше на маршруте встречаются участки с еще меньшим MTU, а пограничные сервера этих участков опять же блокируют ICMP destination unreachable/fragmentation needed.

- **ECN** — обеспечивает обнуление ECN-битов (флаги CWR и ECE) в TCP-заголовке (единственная опция `--ecn-tcp-remove`). Может использоваться только в IPv4-модуле (iptables, но не ip6tables) для TCP-пакетов (`-p tcp`). Данное действие предназначено для защиты пакетов от ECN blackholes (маршрутизаторов, которые некорректно обрабатывают пакеты с установленными в TCP-заголовке ECN-битами) — рекомендуется применять это действие ко всем пакетам, уходящим на такие маршрутизаторы. Обратите внимание, что данное действие никак не влияет на ECN-биты в IP-заголовках (последние два бита поля TOS).

Кроме того, данное действие поддерживает три потенциально опасные опции, не отраженные в документации: `--ecn-tcp-cwr` (установка значения бита CWR в TCP-заголовке, 0 или 1), `--ecn-tcp-ece` (установка значения бита ECE в TCP-заголовке, 0 или 1) и `--ecn-ip-ect` (установка значения ECT codepoint в IPv4-заголовке, от 0 до 3). Использование этих опций настолько опасно, что они не отражены даже во встроенной справке iptables (`iptables -j ECN -h`) — их можно увидеть, только изучив исходный код (http://git.netfilter.org/cgi-bin/gitweb.cgi?p=iptables.git;a=blob;f=extensions/libipt_ECN.c). Без крайней необходимости применять их не рекомендуется.

- **TCPOPTSTRIP** — выполняет удаление заданных TCP-опций из заголовка TCP-пакета (единственный параметр `--strip-options` *значение*`[,`*значение*`[,...]]`). Удаляемые опции могут быть указаны через их номера (согласно списку на сайте IANA (http://www.iana.org/assignments/tcp-parameters/tcp-parameters.xhtml)) или в виде символьных обозначений (список обозначений, поддерживаемых в вашей версии iptables можно посмотреть, выполнив команду `iptables -j TCPOPTSTRIP -h`). Разумеется, данное действие допустимо только для протокола TCP (`-p tcp`). Например,

```
iptables -t mangle -A POSTROUTING -p tcp -j TCPOPTSTRIP --strip-options timestamp
```

обеспечит удаление штампов времени (RFC 1323 (http://tools.ietf.org/html/rfc1323)). С одной стороны, такое правило будет препятствовать удаленному злоумышленнику определить аптайм нашего хоста, а также тех хостов, маршрут от которых до злоумышленника проходит через наш хост. С другой стороны, блокирование штампов времени может негативно

сказаться на быстродействии сети, особенно если вы используете подключения на скорости 100 МБит и выше. Заметим также, что если вам нужно управлять использованием именно штампов времени для TCP IPv4-пакетов, исходящих от вашего хоста, вы можете воспользоваться sysctl-параметром net.ipv4.tcp_timestamps (1 — штампы включены, 0 — выключены).

- **TPROXY** — реализует механизм полностью прозрачного проксирования. Такой подход отличается от традиционно используемого «прозрачного» проксирования (действие REDIRECT таблицы nat, см. ниже) тем, что заголовок пакета никак не модифицируется, в том числе не заменяется IP-адрес назначения (при традиционном прозрачном проксировании он заменяется на адрес проксирующего хоста). Кроме того, полностью прозрачное проксирование является прозрачным с точки зрения обеих общающихся сторон. Например, при проксировании обращений некоторой подсети клиентов к серверам из другой подсети, можно сделать так, чтобы не только клиенты считали, что обращаются напрямую к серверам, но и сервера «видели» настоящие исходные адреса клиентов и могли бы устанавливать с ними обратные соединения (например, в случае активного режима FTP). При традиционном же «прозрачном» проксировании, сервера могут видеть только адрес прокси-сервера.

TPROXY позволяет перенаправить транзитный пакет на локальный сокет типа AF_INET (iptables) или AF_INET6 (ip6tables), заданный портом, а также может промаркировать пакет таким образом, чтобы система марушрутизации (iproute2) не позволила пакету покинуть наш хост. Разумеется, поддержка полностью прозрачного проксирования должна быть реализована и в самом прокси-сервере. В частности, прозрачное проксирование IPv4 HTTP поддерживается (http://wiki.squid-cache.org /Features/Tproxy4) прокси-сервером Squid начиная с версии 3.1.

В качестве примера можно привести простейший случай конфигурации iptables и iproute2 для обеспечения полностью прозрачного проксирования IPv4 HTTP.

```
# Добавляем принудительную локальную маршрутизацию для всех адресов в таблицу 120
ip route add local 0.0.0.0/0 dev lo table 120
# Все пакеты, имеющие соответствующую маркировку, направляем в эту таблицу
ip rule add fwmark 120 table 120
# Выполняем проксирование для новых соединений
iptables -t mangle -I PREROUTING -p tcp --dport 80 -j TPROXY --tproxy-mark 120 --on-port 3129
# Перехватываем, маркируем и пропускаем пакеты, принадлежащие проксируемым соединениям
iptables -t mangle -N proxypackets
iptables -t mangle -A proxypackets -j MARK --set-mark 120
iptables -t mangle -A proxypackets -j ACCEPT
# Такие пакеты можно выделять с помощью критерия socket
iptables -t mangle -I PREROUTING -m socket -j proxypackets
```

Таким образом, пакеты, перехваченные TPROXY-правилом, маркируются и направляются на локальный сокет (порт 3129), который должен прослушиваться специально сконфигурированным прокси-сервером (в случае Squid, в файл конфигурации `squid.conf` нужно добавить строку `http_port 3129 tproxy`). Все пакеты, принадлежащие уже проксируемым соединениям, перехватываются правилом с критерием socket (этот критерий позволяет определить, существует ли на нашем хосте сокет, связанный с данным пакетом), маркируются и немедленно пропускаются. Отметим, что последнее правило будет обрабатываться раньше, чем правило с TPROXY, так как параметр `-I` обеспечивает вставку правил *в начало* цепочки, и при последовательном добавлении нескольких правил с этим параметром, порядок добавленных правил будет обратным порядку их ввода.

Таблица nat

Предназначена для операций stateful-преобразования сетевых адресов и портов обрабатываемых пакетов.

Цепочки

Таблица nat содержит следующие цепочки:

- **PREROUTING** — в эту цепочку пакеты попадают *до* принятия решения о маршрутизации. По сути, термин «решение о маршрутизации» подразумевает деление трафика на входящий (предназначенный самому хосту) и транзитный (идущий через этот хост на другие хосты). Именно на данном этапе нужно проводить операции проброса (DNAT, REDIRECT, NETMAP).
- **OUTPUT** — через эту цепочку проходят пакеты, сгенерированные процессами самого хоста. На данном этапе при необходимости можно повторить операции проброса, так локально сгенерированные пакеты *не* проходят цепочку PREROUTING и *не* обрабатываются ее правилами. См. пример для действия DNAT ниже.
- **POSTROUTING** — через эту цепочку проходят все исходящие пакеты, поэтому именно в ней целесообразно проводить операции маскарадинга (SNAT и MASQUERADE).

Действия

Допустимыми действиями в таблице nat являются:

- **MASQUERADE** — подменяет адрес источника для исходящих пакетов адресом того интерфейса, с которого они исходят, то есть осуществляет маскарадинг. Такая операция позволяет, например, предоставлять доступ в Интернет целым локальным сетям через один шлюз.

Допустим, у нас есть локальная сеть 192.168.1.0/255.255.255.0 с несколькими компьютерами, имеющими адреса 192.168.1.2, 192.168.1.3 и т. д. Адрес 192.168.1.1 имеет внутренний (подключенный к локальной сети) сетевой интерфейс шлюза, назовем этот интерфейс eth1. Другой его интерфейс, назовем его eth0, подключен к сети Интернет и имеет адрес, допустим, 208.77.188.166. Тогда, чтобы обеспечить выход хостов из этой локальной сети в Интернет, на шлюзе достаточно выполнить следующие команды

```
sysctl net.ipv4.ip_forward=1 # Разрешаем шлюзу передавать транзитный трафик
iptables -F FORWARD # На всякий случай очистим цепочку FORWARD
# Разрешаем проходить пакетам по уже установленным соединениям
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# Разрешаем исходящие соединения из локальной сети к интернет-хостам
iptables -A FORWARD -m conntrack --ctstate NEW -i eth1 -s 192.168.1.0/24 -j ACCEPT
iptables -P FORWARD DROP # Весь остальной транзитный трафик — запрещаем.
iptables -t nat -F POSTROUTING # На всякий случай очистим цепочку POSTROUTING таблицы nat
# Маскарадим весь трафик, идущий через eth0
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Теперь, если один из хостов локальной сети, например, 192.168.1.2, попытается связаться с одним из интернет-хостов, например, 204.152.191.37 (kernel.org), при проходе его пакетов через шлюз, их исходный адрес будет подменяться на внешний адрес шлюза, то есть 208.77.188.166. С точки зрения удаленного хоста (kernel.org), это будет выглядеть, как будто с ним связывается непосредственно сам шлюз. Когда же удаленный хост начнет ответную передачу данных, он будет адресовать их именно шлюзу, то есть 208.77.188.166. Однако, на шлюзе адрес назначения этих пакетов будет подменяться на 192.168.1.2, после чего пакеты будут передаваться настоящему получателю. Для такого обратного преобразования никаких дополнительных правил указывать не нужно — это будет делать все та же операция MASQUERADE. Простота трансляции сетевых адресов является одним из важнейших достоинств stateful-фильтрации.

Если же такой трансляции не производить, удаленный хост просто не сможет ответить на адрес 192.168.1.2, так как адресные пространства локальных сетей изолировано от адресного пространства Интернета. В мире могут существовать миллионы локальных сетей 192.168.1.0/255.255.255.0, и в каждой может быть свой хост 192.168.1.2. Эти сети могут и не быть связаны с Интернетом. Но если они с ним связаны — то только благодаря механизмам трансляции сетевых адресов.

- **SNAT** (Source Network Address Translation) — работает аналогично MASQUERADE, однако позволяет указать адрес «внешнего» интерфейса (опция `--to-source`). Такой подход позволяет экономить процессорное время шлюза, так как в случае с MASQUERADE для каждого пакета адрес внешнего интерфейса определяется заново. Таким образом, если в предыдущем примере внешний адрес шлюза

208.77.188.166 является статическим (то есть никогда не меняется), команду

```
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

целесообразно заменить на

```
iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to-source 208.77.188.166
```

Однако, в случае, если внешний адрес шлюза динамический, то есть меняется в начале каждой новой сессии, правильнее будет использовать именно MASQUERADE.

Дополнительно, если на внешнем интерфейсе шлюза «висит» несколько статических адресов, например, 208.77.188.166, 208.77.188.167 и 208.77.188.168, можно использовать балансировку между этими адресами:

```
iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to-source 208.77.188.166-208.77.188.168
```

Теперь для каждого нового соединения адрес для подмены будет выбираться случайным образом.

- **DNAT** (Destination Network Address Translation) — подменяет адрес назначения для входящих пакетов, позволяя «пробрасывать» адреса или отдельные порты внутрь локальной сети.

Возвращаясь к предыдущему примеру, предположим, что нам нужно «пробросить» внешний адрес шлюза 208.77.188.166 на хост 192.168.1.2, то есть все, кто будет обращаться по внешнему адресу шлюза 208.77.188.166, должны попадать на хост 192.168.1.2. Это достигается следующими командами:

```
# Разрешаем шлюзу передавать транзитный трафик
sysctl net.ipv4.ip_forward=1
iptables -t nat -F PREROUTING # На всякий случай очистим цепочку PREROUTING таблицы nat
# Пробрасываем 208.77.188.166 на 192.168.1.2
iptables -t nat -A PREROUTING -d 208.77.188.166 -j DNAT --to-destination 192.168.1.2
iptables -F FORWARD # На всякий случай очистим цепочку FORWARD
# Разрешаем проходить пакетам по уже установленным соединениям
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# Разрешаем входящие соединения к 192.168.1.2
iptables -A FORWARD -m conntrack --ctstate NEW -d 192.168.1.2 -j ACCEPT
iptables -P FORWARD DROP # Весь остальной транзитный трафик – запрещаем.
```

В лучших традициях stateful-фаерволинга, подменяются не только адреса назначения для входящих пакетов (208.77.188.166 -> 192.168.1.2), но и адреса источника для исходящих пакетов (192.168.1.2 -> 208.77.188.166), поэтому для интернет-хостов создается полное впечатление, что они общаются именно с 208.77.188.166.

Однако, при всей своей простоте и наглядности, этот пример имеет два недостатка. Во-первых, из нашей локальной сети 192.168.1.0/24 адрес 208.77.188.166 может быть недоступен, так как 192.168.1.2, увидев, что запрос исходит из его сети, может отправить ответ непосредственно отправителю (например, 192.168.1.3), минуя сам шлюз. Отправитель же будет ждать ответа от 208.77.188.166, и просто проигнорирует ответ от 192.168.1.2. Впрочем, эта проблема легко исправляется добавлением SNAT'а для пробрасываемых пакетов:

```
iptables -t nat -A POSTROUTING -d 192.168.1.2 -s 192.168.1.0/24 -j SNAT --to-source 192.168.1.1
```

Теперь, с точки зрения 192.168.1.2, к нему будет обращаться только сам шлюз (192.168.1.1), поэтому ответ также пойдет через шлюз, где будет корректно оттранслирован. Такое решение тоже имеет недостаток — подмена исходного адреса пакета приводит к тому, что компьютер, на который мы пробрасываем соединение, не может определить реального инициатора соединения. Чтобы уменьшить «вредность» этого правила, в него добавлена опция «-s 192.168.1.0/24», которая ограничивает применение этого правила только пакетами из локальной сети. Таким образом, соединения из локальной сети будут выглядеть как соединения, инициированные шлюзом, а соединения из внешней сети будут иметь свои оригинальные адреса.

Вторым недостатком предложенного примера является некорректная обработка запросов с самого шлюза. Ведь пакеты, генерируемые локальными процессами, не проходят цепочку PREROUTING. Поэтому правило DNAT нужно также добавить и в цепочку OUTPUT таблицы nat:

```
iptables -t nat -A OUTPUT -d 208.77.188.166 -j DNAT --to-destination 192.168.1.2
```

Разумеется, исходящие соединения к 192.168.1.2 и ответы на них должны быть разрешены соответственно в цепочках OUTPUT и INPUT таблицы filter. Самый простой способ сделать это —

```
iptables -F INPUT # Очищаем цепочку INPUT
```

```
# Разрешаем входящие пакеты по уже установленным соединениям
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -P INPUT DROP # Все остальные входящие пакеты — запрещаем
iptables -F OUTPUT # Очищаем цепочку OUTPUT
iptables -P OUTPUT ACCEPT # Разрешаем исходящие пакеты безо всяких ограничений
```

Кроме того, при помощи действия DNAT можно пробрасывать не только сразу весь IP-адрес, но и отдельные TCP- или UDP-порты. Например, если в предыдущем примере мы в каждое из правил

```
iptables -t nat -A PREROUTING -d 208.77.188.166 -j DNAT --to-destination 192.168.1.2
iptables -t nat -A POSTROUTING -d 192.168.1.2 -s 192.168.1.0/24 -j SNAT --to-source 192.168.1.1
iptables -t nat -A OUTPUT -d 208.77.188.166 -j DNAT --to-destination 192.168.1.2
```

добавим

```
-p tcp --dport 80
```

то мы «пробросим» не сам адрес 208.77.188.166, а только его TCP-порт 80 (HTTP). Обращения на все остальные TCP- и UDP-порты, а также ICMP-пакеты, поступившие на адрес 208.77.188.166, будут обрабатываться шлюзом, и лишь входящие TCP-соединения на порт 80 будут передаваться на 192.168.1.2.

Как и SNAT, DNAT поддерживает указание диапазонов адресов для подмены. Таким образом можно, например, балансировать нагрузку между несколькими серверами.

- **REDIRECT** — подменяет номер порта в TCP- или UDP-пакете, а также подменяет адрес назначения на свой собственный. Например,

```
iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 80 -j REDIRECT --to-port 8080
```

позволяет «завернуть» все исходящие из локальной сети TCP-соединения на порт 80, на TCP-порт 8080 шлюза. Если этот порт обслуживается специально настроенным прокси-сервером, то можно организовать «прозрачное проксирование» — клиенты из локальной сети даже не будут подозревать, что их запросы идут через прокси-сервер. Разумеется, входящие соединения из локалки на TCP-порт 8080 должны быть разрешены в таблице filter.

Другой полезный пример:

```
iptables -t nat -A PREROUTING -i eth0 -s 213.180.0.0/16 \
    -p tcp --dport 80 -j REDIRECT --to-port 8000
```

«заворачивает» все входящие из Интернета, а точнее подсети 213.180.0.0/255.255.0.0 TCP-соединения на порт 80, переадресуя их на порт 8000 того же сервера (где, например, может работать специальный веб-сервер). Как обычно, входящие соединения на TCP-порт 8000 должны быть разрешены в таблице filter.

- **SAME** — в зависимости от цепочки (PREROUTING или POSTROUTING) может работать как DNAT или SNAT. Однако, при указании (в параметре `--to-ip`) одного или нескольких диапазонов IP-адресов, определяет для каждого нового соединения подставляемый адрес не случайно, а базируясь на IP-адресе клиента. Таким образом, адрес для подмены остается постоянным для одного и того же клиента при повторных соединениях (что *не* выполняется для обычных DNAT/SNAT). В некоторых случаях это бывает важным.
- **NETMAP** — позволяет «пробросить» целую сеть. Например, для шлюза, стоящего между сетями 192.168.1.0/24 и 192.168.2.0/24 можно организовать следующий проброс:

```
iptables -t nat -A PREROUTING -s 192.168.1.0/24 -d 192.168.3.0/24 -j NETMAP --to 192.168.2.2/24
```

Теперь, при обращении, например, хоста 192.168.1.3 на IP-адрес 192.168.3.2 он будет попадать на 192.168.2.2 (при условии, что такой транзитный трафик разрешен на шлюзе в цепочке FORWARD таблицы filter, а также на хостах сети 192.168.1.0/24 наш шлюз прописан в качестве шлюза для подсети 192.168.3.0/24).

> *Примечание: следующее действие планируется к переносу в еще не написанный раздел статьи (Устаревшие критерии и действия).*

- **MIRROR** — довольно интересная игрушка. Меняет местами адрес источника и назначения и высылает пакет обратно. Создано исключительно для демонстрационных целей. Однако, может, например, применяться для защиты от сканирования портов — в результате атакующий сканирует свои собственные порты. Однако наличие двух встречных MIRROR'ов на двух хостах может повлечь бесконечное блуждание одних и тех же пакетов, забивающее канал. Поэтому применяйте это действие с осторожностью.

Таблица filter

Предназначена для фильтрации трафика, то есть разрешения и запрещения пакетов и соединений.

Цепочки

Таблица filter содержит следующие цепочки:

- **INPUT** — эта цепочка обрабатывает трафик, поступающий непосредственно самому хосту.
- **FORWARD** — позволяет фильтровать транзитный трафик.
- **OUTPUT** — эта цепочка позволяет фильтровать трафик, исходящий от самого хоста.

Действия

Допустимыми действиями в таблице filter являются:

- **ACCEPT** — пропуск пакета. Пакет покидает текущую базовую цепочку и следует дальше по потоковой диаграмме (см. рис.).
- **REJECT** — заблокировать пакет и сообщить его источнику об отказе. По умолчанию об отказе сообщается отправкой ответного ICMP-пакета «icmp-port-unreachable». Однако, это действие поддерживает опцию `--reject-with`, позволяющую указать формулировку сообщения об отказе (возможные значения: `icmp-net-unreachable`, `icmp-host-unreachable`, `icmp-proto-unreachable`, `icmp-net-prohibited`, `icmp-host-prohibited`). Для протокола TCP поддерживается отказ в форме отправки RST-пакета (`--reject-with tcp-reset`).
- **DROP** — заблокировать пакет, не сообщая источнику об отказе. Более предпочтительна при фильтрации трафика на интерфейсах, подключенных к интернету, так как понижает информативность сканирования портов хоста злоумышленниками.

Также определенный интерес представляют действия, предоставляемые модулями xtables-addons (http://xtables-addons.sourceforge.net/) (в настоящее время этот проект уже включен (http://packages.debian.org /squeeze/xtables-addons-common) в Debian testing). Некоторые из них:

- **STEAL** — аналогично DROP, но в случае использования в цепочке OUTPUT при блокировании исходящего пакета не сообщает об ошибке приложению, пытавшемуся отправить этот пакет.
- **TARPIT** — «подвесить» TCP-соединение. Используется лишь в самых крайних случаях, например, при борьбе с DoS-атаками. Отвечает на входящее соединение, после чего уменьшает размер фрейма до нуля, блокируя возможность передачи данных. Соединение будет «висеть» в таком состоянии пока не истечет тайм-аут на атакующей стороне (обычно 20—30 минут). При этом на такое соединение расходуются системные ресурсы атакующей стороны (процессорное время и оперативная память), что может быть весьма ощутимо при

значительном количестве соединений. В случае правильного использования действия TARPIT ресурсы атакуемой стороны практически не расходуются.

Под правильным применением понимается предотвращение обработки таких соединений подсистемой conntrack, так как в противном случае будут расходоваться системные ресурсы самого атакуемого хоста. Например, перед добавлением правила блокирования порта

```
iptables -I INPUT -p tcp --dport 25 -j TARPIT
```

обязательно добавляйте в таблицу raw соответствующее правило

```
iptables -t raw -I PREROUTING -p tcp --dport 25 -j NOTRACK
```

предотвращающее обработку блокируемых соединений подсистемой conntrack.

- **DELUDE** — создать видимость открытого TCP-порта. На SYN-пакеты отвечает пакетами SYN/ACK, на все прочие пакеты отвечает RST. Очень полезно для введения в заблуждение злоумышленника, сканирующего порты вашего хоста.
- **CHAOS** — для каждого нового TCP-соединения случайно выбрать одно из двух действий. Первое из них — REJECT, второе, в зависимости от выбранной опции, либо TARPIT (`--tarpit`), либо DELUDE (`--delude`). В частности, при использовании действия `CHAOS --delude` для всех неиспользуемых портов, сканирующий ваши порты злоумышленник получит совершенно неверную информацию о состоянии ваших портов. В случае с `CHAOS --tarpit` ситуация усугубится еще и «подвисающими» соединениями.

Особо заметим, что все действия, перечисленные в качестве допустимых в таблице filter, можно применять в любой из цепочек любой из таблиц (конечно, с разумными ограничениями — например, не стоит ставить действия TARPIT, CHAOS и DELUDE для собственных исходящих пакетов). Однако эти действия предназначены именно для фильтрации, и применение их за пределами таблицы filter является дурным тоном. Как и, например, применение действий, специфичных для таблицы mangle, в таблицах filter и nat.

Таблица security

Предназначена для изменения маркировки безопасности (меток SELinux) пакетов и соединений.

Цепочки

Аналогичны цепочкам таблицы filter:

- **INPUT** — эта цепочка обрабатывает трафик, поступающий непосредственно самому хосту.
- **FORWARD** — через эту цепочку проходит транзитный трафик.
- **OUTPUT** — эта цепочка позволяет обрабатывать трафик, исходящий от самого хоста.

Так как эта таблица появилась относительно недавно, ее редко можно увидеть на схемах следования пакетов через цепочки и таблицы netfilter'a. Поэтому стоит отметить, что все цепочки таблицы security пакеты проходят непосредственно после одноименных цепочек таблицы filter.

Действия

Данная таблица добавлена в ядро Linux в версии 2.6.27. Ранее операции с метками безопасности выполнялись в таблице mangle, и в целях обратной совместимости все действия, разрешенные для таблицы security, можно использовать и в таблице mangle.

- **SECMARK** — устанавливает для пакета контекст безопасности SELinux (единственная допустимая опция `--selctx`).
- **CONNSECMARK** — позволяет скопировать контекст безопасности SELinux с отдельного пакета на соединение в целом (опция `--save`) и наоборот (опция `--restore`).

Таблица raw

Предназначена для выполнения действий с пакетами до их обработки системой conntrack.

Обратите внимание, что в данной таблице не будут срабатывать критерии, которым для корректной работы необходим conntrack (это критерии conntrack, connmark, connlimit, connbytes).

Цепочки

- **PREROUTING** — в эту цепочку входящие пакеты попадают раньше, чем в любую другую из цепочек iptables, и *до* обработки их системой conntrack.
- **OUTPUT** — аналогично для пакетов, сгенерированных самим хостом.

Действия

- **NOTRACK** — позволяет предотвратить обработку пакетов системой conntrack. Разумеется, применять его стоит не ко всем пакетам

подряд, а только к тем, для которых такая обработка не нужна и даже вредна. Например, к пакетам, к которым впоследствии применяется действие TARPIT (см. выше).

- **CT** — более функциональный инструмент, добавленный в версии Linux 2.6.34. Позволяет задать различные настройки conntrack, в соответствии с которыми будет обрабатываться соединение, открытое данным пакетом. В частности, можно:
  - Отключить отслеживание соединения (опция `--notrack`). Таким образом, функциональность действия CT включает и функциональность NOTRACK, так что, возможно, действие NOTRACK будет объявлено устаревшим.
  - Задать вспомогательный модуль (conntrack helper) для данного соединения. Если раньше указать нестандартные порты для такого модуля можно было только в параметрах его загрузки (`/etc/modprobe.conf` или `/etc/modprobe.d/netfilter.conf`, например, `options nf_conntrack_ftp ports=2121`), то теперь это можно сделать и через правила netfilter:

```
iptables -t raw -I PREROUTING -p tcp --dport 2121 -j CT --helper ftp
```

  - Ограничить список событий conntrack, которые будут генерироваться для данного соединения. Например, правило

```
iptables -t raw -I PREROUTING -p tcp --dport 8000 -j CT --ctevents new,destroy
```

будет предписывать для всех соединений на TCP-порт 8000 генерировать только события «открытие соединения» (NEW) и «завершение соединения» (DESTROY), игнорируя все прочие события. Полный список возможных событий: new, related, destroy, reply, assured, protoinfo, helper, mark, natseqinfo, secmark.

  - Задать *зону conntrack* для данного пакета (параметр `--zone`). Механизм зон conntrack позволяет корректно отслеживать, фильтровать и NAT'ить соединения даже в том случае, если хост подключен к нескольким сетям, использующим одинаковые пространства имен, через различные интерфейсы (например, интерфейсы eth0 и eth1 подключены к двум разным сетям, но обе эти сети используют пространство 192.168.0.0/24). Традиционно, для идентификации соединений conntrack использует кортежи (tuples) — набор значений в который входят адреса и порты (в случае ICMP — типы и коды ICMP) источника и назначения при передаче данных в прямом и обратном направлении. Очевидно, что при наличии нескольких подсетей с одинаковыми адресными

пространствами, возможно возникновение путаницы, когда сразу нескольким соединениям ставится в соответствие одна и та же запись в таблице соединений. Чтобы избежать такой ситуации, в кортеж был добавлен идентификатор зоны conntrack — целое число, которое можно устанавливать через специальное правило в таблице raw в зависимости от входящего/исходящего интерфейса (как уже говорилось выше, цепочки таблицы raw пакеты проходят еще до обработки их conntrack'ом). Например,

```
iptables -t raw -I PREROUTING -i eth1 -j CT --zone 1
iptables -t raw -I OUTPUT -o eth1 -j CT --zone 1
```

Таким образом, пакеты, входящие или исходящие через интерфейс eth1, будут получать идентификатор зоны 1, в то время как пакеты интерфейса eth0 будут по-прежнему использовать зону по умолчанию (идентификатор 0), и путаницы не произойдет.
Кроме того, идентификатор зоны можно задать для каждого интерфейса через sysfs, минуя iptables (псевдофайл `/sys/class/net/имя_интерфейса/nf_ct_zone`):

```
echo 1 > /sys/class/net/eth1/nf_ct_zone
```

Однако, стоит заметить, что iptables/netfilter позволяют реализовать более гибкую логику управления идентификатором зоны, опирающуюся на различные параметры пакетов, и поэтом данный параметр действия CT может быть полезен при решении различных сложных задач управления трафиком при использовании зон conntrack.

- **RAWDNAT** — позволяет выполнять «проброс» адресов и портов «сырым» методом — без использования системы conntrack, то есть без учета состояний соединений. Это действие реализовано в рамках проекта xtables-addons. Применять его можно только в таблице raw. Имеет единственную опцию `--to-destination` *адрес*[*/маска*], по смыслу аналогичную опции `--to` действия NETMAP, то есть при указании маски заменяются только те биты в адресе, которые соответствуют единичным битам маски. Биты адреса, соответствующие нулевым битам маски, остаются неизменными. При отсутствии маски изменяется весь адрес.

  Отметим, что в отличие от действий таблицы nat, которые работают только с соединениями в целом, операции «сырого» преобразования адресов работают только с отдельными пакетами, никак не учитывая

контекст их передачи. Вышесказанное можно проиллюстрировать, скажем, таким простым примером: для элементарной операции «проброса» внешнего адреса на другой адрес при использовании обычных операций NAT достаточно одного правила[4]

```
iptables -t nat -A PREROUTING -i eth0 -d 212.201.100.135 -j  DNAT --to-destination 199.181.132.250
```

в то время как для той же операции при использовании «сырых» преобразований необходимо минимум два правила

```
iptables -t raw -A PREROUTING -i eth0 -d 212.201.100.135 -j  RAWDNAT --to-destination 199.181.132.
iptables -t rawpost -A POSTROUTING -o eth0 -s 199.181.132.250 -j RAWSNAT --to-source 212.201.100.1
```

Как уже говорилось выше, при использовании обычной операции DNAT в ответных пакетах, приходящих с пробрасываемого адреса (199.181.132.250), производится автоматическая (без необходимости писать для этого отдельные правила) подмена исходного адреса на изначальный (212.201.100.135), и обращающиеся по адресу 212.201.100.135 хосты даже не подозревают, что общаются с кем-то другим. В случае же «сырого» преобразования такая автоматическая обработка невозможна, так как она требует возможности проверять пакеты на принадлежность соединению. С другой стороны, операции «сырого» преобразования являются более гибким инструментом, а также могут работать даже с UNTRACKED и INVALID-пакетами.

Таблица rawpost

Данная таблица предназначена для выполнения операций «сырого» преобразования адресов (без использования информации о соединениях), которые не могут быть реализованы в рамках таблицы raw, а именно, для операции «сырой» подмены исходного адреса (маскарадинга).

Как и в таблице raw, в rawpost отсутствует возможность обращения к conntrack, поэтому критерии, использующие эту подсистему (conntrack, connmark, connlimit, connbytes), в этой таблице работать не будут.

Данная таблица реализована в рамках проекта xtables-addons и отсутствует в стандартном комплекте iptables/netfilter.

Цепочки

- **POSTROUTING** — обрабатывает весь исходящий трафик.

Действия

В таблице rawpost можно использовать действие **RAWSNAT**, выполняющее операцию «сырой» подмены исходного адреса. Имеет единственную опцию `--to-source` *адрес*[*/маска*], позволяющую задать новый исходный адрес для обрабатываемых пакетов. Как обычно, при наличии маски, в обрабатываемых адресах изменяются только те биты, которые в маске установлены в единицу, при отсутствии маски адрес изменяется целиком. Например, если по правилу

```
iptables -t rawpost -A POSTROUTING -o eth0 -s 10.125.0.0/16 -d 172.18.1.100 -j RAWSNAT --to-source
```

будет обработан пакет с исходным адресом 10.125.32.28, то этот адрес будет заменен на 192.168.32.28.

Если же при задании правила маску /16 в параметре `--to-source` опустить, адрес будет заменен на 192.168.0.20. Но так лучше не делать, потому что мы выполняем трансляцию для всех пакетов из подсети 10.125.0.0/16, и без отслеживания соединений не сможем отличить, какие пакеты следует вернуть 10.125.32.28, а какие — другим хостам из этой подсети. Сама же операция «возвращения» пакетов требует отдельного RAWDNAT-правила:

```
iptables -t raw -A PREROUTING -i eth0 -s 172.18.1.100 -d 192.168.0.0/16 -j RAWDNAT --to-destinatic
```

которое выполняет обратную подмену, заменяя адреса назначения в пакетах-ответах. Таким образом, хост 172.18.1.100 будет считать, что к нему обращаются хосты из подести 192.168.0.0/16, а вовсе не из 10.125.0.0/16.

Для сравнения, с использованием stateful-преобразования достаточно одного правила

```
iptables -t nat -A POSTROUTING -o eth0 -s 10.125.0.0/16 -d 172.18.1.100 -j NETMAP --to 192.168.0.0
```

которое будет обеспечивать все необходимые преобразования адресов.

Заметим, что для корректной работы обеих описанных конфигураций требуется, чтобы хост 172.18.1.100, а также маршрутизаторы на пути от нашего хоста к нему, считали наш хост шлюзом для сети 192.168.0.0/16.

Критерии

Напоминаем, критерий — это логическое выражение, определяющее, соответствует ли пакет или соединение данному конкретному правилу.

В одном правиле можно указать несколько критериев. Для того, чтобы пакет был обработан правилом, должны выполняться *все* критерии, то есть критерии неявно объединяются логическим AND.

Также многие критерии и параметры критериев можно инвертировать, поставив перед ними восклицательный знак. В описании синтаксиса таких критериев присутствует пометка «`[!]`». Инверсия меняет смысл критерия или его параметра на ровно противоположный. Например, критерию `-s 127.0.0.1` соответствуют все пакеты, имеющие обратный адрес 127.0.0.1, а критерию `! -s 127.0.0.1` — все пакеты, *кроме* имеющих обратный адрес 127.0.0.1. В ранних версиях iptables восклицательный знак можно было ставить между названием критерия и значением, например, `-s ! 127.0.0.1`, однако последние версии iptables (в частности, 1.4.3.2 и выше), уже не поддерживают этот синтаксис, выдавая следующую ошибку:

```
Using intrapositioned negation (`--option ! this`) is deprecated in favor of extrapositioned (`! --
```

Далее мы кратко рассмотрим наиболее важные и интересные критерии.

Универсальные критерии

Под этим термином понимаются критерии, применимые ко все пакетам и соединениям, независимо от протокола транспортного уровня, и не требующие подключения внешних модулей (`-m`).

- `[!]` **`-p`**, `--protocol` *`протокол`*

Позволяет указать протокол транспортного уровня. Наиболее часто употребляются `tcp`, `udp`, `icmp` и `all`. Протокол также можно указать с помощью номера или названия согласно перечню, приведенному в `/etc/protocols`. Значение «любой протокол» можно указать с помощью слова `all` или числа 0. Если протокол не указан, подразумевается «любой протокол».

При указании протокола становится возможным использовать специфичные для него критерии. Например, для TCP и UDP доступны критерии `--sport` и `--dport`, для ICMP — `--icmp-type`. Подробнее эти критерии будут рассмотрены ниже.

- `[!]` **`-s`**, `--src`, `--source` *`адрес`*`[`*`/маска`*`][,`*`адрес`*`[`*`/маска`*`]...]`

Определяет адрес отправителя. В качестве адреса может выступать IP-адрес (возможно с маской), имя хоста из /etc/hosts, или доменное имя (в последних двух случаях перед добавлением правила в цепочку имя резольвится в IP-адрес). Маска подсети может быть указана в

классическом формате (например, 255.255.0.0) либо в формате CIDR (например, 16)[5]. Например,

```
iptables -I INPUT -i eth0 -s 192.168.0.0/16 -j DROP
```

(где eth0 — интерфейс, подключенный к интернету), позволяет заблокировать простейший вид спуфинга — из интернета *не могут* приходить пакеты с обратным адресом, принадлежащим к диапазону, зарезервированному для локальных сетей.

Начиная с версии iptables 1.4.6, в одном параметре `-s` можно указывать более одного адреса, разделяя адреса запятой. При этом для каждого адреса будет добавлено *отдельное* правило. Например, запись

```
iptables -A INPUT -i eth0 -s 192.168.0.0/16,172.16.0.0/12,10.0.0.0/8 -j DROP
```

эквивалентна записи

```
iptables -A INPUT -i eth0 -s 192.168.0.0/16 -j DROP
iptables -A INPUT -i eth0 -s 172.16.0.0/12 -j DROP
iptables -A INPUT -i eth0 -s 10.0.0.0/8 -j DROP
```

При добавлении через команду `-A` (вставка в конец цепочки) порядок добавляемых правил будет соответствовать порядку перечисления адресов в исходной команде, при использовании команды `-I` (вставка в начало цепочки либо после заданного правила) порядок будет обратным исходному.

Настойчиво не рекомендуется использовать доменные имена, для разрешения (резольва) которых требуются DNS-запросы, так как на этапе конфигурирования фаервола DNS может работать некорректно. Также, заметим, имена резольвятся всего один раз — при добавлении правила в цепочку. Впоследствии соответствующий этому имени IP-адрес может измениться, но на уже записанные правила это никак не повлияет (в них останется старый адрес). Если указать доменное имя, которое резольвится в несколько IP-адресов, то для каждого адреса будет добавлено отдельное правило (как и в случае перечисления нескольких адресов, см. выше).

- `[!]` **`-d`**`, --dst, --destination` *адрес*`[/`*маска*`][,`*адрес*`[/`*маска*`]...]`

Определяет адрес получателя. Синтаксис аналогичен `-s`.

Заметим, что, если использовать возможность указания нескольких адресов (перечислив их явно или указав доменное имя, резольвящееся на несколько адресов) *одновременно* в параметрах `-s` и `-d`, то на каждое возможное сочетание адресов будет добавлено свое правило. Например, запись

```
iptables -A FORWARD -s 192.168.1.1,192.168.1.2 -d 192.168.1.3,192.168.1.4 -j REJECT
```

эквивалентна

```
iptables -A FORWARD -s 192.168.1.1 -d 192.168.1.3 -j REJECT
iptables -A FORWARD -s 192.168.1.1 -d 192.168.1.4 -j REJECT
iptables -A FORWARD -s 192.168.1.2 -d 192.168.1.3 -j REJECT
iptables -A FORWARD -s 192.168.1.2 -d 192.168.1.4 -j REJECT
```

- [!] **-i**, --in-interface *имя_интерфейса*

Определяет входящий сетевой интерфейс. Если указанное имя интерфейса заканчивается знаком «+» (например, `tun+`), то критерию соответствуют все интерфейсы, чьи названия начинаются на указанное имя (для нашего примера tun0, tun1, ...). Данный критерий можно использовать в цепочках PREROUTING, INPUT и FORWARD. Например,

```
iptables -I INPUT -i lo -j ACCEPT
```

позволит принимать весь трафик, входящий через интерфейс обратной петли.

- [!] **-o**, --out-interface *имя_интерфейса*

Определяет исходящий сетевой интерфейс. Синтаксис аналогичен `-i`. Критерий можно использовать в цепочках FORWARD, OUTPUT и POSTROUTING. Вспоминая один из наших примеров,

```
iptables -t nat -I POSTROUTING -o eth0 -j MASQUERADE
```

будет маскарадить весь трафик, исходящий через интерфейс eth0.

Критерии, специфичные для протоколов

Протоколы транспортного уровня

TCP

- [!] **--sport**, --source-port *порт*[:*порт*]

Позволяет указать исходящий порт (или их диапазон). Например,

```
iptables -I INPUT -m conntrack --ctstate NEW -p tcp --sport 0:1023 -j DROP
```

заблокирует все входящие соединения с привилегированных портов (*привилегированными* в TCP/UDP считаются порты с 0 по 1023 включительно, так как для их использования нужны привилегии суперпользователя, и обычно такие порты используются демонами только в режиме прослушивания).

- [!] **--dport**, --destination-port *порт*[:*порт*]

Позволяет указать порт назначения (или их диапазон). Синтаксис аналогичен `--sport`. Например,

```
iptables -I INPUT -p tcp --dport 80 -j ACCEPT
```

разрешит все входящие пакеты на 80 порт (HTTP).

- [!] **--tcp-flags** *маска установленные_флаги*

Позволяет указать список установленных и снятых TCP-флагов. В *маске* перечисляются (через запятую, без пробелов) все проверяемые флаги, далее, после пробела, перечисляются (также через запятую) те из них, которые должны быть установлены. Все остальные перечисленные в маске флаги должны быть сняты. Возможные флаги: SYN ACK FIN RST URG PSH. Также можно использовать псевдофлаги ALL и NONE, обозначающие «все флаги» и «ни одного флага» соответственно.

Пример:

```
iptables -I INPUT -p tcp --tcp-flags SYN,RST,ACK,FIN SYN -j LOG --log-level DEBUG --log-prefix "TC
```

будет заносить в лог все входящие TCP SYN-пакеты. У обычного SYN-пакета всегда установлен флаг SYN и сняты флаги RST, ACK и FIN.

Другой пример:

```
iptables -I INPUT -m conntrack --ctstate NEW,INVALID -p tcp --tcp-flags SYN,ACK SYN,ACK -j REJECT
```

будет препятствовать спуфингу от нашего имени. Ведь если мы получаем пакет с установленными флагами SYN и ACK (такой комбинацией флагов обладает только ответ на SYN-пакет) по еще не открытому соединению, это означает, что кто-то послал другому хосту SYN-пакет от нашего имени, и ответ пришел к нам. Однако добавление такого правила в конфигурацию фаервола *не рекомендуется*, поскольку стандартом также предусмотрено использование случайного начального номера последовательности для каждого нового TCP-соединения, как раз для предотвращения возможности спуфинг-атаки. Шансов угадать начальный номер у злоумышленника практически нет, тогда как наш ответ RST-пакетом на такой пакет может серьезно нарушить работоспособность некоторых систем защиты от DoS-атак, полагающихся на метод, называемый out-of-sequence ACK. В случае добавления этого правила в конфигурацию фаервола, системы защиты будут считать, что, раз уж мы отвечаем RST на out-of-sequence ACK, вы действительно пытались установить соединение с ресурсом, и пропустят все последующие поддельные пакеты злоумышленника, отправленные от вашего имени. В результате вы заметно облегчите злоумышленнику осуществление DoS-атаки от вашего имени, и при расследовании этого эпизода следы приведут к вам.

- `[!]` **`--syn`**

Позволяет отлавливать TCP SYN-пакеты (сокращение для `--tcp-flags SYN,RST,ACK,FIN SYN`). Так что приведенный выше пример про логгирование SYN-пакетов можно записать в виде

```
iptables -I INPUT -p tcp --syn -j LOG --log-level DEBUG --log-prefix "TCP SYN: "
```

Более интересный пример:

```
iptables -I INPUT -m conntrack --ctstate NEW -p tcp ! --syn -j DROP
```

будет блокировать все попытки открыть входящее TCP-соединение *не* SYN-пакетом. Попытка установить соединение таким образом может быть либо ошибкой, либо атакой.

- `[!]` **`--tcp-option`** *номер*

Позволяет проверить, установлена ли в заголовке TCP-пакета соответствующая опция. Полный список опций с их номерами представлен на сайте IANA (http://www.iana.org/assignments/tcp-parameters /tcp-parameters.xhtml).

UDP

- [!] **--sport**, --source-port *порт*[:*порт*]

Позволяет указать исходящий порт (или их диапазон). Синтаксис и принцип работы аналогичен описанной выше одноименной опции TCP.

- [!] **--dport**, --destination-port *порт*[:*порт*]

Позволяет указать входящий порт (или их диапазон). Синтаксис и принцип работы аналогичен описанной выше одноименной опции TCP.

SCTP

- [!] **--sport**, --source-port *порт*[:*порт*]

Позволяет указать исходящий порт (или их диапазон). Синтаксис и принцип работы аналогичен описанной выше одноименной опции TCP.

- [!] **--dport**, --destination-port *порт*[:*порт*]

Позволяет указать входящий порт (или их диапазон). Синтаксис и принцип работы аналогичен описанной выше одноименной опции TCP.

- [!] **--chunk-types** {all|any|only} *тип_секции*[:*флаги*][,*тип_секции*[:*флаги*]...]

Позволяет анализировать набор секций (chunks), входящих в состав SCTP-пакета. Возможные типы секций: DATA INIT INIT_ACK SACK HEARTBEAT HEARTBEAT_ACK ABORT SHUTDOWN SHUTDOWN_ACK ERROR COOKIE_ECHO COOKIE_ACK ECN_ECNE ECN_CWR SHUTDOWN_COMPLETE ASCONF ASCONF_ACK. Также, для секций, имеющих флаги (DATA, ABORT и SHUTDOWN_COMPLETE) можно проверить состояние флагов, указав соответствующие буквы после названия секции (для DATA — U, B и E, для ABORT и SHUTDOWN_COMPLETE — T). Литера в верхнем регистре предполагает установленный флаг, в нижнем — снятый. Ключевое слово в начале позволяет определить логику работы: в случае `all` должны присутствовать *все* перечисленные секции, для `any` — хотя бы одна из них, для `only` — пакет должен состоять только из перечисленных секций. Примеры:

```
iptables -I INPUT -p sctp --chunk-types only DATA,INIT -j DROP
```

заблокирует все входящие SCTP-пакеты, состоящие только из секций DATA и INIT, а

```
iptables -I INPUT -p sctp --chunk-types any DATA:Be -j ACCEPT
```

разрешит входящие SCTP-пакеты, содержащие секцию DATA с установленным флагом B и снятым флагом E, то есть первый фрагмент фрагментированной DATA-секции.

DCCP

- [!] **--sport**, --source-port *порт*[:*порт*]

Позволяет указать исходящий порт (или их диапазон). Синтаксис и принцип работы аналогичен описанной выше одноименной опции TCP.

- [!] **--dport**, --destination-port *порт*[:*порт*]

Позволяет указать входящий порт (или их диапазон). Синтаксис и принцип работы аналогичен описанной выше одноименной опции TCP.

- [!] **--dccp-types** *маска*

Позволяет указать тип DCCP-пакета. В *маске* через запятую перечисляются DCCP-типы (допустимые названия: REQUEST RESPONSE DATA ACK DATAACK CLOSEREQ CLOSE RESET SYNC SYNCACK INVALID). Пакет считается удовлетворяющим критерию, если имеет один из типов, перечисленных в маске. Например,

```
iptables -I INPUT -p dccp --dccp-types RESET,INVALID -j LOG --log-level DEBUG --log-prefix "DCCP R
```

занесет в лог все входящие RESET и INVALID DCCP-пакеты.

UDP Lite

Поддержка специфических опций протокола UDPLite (а именно, проверки портов источника и/или назначения) в netfilter реализована не вполне очевидным образом: через критерий multiport (см. раздел вспомогательные критерии).

Если для обычного UDP соответствие номера порта проверялось выражением вида `-p udp --dport` *номер_порта* (или `-p udp --sport` *номер_порта*), то в случае UDPLite аналогичное выражение будет выглядеть как `-p udplite -m multiport --dports` *номер_порта* (или `-p udplite -m multiport --sports` *номер_порта*). Все

опции критерия multiport для UDPLite поддерживаются в полном объеме.

IPv4

Ниже перечислены критерии, которые реализованы только для протокола IPv4 и, соответственно, могут быть вызваны только через iptables, но не через ip6tables.

- `[!] -f, --fragment`

Проверка фрагментации: критерию соответствуют только фрагменты пакета, начиная со второго фрагмента. Пример:

```
iptables -I INPUT -p icmp -f -j DROP
```

блокирует фрагменты ICMP-пакетов. Так как, в силу функционального назначения протокола, ICMP-пакеты должны быть очень небольшими и нормально укладываться в MTU, наличие их фрагментов обычно свидетельствует об ошибке или попытке атаки.

У второго и последующего фрагментов нет заголовка транспортного уровня, поэтому бессмысленно пытаться использовать для них критерии номеров TCP/UDP-портов или типа ICMP.

- **addrtype** — позволяет проверить тип адреса источника и/или назначения с точки зрения подсистемы маршрутизации сетевого стека ядра. Допустимые типы адресов: UNSPEC (адрес 0.0.0.0), UNICAST, LOCAL (адрес принадлежит нашему хосту), BROADCAST, ANYCAST, MULTICAST, BLACKHOLE, UNREACHABLE, PROHIBIT, THROW, NAT, XRESOLVE. Подробнее о большинстве перечисленных типов адресов и их использовании в Linux (точнее, в подсистеме iproute2) можно почитать здесь (http://linux-ip.net/html/routing-tables.html). Данный критерий поддерживает следующие параметры:

`[!] --src-type` *тип*`[,`*тип*`...]` — проверка, принадлежит ли исходный адрес пакета одному из перечисленных типов.

`[!] --dst-type` *тип*`[,`*тип*`...]` — проверка, принадлежит ли адрес назначения пакета одному из перечисленных типов.

`--limit-iface-in` — ограничивает проверку типа адреса только записями для интерфейса, через который пакет вошел на хост. Допускается использовать этот параметр только в цепочках PREROUTING, INPUT и FORWARD.

`--limit-iface-out` — ограничивает проверку типа адреса только записями для интерфейса, через который пакет вышел с хоста. Допускается

использовать этот параметр только в цепочках POSTROUTING, OUTPUT и FORWARD.

Критерии `--limit-iface-in` и `--limit-iface-out` нельзя указывать одновременно.

В качестве практического примера использования данного критерия можно привести простейшую защиту от спуфинга:

```
iptables -I INPUT -m addrtype --src-type LOCAL ! -i lo -j DROP
```

Это правило заблокирует пакеты, которые пришли с внешних интерфейсов, но при этом в качестве обратного адреса у них указан один из адресов, принадлежащих нашему хосту (например, 127.0.0.1).

- **ecn** — позволяет проверять значения битов ECN в заголовках TCP и IPv4. Допустимые параметры:

`[!] --ecn-tcp-cwr` — проверка флага ECN CWR (Congestion Window Received) в TCP-заголовке пакета.

`[!] --ecn-tcp-ece` — проверка флага ECN ECE (ECN Echo) в TCP-заголовке пакета.

`[!] --ecn-ip-ect` *значение* — проверка значения, сформированного ECN-битами поля TOS (последние два бита). Возможные значения от 0 до 3.

- **realm** — проверка области маршрутизации (realm) пакета. Имеет единственный параметр

`[!] --realm` *значение*`[/`*маска*`]`, позволяющий указать численный идентификатор области. Если указана маска, то значение realm каждого проверяемого пакета сначала объединяется с маской при помощи побитового AND, а затем сравнивается со значением, указанным в правиле. Также вместо числа можно указать символьное название области согласно обозначениям в файле `/etc/iproute2/rt_realms` (в этом случае маску использовать нельзя).

- **ttl** — обеспечивает проверку поля TTL в заголовке пакета. Позволяет установить, является ли значение TTL проверяемого пакета большим (`--ttl-gt`), меньшим (`--ttl-lt`), равным или не равным (`[!] --ttl-eq`) указанному значению.

ICMP

ICMP является протоколом контрольных сообщений IPv4. Несмотря на то, что ICMP формально является самостоятельным протоколом, и обращение к соответствующему критерию должно производиться с использованием

синтаксиса для протокола (`-p`), а не для вспомогательного критерия (`-m`), де-факто этот протокол неотделим от IPv4, и поэтому соответствующий критерий может использоваться только в iptables, но не в ip6tables.

Критерий этого протокола имеет единственный параметр

`[!]` **`--icmp-type`** *тип* — обеспечивает проверку типа ICMP-пакета. Список возможных типов выводится по команде `iptables -p icmp -h`. Также можно указать стандартные числовые тип и, при необходимости, код. Например,

```
iptables -I INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

как и аналогичное

```
iptables -I INPUT -p icmp --icmp-type 8 -j ACCEPT
```

пропустят все входящие ICMP-эхо-запросы (пинги).

IPv6

Ниже перечислены критерии, которые реализованы только для протокола IPv6 и, соответственно, могут быть вызваны только через ip6tables, но не через iptables.

Особо отметим, что критерии для некоторых подзаголовков IPv6 (dst, frag, hbh, rt) корректно вызываются *только* с использованием синтаксиса для расширенных критериев (ключ `-m`), хотя имеют номера протоколов в `/etc/protocols` и, формально, могут быть вызваны с использованием синтаксиса для протоколов (ключ `-p`) что, однако, является неверным. В таких случаях ip6tables выдает предупреждение вида

```
Warning: never matched protocol: ipv6-frag. use extension match instead.
```

гласящее, что в такое правило никогда не будет срабатывать. Если вы видите подобное предупреждение, удалите правило, которое его вызвало, и перепишите это правило с использованием корректного синтаксиса (`-m`).

С другими же критериями, которые представляют более или менее самостоятельные протоколы (ICMPv6, MH, IPSec AH и ESP), все ровно наоборот, и для них корректным будет использование именно синтаксиса протокола (ключ `-p`), а не расширенного критерия. Во избежание

путаницы, такие критерии вынесены в отдельные подзаголовки, по аналогии с протоколами транспортного уровня (выше).

- **dst** — проверят опции назначения (подзаголовок 60 Destination Options). Имеет две опции:

`[!] --dst-len` *значение* — проверяет длину подзаголовка;

`[!] --dst-opts` *код*`[:`*длина*`][,`*код*`[:`*длина*`]...]` — позволяет проверить вхождение в данный подзаголовок конкретных опций. Также можно проверить длину каждой из них. Можно указать до 16 опций (представленных числовыми кодами), разделяя их запятой.

- **eui64** — сравнивает младшие 64 бита исходного IPv6-адреса с битами исходного MAC-адреса, преобразованными согласно правилам автоконфигурирования IPv6-адресов на основании MAC-адресов. Таким образом, позволяет проверить, является ли IPv6-адрес отправителя пакета автоматически сконфигурированным (работает, если отправитель находится в одном широковещательном домене с нашим хостом, так как в противном случае сохранность MAC-адреса отправителя не может быть гарантирована).

- **frag** — позволяет проверять параметры фрагментации (подзаголовок 44 Fragment). Допустимые опции:

`[!] --fragid` *мин*`[:`*макс*`]` — сверяет величину идентификатора фрагмента. Если указано одно значение, проверяется равенство, если указан диапазон — проверяется вхождение значения в этот диапазон (включая границы).

`--fragres` — проверяет, заполнены ли нулями зарезервированные поля заголовка (биты 29-30).

`--fragfirst` — проверяет, является ли данный фрагмент первым в последовательности.

`--fraglast`/`--fragmore` — проверяют M-флаг, показывающий, является ли данный фрагмент последним в последовательности (в этом случае `--fraglast` дает истину, а `--fragmore` ложь), или после него должны быть еще фрагменты (соответственно, наоборот). Эти две опции являются взаимоисключающими.

Также в документации можно найти упоминание параметра `--fraglen` , который якобы проверяет длину подзаголовка Fragment. Однако, согласно текущим соглашениям, длина этого подзаголовка фиксирована и равна 8 байтам, а названная опция ни на что не влияет.

- **hbh** — проверяет опции Hop-by-Hop (подзаголовок 0 Hop-by-Hop). Синтаксис аналогичен параметру dst.

- **hl** — обеспечивает проверку поля Hop Limit в IPv6-заголовке. Позволяет установить, является ли значение HL проверяемого пакета большим (`--hl-gt`), меньшим (`--hl-lt`), равным или не равным (`[!] --hl-eq`) указанному значению.

- **ipv6header** — позволяет проверить наличие вспомогательных подзаголовков IPv6. Допустимыми типами заголовков являются: hop (hop-by-hop 0), dst (ipv6-opts 60), route (ipv6-route 43), frag (ipv6-frag 44), auth (ah 50), esp (esp 59). В скобках указаны «длинные» названия и числовые идентификаторы протокола. Используя опцию

`[!] --header` *тип*`[,`*тип*`...]`

вы можете указывать любые из этих обозначений — короткие названия, длинные названия или числовые идентификаторы. Отметим, что пакет будет соответствовать критерию в том случае, если будет содержать *только* перечисленные подзаголовки. Изменить это поведение можно, указав опцию `--soft` — в этом случае достаточно, чтобы пакет содержал хотя бы один из перечисленных подзаголовков.

- **rt** — позволяет проверять параметры маршрутизации пакета (подзаголовок 43 Routing). Допустимые параметры:

`[!] --rt-type` *тип* — тип маршрутизации (0, 1 или 2);

`[!] --rt-segsleft` *мин*`[:`*макс*`]` — проверка значения поля Segment Left (показывает, сколько еще узлов должен пройти данный пакет, прежде чем достигнет цели);

`[!] --rt-len` *значение* — проверка длины всего подзаголовка.

Следующие три параметра работают для типа 0 (`--rt-type 0`):

`--rt-0-res` — проверять зарезервированные поля;

`--rt-0-addrs` *адрес*`[,`*адрес*`...]` — проверяет адреса, перечисленные в подзаголовке. Можно указать не более 16 адресов, разделяя их запятыми. Пакет считается удовлетворяющим критерию, если подзаголовок содержит все адреса, которые были перечислены в правиле, в том же порядке;

`--rt-0-not-strict` — делает условие проверки по предыдущем параметру не таким жестким: достаточно, чтобы хотя бы один из перечисленных адресов совпадал с соответствующим адресом в подзаголовке, при соблюдении порядка перечисления.

ICMPv6

ICMPv6 является протоколом контрольных сообщений IPv6. Несмотря на

то, что ICMPv6 формально является самостоятельным протоколом, и обращение к соответствующему критерию должно производиться с использованием синтаксиса для протокола (`-p`), а не для вспомогательного критерия (`-m`), де-факто этот протокол неотделим от IPv6, и поэтому соответствующий критерий может использоваться только в ip6tables, но не в iptables.

Критерий этого протокола имеет единственный параметр

`[!]` **`--icmpv6-type`** *тип*

по смыслу и синтаксису аналогичный параметру `--icmp-type` критерия icmp (см. предыдущий раздел). Например,

```
ip6tables -I INPUT -p icmpv6 --icmpv6-type echo-request -j ACCEPT
```

как и аналогичное

```
ip6tables -I INPUT -p icmpv6 --icmpv6-type 128 -j ACCEPT
```

пропустит входящие ICMPv6-эхо-запросы (пинги).

Как и в случае icmp, вы можете получить список допустимых типов сообщений, введя

```
ip6tables -p icmpv6 -h
```

MH

Критерий mh позволяет определять пакеты с подзаголовком IPv6 Mobility Header (RFC 3775 (http://tools.ietf.org/html/rfc3775)). Как и в случае с ICMPv6, вызов данного критерия должен выполняться в соответствии с синтаксисом для протокола (через ключ `-p`). Имеет единственную опцию

`[!]` **`--mh-type`** *мин*`[:`*макс*`]`

позволяющую указать точное значение типа или диапазон допустимых значений. Перечень поддерживаемых типов MH можно получить, введя команду

```
```

```
ip6tables -p mh -h
```

Также к данному критерию допустимо обращение по названию ipv6-mh (`-p ipv6-mh`).

IPsec

netfilter «знает» два протокола из семейства IPsec: Authentication Header (`-p ah`) и Encapsulating Security Payload (`-p esp`).

AH

Для протокола AH поддерживается единственная опция

- **`--ahspi`** *значение*`[:`*значение*`]` — позволяет указать значение (или диапазон значений) SPI (Security Parameter Index).

ESP

Протокол ESP имеет аналогичную по смыслу опцию

- **`--espspi`** *значение*`[:`*значение*`]`

В большинстве конфигураций IPsec-пакеты просто пропускаются фаерволом (пусть с ними разбирается IPsec-подсистема):

```
iptables -I INPUT -p ah -j ACCEPT
iptables -I INPUT -p esp -j ACCEPT
```

Критерии состояния соединения

conntrack

Основной критерий, используемый для контроля состояния соединения. Он предоставляет эффективный набор инструментов, позволяющий использовать информацию системы conntrack о состоянии соединения. Затронутую тему весьма проблематично осветить в двух словах и привести по ней простые примеры, не выходя за краткий формат, принятый для нашей статьи. Поэтому ограничимся мы подробно рассмотрим лишь наиболее важный параметр данного критерия — ctstate. Для прочих параметров мы лишь кратко опишем синтаксис и назначение. За более подробной информацией обратитесь к документации, прилагаемой к вашему дистрибутиву. Итак,

- `[!]` **`--ctstate`** *маска*

Маска содержит перечисление через запятую список возможных состояний соединения. Пакет считается удовлетворяющим критерию, если соединение, по которому он проходит, находится в одном из перечисленных состояний.

Возможные состояния:

- **NEW** — соединение не открыто, то есть пакет является первым в соединении. Полезные примеры использования этого состояния приведены выше, в частности, при описании TCP-специфичных критериев.
- **ESTABLISHED** — пакет относится к уже установленному соединению. Обычно такие пакеты принимаются без дополнительной фильтрации, как и в случае с RELATED.
- **RELATED** — пакет открывает новое соединение, логически связанное с уже установленными, например, открытие канала данных в пассивном режиме FTP.

Например:

```
modprobe nf_conntrack_ftp # Подгружаем модуль для распознавания связанных FTP-соединений
# В старых системах этот модуль может называться ip_conntrack_ftp
iptables -F # Очищаем все цепочки таблицы filter
# Ко всем пакетам, которые относятся к уже установленным соединениям, применяем терминальное дейст
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 21 -j ACCEPT # Разрешаем открывать сое
iptables -P INPUT DROP # В качестве действия по умолчанию устанавливаем DROP — блокирование пакета
iptables -P OUTPUT ACCEPT # Разрешаем все исходящие пакеты
```

Этот набор команд обеспечит функционирование на хосте FTP-сервера с поддержкой как активного, так и пассивного режимов. В пассивном режиме FTP клиент сначала устанавливает управляющее соединение на TCP-порт 21 сервера. Когда возникнет необходимость передачи данных, клиент даст серверу команду PASV. Сервер выберет высокий порт и подключится к нему в режиме прослушивания, отправив номер порта клиенту по управляющему соединению. Система conntrack, при наличии подгруженного модуля ядра nf_conntrack_ftp, зафиксирует это сообщение и выделит номер порта. Когда клиент откроет соединение данных на этот высокий порт, система conntrack присвоит первому пакету статус RELATED, в результате чего пакет пройдет по нашему правилу и будет принят. Остальные пакеты в этом соединении будут иметь уже статус ESTABLISHED и тоже будут приняты.

Возможность корректно обрабатывать пассивный режим FTP, как это было описано выше, из всех unix-фаерволов доступна только в iptables. В других фаерволах для решения данной проблемы используются различные «костыли». Например, в pf управляющие FTP-соединения пропускаются

(http://www.openbsd.org/faq/pf/ftp.html#client) через специальную программу ftp-proxy (http://www.openbsd.org/cgi-bin/man.cgi?query=ftp-proxy&sektion=8), которая анализирует трафик подобно conntrack и «на лету» добавляет правила для соединений данных. Однако даже подобные «костыли» существуют далеко не для всех протоколов. Поэтому можно утверждать, что в данном аспекте iptables находится вне конкуренции.

С обработкой активного режима таких проблем не возникает, так как сервер сам устанавливает соединение данных с порта 20, поэтому первый пакет пропускается согласно правилу по умолчанию цепочки OUTPUT. Правда, проблемы могут возникнуть на стороне фаервола клиента, но это уже не относится к компетенции сервера.

- **INVALID** — пакет по смыслу должен принадлежать уже установленному соединению (например, ICMP-сообщение port-unreachable), однако такое соединение в системе не зарегистрировано. Обычно к таким пакетам применяют действие DROP:

```
iptables -I INPUT -m conntrack --ctstate INVALID -j DROP
```

- **UNTRACKED** — отслеживание состояния соединения для данного пакета было отключено. Обычно оно отключается с помощью действия NOTRACK в таблице raw.

- **DNAT** — показывает, что к данному соединению применена операция подмены адреса назначения (об операциях преобразования адресов см. выше).

- **SNAT** — показывает, что к данному соединению применена операция подмены адреса источника (об операциях преобразования адресов см. выше).

Остальные параметры критерия conntrack, как уже говорилось, мы опишем лишь конспективно.

- [!] **--ctstatus** *маска*

Применяется для определения статуса соединения в системе conntrack. Возможные статусы:

- **EXPECTED** — данное соединение ожидалось системой conntrack по результатам анализа других соединений. Например, после того, как клиент и сервер через управляющее FTP-соединение согласуют номер порта для соединения данных в пассивном режиме, система conntrack на сервере будет ожидать входящее соединение на этот порт.

- **CONFIRMED** — подтвержденное соединение. Такой статус присваивается соединению после того, как инициатор начал передачу пакетов.
- **SEEN_REPLY** — соединение, по которому поступил ответ, то есть имеет место передача данных в обоих направлениях (поддержка данного состояния появилась сравнительно недавно).
- **ASSURED** — соединение можно считать полностью установленным. Этот статус присваивается соединению после передачи определенного количества данных. Присвоение данного статуса приводит к увеличению conntrack-тайм-аута для данного соединения (эти тайм-ауты используются для определения и удаления «повисших» и оборванных соединений).
- **NONE** — нет статуса. Соединение не соответствует ни одному из перечисленных критериев.

- `[!] **--ctproto** *протокол*`

Протокол транспортного уровня, определенный системой conntrack. Синтаксис аналогичен стандартному критерию `-p`.

- `[!] **--ctdir** {ORIGINAL|REPLY}`

Позволяет указать направление прохождения пакетов (ORIGINAL — от инициатора к отвечающему, REPLY — наоборот). Если не указывать эту опцию, под критерий будут подпадать пакеты, идущие в обоих направлениях.

- `[!] **--ctorigsrc** *адрес*[*/маска*], [!] **--ctorigdst** *адрес*[*/маска*], [!] **--ctreplsrc** *адрес*[*/маска*], [!] **--ctrepldst** *адрес*[*/маска*]`

Позволяет определять адреса источника (src) и назначения (dst) при передаче данных от инициатора к отвечающему (orig) и наоборот (repl). Синтаксис аналогичен описанному выше стандартному критерию `-s`.

Обычно адрес-порт источника при передаче в прямом направлении совпадают с адресом-портом назначения при передаче в обратном направлении, и наоборот, адрес-порт источника при передаче в обратном направлении совпадает с адресом-портом назначения при передаче в прямом направлении. То есть, если клиент 192.168.1.2 обращается к серверу 192.168.1.1, то с точки зрения системы conntrack ситуация выглядит следующим образом:

→ В прямом направлении (от инициатора к отвечающему):
— Адрес источника (ctorigsrc): 192.168.1.2 (адрес клиента)
— Адрес назначения (ctorigdst): 192.168.1.1 (адрес сервера)
← В обратном направлении (от отвечающего к инициатору):
— Адрес источника (ctreplsrc): 192.168.1.1 (адрес сервера)
— Адрес назначения (ctrepldst): 192.168.1.2 (адрес клиента)

Однако для соединений, к которым применена трансляция адресов или портов, это может не выполняться. Например, рассмотрим ситуацию, когда клиент из нашей подсети (192.168.1.2) обращается к некоторому серверу в интернете (204.152.191.37) через наш сервер (внешний адрес нашего сервера 208.77.188.166, внутренний 192.168.1.1). Для корректной работы такой схемы нужно обеспечить на нашем сервере подмену исходного адреса (SNAT), например,

```
iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to-source 208.77.188.166
```

Тогда, с точки зрения системы conntrack нашего сервера,

→ В прямом направлении (от инициатора к отвечающему):
— Адрес источника (ctorigsrc): 192.168.1.2 (внутренний адрес клиента)
— Адрес назначения (ctorigdst): 204.152.191.37 (адрес интернет-сервера)
← В обратном направлении (от отвечающего к инициатору):
— Адрес источника (ctreplsrc): 204.152.191.37 (адрес интернет-севера)
— Адрес назначения (ctrepldst): 208.77.188.166 (внешний адрес нашего сервера)

В том случае, если адрес-порт источника при передаче в прямом направлении отличаются от адреса-порта назначения при передаче в обратном направлении, соединению будет присвоен статус SNAT.

Аналогично, если один из внешних адресов нашего сервера «проброшен» на внутренний сервер, например, так

```
iptables -t nat -A PREROUTING -d 208.77.188.166 -j DNAT --to-destination 192.168.1.2
```

то при обращении на этот адрес клиента извне (допустим, все тот же 204.152.191.37), получим

→ В прямом направлении (от инициатора к отвечающему):
— Адрес источника (ctorigsrc): 204.152.191.37 (адрес клиента)
— Адрес назначения (ctorigdst): 208.77.188.166 (наш внешний адрес)
← В обратном направлении (от отвечающего к инициатору):
— Адрес источника (ctreplsrc): 192.168.1.2 (адрес внутреннего сервера)
— Адрес назначения (ctrepldst): 204.152.191.37 (адрес клиента)

В том случае, если адрес-порт назначения при передаче в прямом направлении отличаются от адреса-порта источника при передаче в обратном направлении, соединению будет присвоен статус DNAT.

- [!] **--ctorigsrcport** *порт*, [!] **--ctorigdstport** *порт*, [!] **--ctreplsrcport** *порт*, [!] **--ctrepldstport** *порт*

Позволяет определять порты источника (src) и назначения (dst) при передаче данных от инициатора к отвечающему (orig) и наоборот (repl) для протоколов транспортного уровня, имеющих порты (TCP, UDP, SCTP, DCCP).

Ситуация с портами в принципе аналогична описанной выше ситуации с адресами, поэтому рассматривать ее отдельно мы не будем.

- [!] **--ctexpire** *мин_время*[:*макс_время*]

Позволяет указать в качестве критерия оставшееся по тайм-ауту время в секундах для данного соединения.

Система conntrack присваивает каждому соединению тайм-аут (счетчик обратного отсчета времени). Когда этот счетчик доходит до нуля, система conntrack удаляет информацию о соединении из своих таблиц. Тайм-аут устанавливается в значение по умолчанию каждый раз, когда по соединению передаются данные. Таким образом, активно используемые соединения никогда не будут сброшены.

Заметим, что значения тайм-аута по умолчанию для разных протоколов, и даже для одного протокола на разных стадиях установки соединения, могут различаться. Например, для протокола TCP вводятся отдельные значения тайм-аутов для состояний SYN_SENT, SYN_RECV, ESTABLISHED (не путать TCP-состояние ESTABLISHED с conntrack-состоянием ESTABLISHED), FIN_WAIT, CLOSE_WAIT, LAST_ACK, TIME_WAIT, CLOSE. Для UDP существуют два тайм-аута: для соединений, не получивших статуса ASSURED (conntrack_udp_timeout) и для соединений, по которым передано достаточное количество данных в обе стороны для получения этого статуса (conntrack_udp_timeout_stream). Для ICMP тайм-аут только один.

state

Идеологический предшественник критерия conntrack. Имеет единственный параметр `--state`, аналогичный параметру `--ctstate` критерия conntrack (но, в отличие от него, не поддерживающий состояния DNAT и SNAT).

Долгое время был основным критерием определения состояния, и до сих пор фигурирует во многих руководствах и примерах. Однако в настоящее время разработчики iptables рекомендуют использовать вместо него критерий conntrack. Возможно, что критерий state вообще будет удален из будущих версий iptables/netfilter.

Дополнительные критерии

Дополнительные критерии iptables подгружаются при помощи параметра `-m` (`--match`). Далее можно указать дополнительные параметры, специфичные для данного конкретного критерия. Если дополнительных критериев несколько, не смешивайте их параметры — указывайте нужные параметры каждого критерия непосредственно после его вызова через `-m`.

Перед некоторыми дополнительными параметрами критерия может стоять восклицательный знак. В соответствии с принятыми в iptables соглашениями, это означает логическую инверсию параметра, то есть его смысл меняется на противоположный. Но такая возможность предусмотрена отнюдь не для всех параметров.

Заметим, что перечисленные выше критерии состояния и критерии, специфичные для протоколов, по сути также являются параметрами соответствующих дополнительных критериев (например, при указании `-p tcp` подразумевается еще и `-m tcp`). Однако в силу ряда причин более логичным и понятным было их отделение от остальных дополнительных критериев. Обратите внимание, что *все* параметры вышеперечисленных критериев допускают логическое отрицание (например, `-m conntrack ! --ctstate NEW`).

Справочную информацию по любому из дополнительных критериев можно получить, используя команду

```
iptables -m название_критерия -h
```

Вспомогательные критерии

К этой группе можно отнести критерии, расширяющие возможности других критериев.

- **multiport** — позволяет указать несколько (до 15) портов и/или их диапазонов (для протоколов TCP, UDP, SCTP, DCCP и UDP Lite). Поддерживает следующие параметры

`[!] --sports, --source-ports` *порт*`[:`*порт*`][,`*порт*`[:`*порт*`][,...]]` — улучшенная версия критерия `--sport`, описанного выше. Позволяет перечислить (без пробелов, через запятую) до 15 портов или их диапазонов.

`[!] --dports, --destination-ports` *порт*`[:`*порт*`][,`*порт*`[:`*порт*`][,...]]` — улучшенная версия критерия `--dport`, описанного выше. Например,

```
iptables -I INPUT -p tcp -m multiport --dport 80,8000:8008 -j ACCEPT
```

позволит принимать TCP-пакеты, приходящие на порты 80 и с 8000 по 8008.

`[!] --ports` *порт*`[:`*порт*`][,`*порт*`[:`*порт*`][,...]]` — пакет будет подпадать под этот критерий, если его исходный порт *или* порт назначения присутствует в указанном списке.

- **iprange** — позволяет указать диапазон IP-адресов, не являющийся подсетью. Поддерживает следующие параметры:

`[!] --src-range` *адрес*`[-`*адрес*`]` — позволяет указать диапазон исходных адресов. Например,

```
iptables -I INPUT -m iprange --src-range 192.168.0.8-192.168.0.25 -j DROP
```

заблокирует все пакеты, исходный адрес которых лежит в диапазоне с 192.168.0.8 по 192.168.0.25 включительно.

`[!] --dst-range` *адрес*`[-`*адрес*`]` — позволяет указать диапазон адресов назначения.

Критерии маркировки

- **mark** — позволяет выделять пакеты с заданной маркировкой (nfmark). Имеет единственную опцию

`[!] --mark` *значение*`[/`*маска*`]` — указывает значение маркировки. Простейший пример:

```
-m mark --mark 15
```

будет выделять пакеты с маркировкой 15.

Если указана маска, то перед сравнением с заданным значением маркировка каждого пакета комбинируется с этой маской посредством логической операции AND, то есть проверяется условие `x &` *маска* `==` *значение* (где x — маркировка текущего пакета). Такой подход позволит сравнивать значения отдельных бит. Например, критерию

```
-m mark --mark 64/64
```

будет отлавливать пакеты, в маркировке которых установлен 7-й бит ($2^6 = 64$, при этом первый бит соответствует 2^0). В частности, 64…127, 192…255, 320…383 и т. д.

Еще один пример —

```
-m mark --mark 2/3
```

будет определять пакеты, в маркировке которых установлен второй бит, но снят первый. Такие числа будут нацело делиться на два, но не делиться на четыре — 2, 6, 10, 14, …

- **connmark** — полностью аналогичен mark, но проверяет не маркировку пакета (nfmark), а маркировку соединения (ctmark). Также имеет параметр `--mark` с аналогичным синтаксисом.

В качестве практического примера использования меток пакетов и соединений рассмотрим улучшение работы l7-filter-userspace.

Userspace-вариант l7-filter является демоном, взаимодействующим с netfilter через подсистему nfnetlink_queue — действие NFQUEUE в терминологии iptables. При помощи этого действия определенные пакеты можно направить на анализ демону l7-filter. По результатам анализа демон выставит маркировку пакетов: 1 — пакет принадлежит новому соединению, тип которого пока не идентифицирован; 2 — пакет принадлежит соединению, тип которого идентифицировать так и не удалось. Другие значения соответствуют установленным типам соединений (соответствие задается в конфигурационном файле демона) [1] (http://l7-filter.sourceforge.net/HOWTO-userspace#Does).

Задача l7-filter — определить тип протокола прикладного уровня (см. модель OSI) для данного пакета/соединения. Решается эта задача путем анализа содержимого пакета с применением регулярных выражений, позволяющих определить типовые лексемы, характерные для различных протоколов (например, «`220 ftp server ready`» для FTP или «`HTTP/1.1 200 OK`» для HTTP). Пакет, в котором встречаются такие лексемы, может быть однозначно классифицирован. В принципе, это дает достаточное основание классифицировать соединение в целом. Однако, в этом поведение kernel и userspace версий l7-filter существенно различается.

Как описывается в документации (http://l7-filter.sourceforge.net/technicaldetails), версия l7-filter-kernel *хранит* данные о соединениях и *использует* их для классификации пакетов, принадлежащих к соединениям, тип которых уже установлен. В то же время, аналогичное утверждение в отношении l7-filter-userspace в документации отсутствует.

И, как показывает практика, userspace-версия *не использует* информацию о соединениях. Возможно, это обусловлено техническими ограничениями nfnetlink_queue как средства взаимодействия l7-filter с системой netfilter.

Описанный недостаток значительно снижает эффективность l7-filter — ведь однозначно классифицированы могут быть всего несколько пакетов из каждого соединения, а всего в соединении могут быть миллионы и миллиарды пакетов. Соответственно, применение l7-filter по своему основному назначению — классификация трафика для последующего шейпинга — не оправдывает себя.

Итак, рассмотрим, как, используя возможности netfilter, можно исправить этот недостаток.

Идея решения проста: после обработки пакетов демоном l7-filter, нужно добавить операции по переносу маркировки пакета на соединение (чтобы классифицировать соединение в целом) и с соединения на пакеты (чтобы пометить уже *все* пакеты в соединении для дальнейшей обработки шейпером, так как шейпер воспринимает только метки пакетов). Таким образом, классификация одного пакета в соединении влечет классификацию всех последующих пакетов.

Для начала, запустим демон l7-filter-userspace. Небольшое замечание: в его конфигурационном файле (назовем его, например, `l7-filter.conf`) будем помечать протоколы метками в диапазоне от 16 до 31 включительно (почему — станет понятно из дальнейших пояснений).

```
l7-filter -f /etc/l7-filter.conf -q 2 -m 0x1f
```

Параметр `-f` указывает путь к конфигурационному файлу, `-q` — номер очереди, `-m` — задает биты маркировки, модифицируемые демоном l7-filter (в нашем случае — с первого по пятый, что соответствует диапазону значений маркировки от 0 до 31).

Далее, добавим правила, направляющие весь входящий и исходящий трафик (кроме локального) на анализ демону l7-filter:

```
iptables -t mangle -F # На всякий случай очищаем таблицу mangle
# Направляем на анализ входящий трафик, включая транзитный
iptables -t mangle -A PREROUTING ! -i lo -j NFQUEUE --queue-num 2
# Направляем на анализ исходящий трафик, кроме транзитного
iptables -t mangle -A OUTPUT ! -o lo -j NFQUEUE --queue-num 2
```

Добавлять второе из этих правил в цепочку POSTROUTING не стоило — ведь в нее попадает как трафик, исходящий от самого хоста, так и транзитный трафик, который уже был обработан ранее, в цепочке

PREROUTING. Посмотрев на диаграмму выше, вы можете убедиться, что приведенные правила обрабатывают весь трафик, как принадлежащий самому хосту, так и транзитный, за исключением локального. Локальный трафик (идущий через интерфейс lo), бессмысленно шейпить, а значит, не стоит и классифицировать.

Теперь добавим правила, обеспечивающие копирование маркировки пакетов в маркировку соединений и обратно:

```
# Для входящего трафика (кроме транзитного)
# Копируем маркировку пакетов в маркировку соединений
iptables -t mangle -A INPUT -m mark --mark 0x10/0xfffffff0 -j CONNMARK --save-mark
# И наоборот
iptables -t mangle -A INPUT -m connmark --mark 0x10/0xfffffff0 -j CONNMARK --restore-mark
# Аналогично для исходящего трафика (включая транзитный)
iptables -t mangle -A POSTROUTING -m mark --mark 0x10/0xfffffff0 -j CONNMARK --save-mark
iptables -t mangle -A POSTROUTING -m connmark --mark 0x10/0xfffffff0 -j CONNMARK --restore-mark
```

Поясним два момента. Во-первых, добавление этих правил в цепочки PREROUTING и OUTPUT, сразу после правил, передающих трафик демону l7-filter, не имеет смысла — после обработки пакетов демон применяет (http://l7-filter.sourceforge.net/HOWTO-userspace#Does) к ним действие ACCEPT, прекращающее обработку пакета в рамках исходных цепочек. Поэтому мы добавляем эти правила в цепочки, идущие «ниже по течению». Как вы можете заметить по диаграмме выше, такая комбинация правил также обеспечивает обработку всего трафика.

Второй момент, который стоит пояснить — маски специального вида. По сути, они позволяют проверить, лежит ли маркировка пакета в диапазоне от 16 до 31. Такая защита позволяет избежать обработки маркировок 0 (такую маркировку имеет локальный трафик, так как он не проходит процедуру анализа), 1 и 2 (эти значения маркировки, как уже было замечено выше, означают, что тип пакета не определен), а также 32 и выше (эти значения мы оставляем для других задач).

Как показывает практика (http://www.linux.org.ru/jump-message.jsp?msgid=3790164&cid=3791754), даже в самом примитивном случае (детекция протокола HTTP, сервер — nginx 0.6.32, клиент — wbox (http://www.hping.org/wbox/) 4), эффективность детекции возрастает — без использования маркировки соединений регистрируются лишь 2 исходящих пакета (l7-filter работает на сервере), с использованием — 3 исходящих и 2 входящих. Детальное исследование показывает, что детекции избегают лишь первые четыре пакета в соединении — 2 SYN-пакета и 2 пакета с данными. Это цифры, характерные для тестовой задачи — при передаче больших объемов данных количество детектированных пакетов будет значительно больше, в то время как количество не определенных пакетов сохранит тот же порядок.

Более того, предложенный метод решает задачу, не решенную даже в

реализации l7-filter-kernel — маркировка связанных соединений. Согласно документации iptables, маркировка соединений автоматически копируется с исходных соединений на связанные с ними (например, с управляющего FTP-соединения на соединение данных).

> *Примечание: следующий пример планируется к переносу в еще не написанный раздел статьи (Прочие критерии → statistic).*

В качестве практического примера использования меток пакетов и соединений можно рассмотреть задачу стохастической балансировки соединений между несколькими аплинками. Допустим, у нас есть три провайдера, подключенных к интерфейсам eth0, eth1 и eth2 (это могут быть и VLAN-порты одного интерфейса, суть от этого не меняется, только названия), и их шлюзы имеются соответственно IP-адреса 208.77.188.1, 208.77.189.1, 208.77.190.1.

Для начала, создадим для каждого провайдера свою таблицу маршрутизации

```
echo -e "\n110\tstatic\n111\tprov1\n112\tprov2\n113\tprov3" >> /etc/iproute2/rt_tables
```

Этот код добавит в конец файла `/etc/iproute2/rt_tables` строки

```
110    static
111    prov1
112    prov2
113    prov3
```

устанавливающие соответствие между внутренними номерами таблиц маршрутизации и их символьными именами. Используемые здесь имена prov1, prov2 и prov3, разумеется, условны. Таблица static — особая, ее мы рассмотрим чуть ниже.

Обратите внимание, что это действие выполняется только один раз — не надо повторять его при каждой загрузке системы!

Далее, сделаем каждого провайдера шлюзом по умолчанию в «своей» таблице:

```
ip route add default via 208.77.188.1 dev eth0 table prov1
ip route add default via 208.77.189.1 dev eth1 table prov2
ip route add default via 208.77.190.1 dev eth2 table prov3
```

Добавим для каждой таблицы правило, отправляющее в нее пакеты с

соответствующей маркировкой:

```
ip rule add fwmark 1 table prov1
ip rule add fwmark 2 table prov2
ip rule add fwmark 3 table prov3
# Но прежде всего пакеты должны пройти таблицу static
ip rule add table static prio 1
```

Таблица static предназначена для обслуживания статических маршрутов. В частности, в нее мы занесем подсети провайдеров (предположим, что все они класса 1C), а также наши внутренние локальные сети (если таковые есть):

```
# Провайдеры
ip route add 208.77.188.0/24 dev eth0 table static
ip route add 208.77.189.0/24 dev eth1 table static
ip route add 208.77.190.0/24 dev eth2 table static
# Две наших локалки
ip route add 192.168.1.0/24 dev eth3 table static
ip route add 192.168.2.0/24 dev eth4 table static
# Сбрасываем кеш маршрутов
ip route flush cache
```

Таким образом, если нашему хосту нужно будет обратиться в подсеть провайдера prov2 (208.77.189.0/24), то маршрут пойдет сразу через интерфейс eth1. Также в этой таблице присутствуют маршруты для наших внутренних локальных сетей — с ними тоже все просто.

По сути дела, таблица static обычно содержит те же маршруты, что и таблица main (главная таблица маршрутизации), за исключением маршрута по умолчанию — таких маршрутов у нас несколько и каждый из них размещается в отдельной таблице, выбор между которыми осуществляется на основании назначенной iptables/netfilter маркировки.

Заметим, что ни в таблицу static, ни в какие-либо другие таблицы не нужно вносить loopback-маршруты, например «127.0.0.1/8 dev lo», так как все эти маршруты фигурируют в автоматически создаваемой таблице local, которую любой пакет проходит в первую очередь (нетрудно убедиться в этом, посмотрев вывод команды «ip rule show»).

Далее, отключим статическую антиспуфинговую фильтрацию:

```
sysctl net.ipv4.conf.all.rp_filter=0
```

Reverse path filtering — штука, конечно, удобная и полезная но, к сожалению, совершенно не совместимая с динамической маршрутизацией.

Если вы планируете использовать этот компьютер не только как шлюз, и но и как интернет-сервер (то есть предоставлять доступ к нему извне), необходимо выполнить привязку входящих соединений к их интерфейсам — в противном случае могут возникнуть проблемы, если обращение извне придет через одного провайдера, а сервер ответит через другого.

```
iptables -t mangle -N bind_connect # Создаем отдельную цепочку (для простоты управления)
# Следите за правильным соответствием значений меток и интерфейсов!
iptables -t mangle -A bind_connect -i eth0 -j CONNMARK --set-mark 1
iptables -t mangle -A bind_connect -i eth1 -j CONNMARK --set-mark 2
iptables -t mangle -A bind_connect -i eth2 -j CONNMARK --set-mark 3
# Пропускаем через эту процедуру все новые соединения к нашему серверу
iptables -t mangle -I INPUT -m conntrack --ctstate NEW -j bind_connect
```

Теперь, в сочетании с операцией `-j CONNMARK --restore-mark`, которой мы подвергнем исходящий с нашего сервера трафик (см. ниже), эта процедура обеспечит корректную обработку входящих соединений. (Отметим, что, если бы нам не нужно было бы балансировать исходящие соединения, мы могли бы обойтись вообще без помощи iptables/netfilter, выполнив привязку входящих соединений через правила вида `ip rule add from 208.77.188.100 table prov1` и т.п. — ответные пакеты всегда уходят с того же адреса, на который пришел запрос, так что в качестве критерия для выбора шлюза можно использовать исходный адрес.)

Ввиду того, что выбор исходящего адреса для каждого нового соединения осуществляется на основании правил статической маршрутизации (таблица main), могут возникнуть ошибки. Например, если в маршруте по умолчанию (default) в таблице main указан интерфейс eth0, то все исходящие от нас во внешнюю сеть (интернет) соединения будут иметь в качестве исходного адреса первый адрес интерфейса eth0, и ответные пакеты пойдут именно на этот интерфейс. Чтобы избежать возникновения таких ситуаций, добавим маскарадинг для всех исходящих соединений (предполагается, что у всех провайдеров наши внешние адреса имеют вид 208.77.x.100):

```
iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to-source 208.77.188.100
iptables -t nat -A POSTROUTING -o eth1 -j SNAT --to-source 208.77.189.100
iptables -t nat -A POSTROUTING -o eth2 -j SNAT --to-source 208.77.190.100
```

А теперь — самое интересное. Используя описанный ниже критерий statistic, мы будем случайно распределять метки между пакетами.

```
iptables -t mangle -N select_prov # Создаем для этого специальную цепочку
iptables -t mangle -A select_prov -j CONNMARK --set-mark 1 # Ставим всем соединениям маркировку 1
iptables -t mangle -A select_prov -m statistic --mode random --probability 0.34 -j RETURN # С верс
```

```
iptables -t mangle -A select_prov -j CONNMARK --set-mark 2 # Ставим всем оставшимся соединениям ма
iptables -t mangle -A select_prov -m statistic --mode random --probability 0.5 -j RETURN # С вероя
iptables -t mangle -A select_prov -j CONNMARK --set-mark 3 # Всем, кто дошел досюда, ставим маркир
```

Первое правило в этой цепочке пройдут все пакеты, вошедшие в нее. После этого, 34 % (примерно треть из них) покинет цепочку согласно второму правилу. Далее, оставшиеся пакеты (66 % от первоначального количества) получат маркировку 2. После этого половина из них (то есть 33 % от начального) покинут цепочку с этой маркировкой. Оставшаяся половина (тоже 33 % от начального количества) получат маркировку 3.

После этого, создадим вспомогательную цепочку, осуществляющую маркировку соединений и пакетов:

```
iptables -t mangle -N sort_connect
iptables -t mangle -A sort_connect -o lo -j RETURN # Локальным соединениям балансировка не нужна
# Аналогично выгоняем пакеты, выходящие через интерфейсы внутренней локалки, если она есть
iptables -t mangle -A sort_connect -o eth3 -j RETURN
iptables -t mangle -A sort_connect -o eth4 -j RETURN
iptables -t mangle -A sort_connect -m conntrack --ctstate NEW -j select_prov # Все новые пакеты пр
iptables -t mangle -A sort_connect -j CONNMARK --restore-mark # Копируем маркировку соединений на
```

Через цепочку `select_prov` мы прогоняем только новые пакеты, то есть первые пакеты каждого соединения. После этой процедуры соединение уже имеет маркировку. К сожалению, на данный момент роутинговая подсистема ядра Linux не умеет маршрутизировать пакеты на основании маркировки соединения — только на основании маркировки пакетов. Поэтому действием `CONNMARK --restore-mark` мы копируем маркировку соединений в маркировку пакетов.

Осталось только добавить вызов этой цепочки в таблицу mangle:

```
iptables -t mangle -I OUTPUT -j sort_connect
```

Теперь все ваши исходящие соединения будут балансироваться согласно описанным правилам.

Аналогичную функциональность можно реализовать и для транзитных соединений. Для этого достаточно добавить вызов `sort_connect` в цепочку FORWARD таблицы mangle:

```
iptables -t mangle -I FORWARD -j sort_connect
```

Разумеется, при этом должна быть разрешена передача транзитного трафика, как в таблице filter, так и на уровне sysctl. Как это делается — см. выше.

Заметим, что кроме алгоритма случайной балансировки, критерий statistic позволяет реализовать балансировку в режим round robin. Для этого поменяем цепочку `select_prov` следующим образом:

```
iptables -t mangle -F select_prov # Очищаем ее
iptables -t mangle -A select_prov -j CONNMARK --set-mark 1 # Ставим всем соединениям маркировку 1
iptables -t mangle -A select_prov -m statistic --mode nth --every 3 -j RETURN # Первый из трех пак
iptables -t mangle -A select_prov -j CONNMARK --set-mark 2 # Ставим всем оставшимся соединениям ма
iptables -t mangle -A select_prov -m statistic --mode nth --every 2 -j RETURN # Один из оставшихся
iptables -t mangle -A select_prov -j CONNMARK --set-mark 3 # Последний
```

Подробнее о принципах работы критерия statistic см. ниже.

В завершение нашего обсуждения стоит заметить, что в некоторых случаях для балансировки соединений достаточно единственной команды

```
ip route add default scope global nexthop via 208.77.188.1 dev eth0 weight 1 nexthop via 208.77.18
```

Стоит обратить особое внимание на тот факт, что данный метод балансирует *пакеты*, а не соединения. Например, в том случае, если вы хотите организовать выход в интернет из локальной сети через нескольких провайдеров, не имея единого внешнего адреса, этот метод может работать некорректно — часть пакетов пройдет через одного провайдера и после операции NAT получит один исходный адрес, часть пойдет через другого и соответственно получит другой адрес, и в результате удаленные хосты не смогут правильно обрабатывать соединения, исходящие из вашей сети. Однако, в большинстве случаев этот негативный эффект нивелируется другим фактором — кэшированием маршрутов. При прохождении через такое правило серии пакетов, адресованных некоторому хосту, действительно случайным выбор будет только для первого из них, после чего выбранный маршрут будет закэширован, и остальные пакеты к этому хосту будут маршрутизироваться через тот же шлюз. С одной стороны, подобный эффект позволяет соединениям корректно функционировать, с другой стороны — балансировка оказывается не такой уж и случайной. К тому же, после очистки кэша маршрутов (например, посредством ввода команды `ip route flush cached`), работа существующих на этот момент соединений может быть нарушена. В качестве наиболее безопасного и целесообразного применения описанного метода можно привести задачу балансировки транзитного трафика в сетях без NAT (условия «прямой видимости» между балансирующим маршрутизатором и точкой схождения потоков трафика). В том случае, если доступ к шлюзам осуществляется через один сетевой интерфейс,

этим методом можно балансировать и соединения, исходящие от самого хоста.

Также заметим, что описанные в этом примере методы балансировки не учитывают загруженность каналов. Для этого рекомендуется использовать критерий rateest и действие RATEEST совместно с уже знакомым нам CONNMARK.

Лимитирующие критерии

К этой группе относятся критерии, позволяющие ограничивать количество пакетов, соединений, и переданных байт.

- **limit** — позволяет ограничить количество пакетов в единицу времени. Параметры:

`--limit` *`количество`*`[/second|/minute|/hour|/day]` — задает ограничение на количество пакетов в секунду (second), минуту (minute), час (hour) или сутки (day). Пакеты в пределах этого количества считаются удовлетворяющими критерию, сверх этого количества — не удовлетворяющими.

`--limit-burst` *`количество`* — задает длину очереди, то есть максимальную пропускную способность.

Критерий -limit использует модель «дырявого ведра», и --limit-burst задает «объем ведра», а --limit — «скорость вытекания». Каждому такому критерию соответствует своя очередь, длина которой задается параметром --limit-burst. Если в очереди есть пакеты, то со скоростью, заданной в --limit, они покидают очередь и считаются удовлетворяющими критерию. Если же вся очередь занята, то новые пакеты в ней не регистрируются и считаются *не* удовлетворяющими критерию. Например,

```
iptables -I INPUT -m limit --limit 3/min --limit-burst 5 -j LOG --log-level debug --log-prefix "IN
```

предполагает очередь на пять пакетов, которая «продвигается» со скоростью 3 пакета в минуту. При непрерывном поступлении входящих пакетов, очередь всегда будет заполнена, и в лог будут заноситься в среднем по три пакета в минуту. Однако, если входящих пакетов долго не будет, то очередь успеет очиститься, и при поступлении пяти и менее новых пакетов, они пойдут в лог подряд. В любом случае, скорость попадания пакетов в лог остается неизменной.

Типичная ошибка новичков — использовать limit для ограничения TCP-трафика, например, так:

```
```

```
iptables -A INPUT -p tcp --dport 80 -m limit --limit 10000/sec --limit-burst 10000 -j ACCEPT
iptables -P INPUT DROP
```

Это пример попытки защитить web-сервер от DDoS-атаки, ограничив количество пакетов в единицу времени. Однако, это правило не помешает без особого труда завалить сервер запросами (считая, что на один запрос требуется два входящих пакета — SYN-пакет и пакеты данных, содержащий, например, только GET /, согласно спецификации HTTP 0.9). При этом могут возникнуть помехи для легальных пользователей, например, загружающих на сервер большой файл методом POST. Более корректным решением будет ограничивать не скорость входящего потока данных, а скорость открытия новых соединений:

```
iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -m limit --limit 32/sec --limit-bur
iptables -P INPUT DROP
```

Теперь мы ограничиваем количество не всех пакетов, а только новых, то есть мы разрешаем открывать не более 32 новых соединений в секунду. Впрочем, число 32 приведено здесь только для примера. Конкретное значение скорости для вашей задачи рекомендуем определять самостоятельно.

- **hashlimit** — позволяет применять ограничения, аналогичные критерию limit, к группам хостов, подсетей или портов, используя всего одно правило. При этом для каждого хоста, подсети или порта создается отдельная очередь. Можно задвать, например, такие критерии:

«1000 пакетов в секунду с каждого хоста из подсети 192.168.0.0/16»

```
-s 192.168.0.0/16 -m hashlimit --hashlimit-upto 1000/sec --hashlimit-mode srcip
```

«100 пакетов в секунду с каждого TCP-порта хоста 192.168.1.1»

```
-s 192.168.1.1 -m hashlimit --hashlimit-upto 100/sec --hashlimit-mode srcport
```

«10000 пакетов в минуту с каждой подсети префикса 28 (маска 255.255.255.240) из диапазона 10.0.0.0/8»

```
```

```
-s 10.0.0.0/8 -m hashlimit --hashlimit-upto 10000/min --hashlimit-mode srcip --hashlimit-srcmask 2
```

Обратите внимание, что в *каждом* из этих правил нужно обязательно указать имя таблицы очередей (haslimit-name, см. ниже), однако, в целях простоты изложения, в некоторых примерах этот параметр опущен.

Рассмотрим основные параметры этого критерия более подробно:

`--hashlimit-mode {srcip|srcport|dstip|dstport}[,...]` — задает список контролируемых параметров: адреса (ip) и порты (port) источника (src) и назначения (dst). Например, если указать параметр

```
--hashlimit-mode srcip,dstip
```

то будет создаваться отдельная очередь для каждой пары «адрес источника — адрес назначения», то есть ограничение будет вводиться на количество пакетов, передаваемых с каждого хоста на другой хост (разумеется, если эти пакеты идут через наш сервер). Или, например,

```
--hashlimit-mode srcip,srcport
```

будет создавать отдельную очередь для каждого исходного порта каждого хоста.

`--hashlimit-upto` *`количество`*`[/second|/minute|/hour|/day]` — этот параметр задает скорость движения очереди (аналог `--limit`). Если пакеты поступают с такой же или меньшей скоростью, они считаются подпадающими под критерий. Обратите внимание, что этот параметр *не* поддерживает логической инверсии (отрицания) через восклицательный знак (см. ниже).

`--hashlimit-above` *`количество`*`[/second|/minute|/hour|/day]` — этот параметр таке задает скорость движения очереди, но смысл его противоположный — если пакеты поступают с *большей* скоростью, они считаются подпадающими под критерий.

`--hashlimit-burst` *`количество`* — задает длину каждой очереди. Аналогичен `--limit-burst`.

`--hashlimit-srcmask` *`префикс`* — задает размер подсети исходных адресов, для которой вводится своя очередь. Имеет смысл, только если в haslimit-mode указан режим srcip. Как уже говорилось выше,

```

```

```
-m hashlimit --hashlimit-mode srcip --hashlimit-srcmask 28
```

будет заводить отдельную очередь для каждой подсети с маской 255.255.255.240 (подробнее про префиксы и маски см. CIDR). По умолчанию 32 (своя очередь для каждого отдельного хоста). Если указать 0, то контроль по параметру srcip теряет смысл.

`--hashlimit-dstmask` *префикс* — задает размер подсети исходных адресов, для которой вводится своя очередь. Имеет смысл, только если в haslimit-mode указан режим dstip. Принцип использования аналогичен hashlimit-srcmask.

`--hashlimit-name` *имя* — позволяет использовать несколько независимых таблиц очередей (hashes), распознаваемых по имени. Обязательный параметр. Например,

```
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Пропускаем все, что идет
# Вводим ограничения для новых подключений по FTP
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 21 -m hashlimit --hashlimit-upto 5/mir
# Вводим ограничения для новых подключений по rsync.
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 873 -m hashlimit --hashlimit-upto 5/mi
iptables -P INPUT DROP # Всех остальных не пускаем
```

Теперь очереди ограничений для подключения к службам FTP и rsync для нашего сервера будут вводиться независимо, то есть если какой-либо хост превысит предел подключений по FTP, то это никак не повлияет на подключения по rsync. Заметим, что в данном конкретном случае аналогичную функциональность можно реализовать, даже не используя разные хеши:

```
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Пропускаем все, что идет
# Вводим ограничения для новых подключений по FTP и rsync
iptables -A INPUT -m conntrack --ctstate NEW -p tcp -m multiport --dport 21,873 -m hashlimit --has
iptables -P INPUT DROP # Всех остальных не пускаем
```

Впрочем, выбирая между этими двумя реализациями, стоит учитывать, что размер каждого хеша ограничен.

Не забывайте указывать имя таблицы очередей при каждом использовании критерия hashlimit. Если вы не планируете совместно использовать одни и те же таблицы в нескольких правилах, эти имена должны различаться.

Доступ к таблицам очередей возможен через procfs (файл `/proc/net/ipt_hashlimit/имя_таблицы`). Подробнее о формате этого файла можно почитать здесь (http://www.opennet.ru/base/net/iptables_tbf.txt.html), однако учтите,

что большая часть сведений, приведенных в указанной статье, устарела.

Критерий haslimit также имеет ряд параметров, не описанных здесь (максимальный размер таблицы, время жизни записи в таблице, интервал «сбора мусора», и т. п.). За подробными сведениями обратитесь к документации.

- **connlimit** — позволяет ограничивать количество одновременно открытых соединений с каждого IP-адреса (или подсети). Параметры:

`[!] --connlimit-above количество` — минимальное количество соединений. Все пакеты, проходящие по соединениям, установленным сверх заданного количества, считаются удовлетворяющими критерию. Например,

```
iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 3 -j DROP
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

разрешит не более трех одновременных соединений к нашему веб-серверу с одного IP-адреса. Аналогичного эффекта можно добиться командами

```
iptables -A INPUT -p tcp --dport 80 -m connlimit ! --connlimit-above 3 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j DROP
```

`--connlimit-mask` — маска подсети, для которой устанавливается общее ограничение на количество соединений. Например,

```
iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 5 --connlimit-mask 24 -j DROP
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

разрешит не более пяти одновременных соединений на порт 80 с каждой подсети класса 1C.

Не надо путать connlimit с hashlimit — hashlimit позволяет ограничить *скорость* поступления пакетов с хостов или подсетей (в сочетании с `-m conntrack --ctstate NEW` — скорость открытия новых соединений), а connlimit — *количество* одновременно открытых соединений.

- **connbytes** — позволяет ограничивать количество переданных байт или пакетов. Параметры:

`[!] --connbytes` *мин*`:[`*макс*`]` — собственно задает количество пакет или байт. Можно указать просто минимальное количество или диапазон.

`--connbytes-dir {original|reply|both}` — задает направление движения пакетов. Проверяются только пакеты, движущиеся от инициатора соединения к отвечающему (original), наоборот (reply) или в обоих направлениях (both).

`--connbytes-mode {packets|bytes|avgpkt}` — задает собственно учитываемую величину: количество пакетов (packets), количество байт (bytes) или средний размер пакета (avgpkt).

Пример:

```
# Для соединений, по которым передано более 500 КБ, ставим минимальный приоритет TOS (требование м
iptables -t mangle -I INPUT -m connbytes --connbytes 512000: --connbytes-dir both --connbytes-mode
iptables -t mangle -I OUTPUT -m connbytes --connbytes 512000: --connbytes-dir both --connbytes-moc
# Для соединений, по которым передано менее 50 КБ, ставим максимальный приоритет TOS (требование м
iptables -t mangle -I INPUT -m connbytes ! --connbytes 51200: --connbytes-dir both --connbytes-moc
iptables -t mangle -I OUTPUT -m connbytes ! --connbytes 51200: --connbytes-dir both --connbytes-mc
```

Таким образом, все пакеты, принадлежащие «легким» соединениям, будут маркироваться как наиболее приоритетные, а пакеты, проходящие по «тяжеловесным» соединениям — как наименее приоритетные. Такой подход позволит рационально распределять канал между различными задачами, например, просмотром веб-страниц («легкие» соединения) и скачиванием файлов («тяжелые» соединения).

Конечно, при таком подходе даже «тяжелые» соединения будут занимать канал с наивысшим приоритетом, пока не превысят ограничение. Но даже на медленных соединениях (128 Кбит/с) для этого достаточно всего четырех секунд (если эта закачка в данный момент единственная). После превышения лимита 50 Кбайт соединение получит средний приоритет (TOS не установлен), а еще через некоторое время соединение превысит второй лимит — 500 Кбайт, и получит минимальный приоритет. В то же время, «легкие» соединения в большинстве случаев не превышают даже первого лимита, и поэтому целиком проходят с повышенным приоритетом.

- **quota** — позволяет задать квоту в байтах для данного конкретного правила. Имеет единственный параметр

`--quota количество_байт` — собственно задает квоту.

Для каждого правила, содержащего такой критерий, создается счетчик, который уменьшается с каждым прошедшим по этому правилу пакетом на количество байт, равное размеру этого пакета. Когда значение счетчика доходит до нуля, правило перестает срабатывать. Чтобы восстановить счетчик, удалите правило и добавьте его снова.

Например:

```

```

```
sysctl net.ipv4.ip_forward=1 # Разрешаем шлюзу передавать транзитный трафик
iptables -F FORWARD # Очищаем цепочку FORWARD
iptables -A FORWARD -m quota --quota 1073741824 -j ACCEPT
iptables -P FORWARD DROP
```

разрешает хосту передавать не более одного гигабайта транзитного трафика (учитывается трафик в обоих направлениях).

- **quota2** — критерий из уже упоминавшегося набора xtables-addons. Является результатом развития идей, заложенных в критерии quota. Позволяет создавать именованные счетчики квот (для совместного использования одного счетчика в нескольких правилах), считать не только байты, но и пакеты, а также создавать отдельные правила для пакетов, превышающих квоту (логическая инверсия параметра `--quota`). Поддерживаются следующие параметры:

`--name` *имя* — задает имя счетчика. Текущее значение счетчика содержится в файле `/proc/net/xt_quota/`*имя*. Его можно вывести на экран

```
cat /proc/net/xt_quota/имя
```

или задать вручную

```
echo значение > /proc/net/xt_quota/имя
```

Для осуществления этих действий, как и для работы с iptables, нужны полномочия суперпользователя.

`[!] --quota` *количество* — проверяет текущее значение счетчика и уменьшает его на размер текущего пакета. Этот же параметр задает начальное значение счетчика (если счетчик с таким именем уже существует, он не сбрасывается). Пакет считается подпадающим под критерий, если счетчик еще не дошел до нуля (при указании логической инверсии — наоборот). Если немного усложнить наш предыдущий пример,

```
sysctl net.ipv4.ip_forward=1 # Разрешаем шлюзу передавать транзитный трафик
iptables -F FORWARD # Очищаем цепочку FORWARD
# Устанавливаем квоту для первой подсети
iptables -A FORWARD -s 192.168.1.0/24 -m quota2 --name first --quota 1073741824 -j ACCEPT
# Для тех, кто идет из первой подсети сверх квоты - блокируем и сообщаем об отказе.
# Кстати, счетчик first уже создан, поэтому теперь в параметре quota можно писать любое число. Но
iptables -A FORWARD -s 192.168.1.0/24 -m quota2 --name first ! --quota 1073741824 -j REJECT
# Устанавливаем квоту для второй подсети
iptables -A FORWARD -s 192.168.2.0/24 -m quota2 --name second --quota 536870912 -j ACCEPT
iptables -P FORWARD DROP
```

Теперь для подсети 192.168.1.0/24 установлена квота в 1 Гб, и при превышении этой квоты пакеты блокируются, причем их отправителям хост отвечает пакетами icmp-port-unreachable, уведомляя их, что передача данных заблокирована. Для подсети 192.168.2.0/24 устанавливается квота 512 Мб, и при ее превышении никаких сообщений не посылается (пакеты блокируются «молча»). Так же молча блокируются транзитные пакеты из всех остальных подсетей.

`--packets` — при указании этого параметра счетчик учитывает количество пакетов, а не их суммарный размер.

`--grow` — *увеличивает* счетчик, вместо того, чтобы уменьшать его. Критерий с этим параметром всегда возвращает истину. Позволяет создавать счетчики «приходно-расходного типа». Например,

```
-A INPUT -p tcp --dport 6881 -m quota2 --name bt --grow
-A OUTPUT -p tcp --sport 6881 -m quota2 --name bt
```

позволяет в любой момент определять разность сумм входящих (на TCP-порт 6881) и исходящих (с этого же порта) байт через файл `/proc/net/xt_quota/bt`.

Вот еще один пример (http://jengelh.medozas.de/2009/0717-bandwidth-monitoring.php) использования этого критерия для сбора статистики по трафику.

Обратите внимание, что при указании параметра grow критерий quota2 *всегда* будет возвращать истину. Действие для такого правила обычно назначать не имеет смысла — правило изменяет только счетчик квоты, а не сам пакет.

- **length** — позволяет использовать в качестве критерия размер пакета. Проверятся размер пакета протокола транспортного уровня (TCP, UDP, SCTP, DCCP и т. д.) или, иначе говоря, размер пакета протокола сетевого уровня (IP, IPv6) за вычетом размера заголовков сетевого уровня. Данный критерий имеет единственный параметр

`[!] --length` *размер*`[:`*макс_размер*`]`, который может употребляться в нескольких формах:

`--length` *размер* — пакет проверяется на точное соответствие заданному размеру,

`--length` *мин_размер*`:` — размер пакета должен быть больше или равен заданной величине,

`--length :`*макс_размер* — размер пакета должен быть меньше или равен заданной величине,

`--length` *мин_размер*`:`*макс_размер* — размер пакета должен попадать в указанный диапазон (включая границы).

Смысл всех приведенных форм может быть заменен на противоположный, как обычно, указанием перед данным параметром восклицательного знака. Все размеры задаются в байтах.

В качестве практического примера использования данного критерия можно рассмотреть установку высокого приоритета TOS (требование минимальной задержки) для небольших транзитных пакетов (размер до 512 байт):

```
iptables -t mangle -I INPUT -m length --length :512 -j TOS --set-tos Minimize-Delay
```

- **length2** — расширение критерия length, доступное в наборе xtables-addons. Отличается от классического length возможностью проверять размеры пакетов на различных уровнях модели OSI. Принимает следующие опции:

`--layer3` — проверка размера пакета сетевого уровня (например, IPv4-пакета).

`--layer4` — проверка размера пакета транспортного уровня (например, TCP-пакета).

`--layer5` — проверка размера полезной нагрузки транспортного уровня (например, содержимого TCP-пакета, *не* учитывая размер заголовков TCP). Работает корректно только для некоторых протоколов транспортного уровня, на момент написания этих строк (конец 2009) поддерживаются: TCP, UDP, UDPLite (en), ICMP, ICMPv6 (en), DCCP, SCTP, IPSec (AH и ESP). Для SCTP-пакета в этот размер включаются все секции (chunks) вместе с их заголовками.

`--layer7` — для SCTP проверяется суммарный размер DATA-секций в пакете (то есть его полезная нагрузка), для остальных протоколов аналогично `--layer5`.

По умолчанию используется `--layer3`, однако во избежание недоразумений рекомендуется явно указывать одну из перечисленных опций. Если этого не делать, при вводе соответствующей команды появится сообщение

```
iptables: length match: Defaulting to --layer3. Consider specifying it explicitly.
```

Это *не* является ошибкой (только предупреждением о возможной неоднозначности). Заданное вами правило должно добавиться успешно.

`[!] --length` *размер*`[:`*макс_размер*`]` — собственно, задает нужные значения размера. В отличие от аналогичного параметра в классическом критерии length, его синтаксические формы несколько беднее. Допускаются два варианта записи:

> `--length` *размер* — пакет проверяется на точное соответствие заданному размеру,
>
> `--length` *мин_размер*`:`*макс_размер* — размер пакета должен быть попадать в указанный диапазон (включая границы).

Впрочем, недостающие варианты легко выразить через имеющиеся: для имитации `--length :`*макс_размер* используйте `--length 0:`*макс_размер*, а вместо `--length` *мин_размер*`:` применяйте `--length` *мин_размер*`:65535`.

Будьте внимательны: в отличие от length, length2 не проверяет взаимное соответствие границ диапазона (*мин_размер* <= *макс_размер*), и если вы зададите их неправильно (*мин_размер* > *макс_размер*), ваше правило просто никогда не будет срабатывать.

Подводя краткий итог по разделу «Лимитирующие критерии», хотелось бы заметить, что перечисленные в этом разделе критерии предназначены главным образом для ограничения доступа и защиты от различных атак. Не стоит пытаться ограничивать с их помощью трафик. Для ограничения и приоритезации трафика в Linux рекомендуется использовать стандартный шейпер ядра, управляемый при помощи утилиты tc.

Критерий recent

recent — это специальный критерий, позволяющий запоминать проходящие через него пакеты, а затем использовать полученную информацию для принятия решений. Ввиду его уникальности, широких возможностей и, как следствие, некоторых трудностей в понимании новичками принципов его использования, рассказ о нем был вынесен в отдельный подраздел.

Если быть точным, recent запоминает не сами пакеты, а их количество, время поступления, адрес источника (в последних версиях iptables также может запоминать и адрес назначения), а также, при необходимости, TTL.

В начале кратко рассмотрим его опции:

`[!] --set` — запомнить адрес источника/назначения пакета (внести его во внутренний список). Если такая запись уже присутствует в списке —

обновить время последнего доступа для нее. Обратите внимание, что критерию recent с опцией `--set` удовлетворяют *все* пакеты. Чтобы ему не удовлетворял ни один пакет — используйте логическую инверсию (`! --set`).

`[!] --rcheck` — позволяет проверить, присутствует ли адрес источника/назначения пакета во внутреннем списке. Критерий recent с этой опцией вернет истину, если адрес в списке присутствует.

`[!] --update` — работает аналогично `--rcheck`, но еще и обновляет время последнего доступа для данной записи.

`[!] --remove` — работает аналогично `--rcheck`, но еще и удаляет найденную запись из списка. Если запись не найдена, пакет считается *не* соответствующим критерию.

`--seconds` *число* — дополнительная опция в режимах `--rcheck` и `--update`. Пакет считается соответствующим критерию, только если последний доступ к записи был не позднее, чем ч*исло* секунд назад.

`--hitcount` *число* — дополнительная опция в режимах `--rcheck` и `--update`. Пакет считается удовлетворяющим критерию, если было не менее *число* обращений к данной записи. Обычно используется вместе с `--seconds` — тогда критерий имеет смысл «не менее n обращений за последние m секунд».

Заметим, что, хотя в официальной документации для параметров `--seconds` и `--hitcount` указывается возможность отрицания (указания перед ними восклицательного знака), на самом деле такое отрицание никак не обрабатывается в коде и не меняет смысла параметров, так что указывать его бессмысленно. Эта ошибка в документации исправлена в версии iptables 1.4.7.

`--rttl` — дополнительно к адресу источника/назначения, заносить в список и проверять еще и TTL пакета.

`--rsource` — эта опция появилась в последних версиях iptables, с тех пор, как критерий recent начал поддерживать запоминание адресов не только источника, но и назначения. Позволяет явно указать, что в список вносится именно адрес источника пакета. Однако в целях обратной совместимости этот режим используется по умолчанию, и поэтому указывать данную опцию не обязательно.

`--rdest` — эта опция появилась в последних версиях iptables. Позволяет явно указать, что в список вносится именно адрес назначения пакета.

`--name` *имя* — позволяет указать имя списка при использовании нескольких списков. По умолчанию используется список DEFAULT. Каждый список представлен псевдофайлом `/proc/net/xt_recent/`*имя* (на старых ядрах критерий recent реализован только для IPv4, но не для IPv6, поэтому вместо `xt_recent`

будет `ipt_recent`). В частности, вы можете:

```
cat /proc/net/xt_recent/имя # вывести список на экран
echo +адрес > /proc/net/xt_recent/имя # добавить адрес в список
echo -адрес > /proc/net/xt_recent/имя # удалить адрес из списка
echo / > /proc/net/xt_recent/имя # очистить список
```

Теперь давайте рассмотрим несколько примеров.

- Блокирование bruteforce-атак (подбор пароля вслепую) на SSH и аналогичные сервисы.

```
iptables -N ssh_brute_check # Создаем цепочку для проверки попыток соединений на защищаемый порт
# Если за последние 10 минут (600 секунд) с одного адреса было 3 или более новых соединений — блок
iptables -A ssh_brute_check -m conntrack --ctstate NEW -m recent --update --seconds 600 --hitcount
# В противном случае — разрешаем, и при этом заносим в список
iptables -A ssh_brute_check -m recent --set -j ACCEPT
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Разрешаем пакеты по устан
# Все попытки открыть новое соединение по SSH направляем на проверку
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 22 -j ssh_brute_check
# Здесь можно разрешать те порты, для которых такая проверка не нужна. Например, HTTP
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 80 -j ACCEPT
iptables -P INPUT DROP # Что не разрешено — то запрещено
```

Теперь все попытки открыть новое SSH-соединение проверяются, и с одного IP-адреса можно открывать не более 2 соединений за 10 минут. Обратите внимание, что за одно соединение злоумышленник может проверить несколько паролей — число попыток аутентификации до обрыва соединения задает параметр MaxAuthTries в файле `/etc/ssh/sshd_config`. По умолчанию это число равно 6, так что в нашем примере злоумышленник сможет проверять не более 12 паролей за 10 минут.

Впрочем, данный пример весьма тривиален, и сходную функциональность можно получить и при помощи критерия hashlimit:

```
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 22 -m hashlimit --hashlimit-mode srcip
```

разрешит не более 5 новых соединений в час. Данная реализация имеет недостаток по сравнению с recent — вы не можете произвольно задавать временной период.

- Защита от сканирования портов (заметим, что кроме критерия recent для этой задачи можно также использовать критерии psd и lscan из комплекта xtables-addons).

```
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Разрешаем пакеты по устан
# Если за последний час было 10 или более запросов на нерабочие порты — блокируем
iptables -A INPUT -m recent --rcheck --seconds 3600 --hitcount 10 --rttl -j RETURN
# Если за последнюю минуту было 2 или более запросов на нерабочие порты — блокируем
iptables -A INPUT -m recent --rcheck --seconds 60 --hitcount 2 --rttl -j RETURN
# Разрешаем рабочие порты
iptables -A INPUT -m conntrack --ctstate NEW -p tcp -m multiport --dport 21,25,53,80,110 -j ACCEPT
iptables -A INPUT -m conntrack --ctstate NEW -p udp -m multiport --dport 53,123 -j ACCEPT
# Всех, кто ломится в нерабочие порты — регистрируем
iptables -A INPUT -m recent --set
iptables -P INPUT DROP # Что не разрешено — то запрещено
```

Все пакеты, попадающие на нерабочие порты сервера, регистрируются. После нескольких таких попыток за заданный интервал времени адрес их источника блокируется. Проверка TTL добавлена для защиты от блокировки легитимных клиентов после спуфинга злоумышленником от их имени. Хотя, с другой стороны, эта мера позволяет ему обойти защиту от сканирования, выставляя своим пакетам различный TTL. Так что вопрос о нужности этой проверки в вашем конкретном случае вам придется решать самостоятельно.

Другой пример использования критерия recent для детекции и блокирования сканирования портов:

```
# Создаем цепочку для проверки и блокирования портсканов
iptables -N portscan_check
# Блокируем тех, кто был уличен в портскане в течение последних суток
iptables -A portscan_check -m recent --name portscan --rcheck --seconds 86400 -j DROP
# По прошествии суток можно удалять их из списка
iptables -A portscan_check -m recent --name portscan --remove
# Ну а сейчас — собственно процедура определения портскана
iptables -A portscan_check -p tcp --dport 139 -m recent --name portscan --set -j DROP
# Пускаем на проверку всех, кто пришел к нам из интернета (интерфейс eth0)
iptables -I INPUT -i eth0 -j portscan_check
```

В данном случае сканированием портов считается обращение на порт 139/tcp (SMB). Впрочем, вы можете задать другой порт или, используя критерий multiport, даже список портов. В отличие от предыдущего примера, здесь производится автоматическая очистка списка IP-адресов: по прошествии суток с момента блокировки, первое же обращение с заблокированного адреса на наш сервер приводит к удалению этого адреса из нашего recent-списка. Однако, если это обращение вновь является попыткой сканирования портов, то есть направлено на порт 139/tcp, адрес тут же блокируется вновь. Именно поэтому процедура блокировки расположена «ниже по течению», чем процедура удаления адреса из списка.

Для предыдущего примера реализация очистки списка адресов достигается следующим образом:

```
# Создаем цепочку для проверки на необходимость удаления
iptables -N recent_remove
# Последнее обращение на неиспользуемые порты было менее суток назад
# поэтому оставим-ка пока этот адресок в черном списке
iptables -A recent_remove -m recent --rcheck --seconds 86400 -j RETURN
# Если адрес в есть в списке — удаляем его оттуда
iptables -A recent_remove -m recent --remove
# Вставляем эту проверку вторым правилом (сразу после проверки ctstate)
iptables -I INPUT 2 -j recent_remove
```

Проводя аналогию с ныне почившим проектом PortSentry (http://www.opennet.ru/docs/RUS/portsentry/portsentry-security.html.gz), первый наш пример соответствует режиму PortSentry «Advanced Stealth Scan Detection», а второй — «Enhanced Stealth Scan Detection». (Правда, заметим, что в режиме Advanced Stealth Scan Detection блокировка производится после *первого* попадания пакета на нерабочий порт — для достижения такого же эффекта в нашем примере достаточно выкинуть проверки на seconds и hitcount из блокирующих правил.) Однако, использование iptables/recent имеет значительное преимущество перед примитивными userspace-системами наподобие PortSentry — возможность интеграции с системой conntrack, что позволяет избежать ошибочной блокировки, например, при использовании высоких портов для NAT.

- Открытие порта «по стуку».

Port knocking — метод защиты портов, при котором доступ к определенному порту с отдельно взятого IP-адреса открывается после серии пакетов на заданную последовательность портов с заданными интервалами. Обычно эта задача решается при помощи демона knockd либо критерия pknock из комплекта xtables-addons.

Несколько самых простых вариантов такой защиты можно организовать и средствами критерия recent.

Самое простое — открывать порт ssh (22) после стука в заданный высокий порт:

```
iptables -N ssh_knock # Создаем цепочку для проверки попыток соединений на защищаемый порт
# Если за последние 60 секунд было 2 и более стука — блокируем, на всякий случай
iptables -A ssh_knock -m recent --rcheck --seconds 60 --hitcount 2 -j RETURN
# Если за последние 10 секунд стук в нужный порт был — разрешить соединение
iptables -A ssh_knock -m recent --rcheck --seconds 10 -j ACCEPT
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Разрешаем пакеты по уста
# Все попытки открыть новое соединение по SSH направляем на проверку
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 22 -j ssh_knock
# Здесь мы добавляем правило для регистрации стука
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 27520 -m recent --set
# Опять же на всякий случай — при стуке в соседние порты закрываем SSH
iptables -A INPUT -m conntrack --ctstate NEW -p tcp -m multiport --dport 27519,27521 -m recent --
iptables -P INPUT DROP # Что не разрешено — то запрещено
```

Даже в таком простом примере присутствуют жесткие меры защиты: защищаемый порт (22) открывается на 10 секунд после стука в заданный порт (27520), при этом более одного стука в этот порт в течение минуты считается ошибкой. Также стук в соседние с заданным порты сразу закрывает защищаемый порт. Это делается в целях защиты от подбора стука.

Если вам не интересны такие параноидальные меры безопасности, то обратите внимание на этот простой пример:

```
iptables -N ssh_knock # Создаем цепочку для проверки
# Если за последние 10 минут было 5 и более попыток соединения — блокируем
iptables -A ssh_knock -m recent --name ssh --update --seconds 600 --hitcount 5 -j RETURN
# Регистрируем
iptables -A ssh_knock -m recent --name ssh --set
# Если за последние 5 секунд было менее двух попыток — блокируем
iptables -A ssh_knock -m recent --name ssh ! --rcheck --seconds 5 --hitcount 2 -j RETURN
# Все остальное — разрешаем
iptables -A ssh_knock -j ACCEPT
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Разрешаем пакеты по устан
# Все попытки открыть новое соединение по SSH направляем на проверку
iptables -A INPUT -m conntrack --ctstate NEW -p tcp --dport 22 -j ssh_knock
iptables -P INPUT DROP # Что не разрешено — то запрещено
```

Идея метода проста: защищаемый порт открывается со второй попытки, то есть стук идет непосредственно в него. Вторую попытку нужно сделать в течение 5 секунд после первой. При этом 5 и более попыток за десять минут блокируются (во избежание брутфорса).

Также можно организовать защиту через стук в серию портов:

```
iptables -N reset_knock # Цепочка для сброса процесса стука
iptables -A reset_knock -m recent --name PHASE1 --remove
iptables -A reset_knock -m recent --name PHASE2 --remove
iptables -A reset_knock -m recent --name PHASE3 --remove
iptables -A reset_knock -m recent --name PHASE4 --remove
iptables -N in_phase_2 # Создаем цепочку для фазы 2
iptables -A in_phase_2 -m recent --name PHASE1 --remove # Удаляем запись из списка первой фазы
iptables -A in_phase_2 -m recent --name PHASE2 --set # Добавляем ее в список второй фазы
iptables -N in_phase_3 # Создаем цепочку для фазы 3
iptables -A in_phase_3 -m recent --name PHASE2 --remove # Удаляем запись из списка второй фазы
iptables -A in_phase_3 -m recent --name PHASE3 --set # Добавляем ее в список третьей фазы
iptables -N in_phase_4 # Создаем цепочку для фазы 4
iptables -A in_phase_4 -m recent --name PHASE3 --remove # Удаляем запись из списка третьей фазы
iptables -A in_phase_4 -m recent --name PHASE4 --set # Добавляем ее в список четвертой фазы
iptables -N checked # Для записей, прошедших проверку
iptables -A checked -j reset_knock # Очищаем списки
iptables -A checked -j ACCEPT # Разрешаем пакет
iptables -F INPUT # Очищаем цепочку INPUT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT # Разрешаем пакеты по устан
# Первая фаза
iptables -A INPUT -p tcp --dport 21210 -m recent --name PHASE1 --set -j RETURN
# Для тех, кто присутствует в списке первой фазы — переход во вторую
iptables -A INPUT -p tcp --dport 11992 -m recent --rcheck --name PHASE1 --seconds 5 -g in_phase_2
# И т.д.
iptables -A INPUT -p tcp --dport 16043 -m recent --rcheck --name PHASE2 --seconds 5 -g in_phase_3
iptables -A INPUT -p tcp --dport 23050 -m recent --rcheck --name PHASE3 --seconds 5 -g in_phase_4
```

```
# Если стучатся не в том порядке — сброс
iptables -A INPUT -p tcp -m multiport --dport 21210,11992,16043,23050 -j reset_knock
# Для тех, кто прошел все четыре фазы — разрешаем доступ
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m recent --rcheck --name PHASE4 --
iptables -P INPUT DROP # Дефолтное правило цепочки INPUT
```

Принцип прост — при стуке в порт, соответствующий очередной фазе, проверяется наличие записи в предыдущей фазе. Теперь порт 22 откроется на 5 секунд после стука в порты 21210, 11992, 16043, 23050 со строгим соблюдением порядка перечисления и интервалами не более 5 секунд.

Обратите внимание на технику реализации процесса — цепочки фаз вызываются не через `-j`, а через `-g`, поэтому после прохождения этих цепочек к пакетам сразу применяется правило по умолчанию цепочки INPUT, то есть DROP. В противном случае пакеты доходили бы до правила сброса (которое с multiport), и процесс стука все время сбрасывался бы.

Последовательный стук в порты можно организовать, например, утилитой netcat

```
for it in {21210,11992,16043,23050}; do
    echo " " | nc -w 1 host $it
done
ssh user@host
```

Другие примеры реализации port knocking’а при помощи критерия recent можно посмотреть на OpenNet wiki (http://wiki.opennet.ru/Iptables_-m_recent), debian-administration.org (http://www.debian-administration.org/articles/268), silverghost.org.ua (http://silverghost.org.ua/2007/06/13/ssh-portknocking/), форуме OpenNet (http://www.opennet.ru/openforum/vsluhforumID3/41732.html#32).

Критерий u32

Данный критерий предоставляет очень гибкий инструмент, позволяющий производить сложные операции по анализу содержимого пакета. Алгоритм обработки содержимого исследуемого пакета задается на специальном языке программирования, своеобразном «ассемблере». Такой подход позволяет получить огромную гибкость, но при этом его изучение и использование может представлять определенные трудности для пользователей, не знакомых с основами программирования и принципами работы машины Тьюринга.

Этот критерий имеет единственный параметр — `[!] --u32`, после которого в двойных кавычках записывается текст «программы». Если все проверки, заданные в программе, дают истинный результат, пакет считается удовлетворяющим данному критерию. Традиционно, если перед

параметром указан восклицательный знак, означающий логическое отрицание, то удовлетворяющими критерию считаются только те пакеты, для которых хотя бы одна проверка дала ложный результат.

Ключевые операторы языка критерия u32:

- **=** — оператор сравнения. Собственно, на его основе и формируются все производимые проверки. Синтаксис его вполне очевиден: `выражение = значение`. В качестве значения можно использовать одно число, диапазон (в формате `мин:макс`) или список чисел и/или диапазонов разделенных запятой (например, `1,3:5,8,11:14`). В одном списке можно указывать не более 10 чисел/диапазонов.
- **&&** — оператор логического И, объединяющий все производимые в программе проверки. В одном критерии u32 можно указывать не более 10 таких проверок. Обратите внимание, что операция логического ИЛИ отсутствует, так как это противоречит идеологии iptables (в каждом правиле все проверки объединяются логическим И, а логическое ИЛИ объединяет правила в одной цепочке).
- **&** — оператор побитового логического И, позволяющий применить к обрабатываемому значению битовую маску.
- **<<** и **>>** — операторы побитового сдвига влево и вправо соответственно. Эквивалентны домножению или делению обрабатываемого значения на 2 в соответствующей степени. Обратите внимание, что в одном выражении для проверки можно использовать не более 9 операторов &, << и >>.
- **@** — оператор задания смещения. Добавляет результат предыдущего выражения к текущему значению начального смещения (в начале работы программы, до выполнения @-операций, начальное смещение считается нулевым). Очень полезен в тех случаях, когда нужно сместиться на некоторое значение, полученное в результате обработки пакета, например, узнав после некоторых вычислений длину IP-заголовка, мы можем сместиться на данные непосредственно после него, т.е. на начало полезной нагрузки IP-пакета (как правило, там находится заголовок транспортного уровня).

Все операторы имеют равный приоритет, исполнение операций всегда происходит слева направо.

Обратите внимание, что программа рассматривает все значения как целые беззнаковые величины, заданные 32 битами, или четырьмя октетами (что соответствует значениям от 0 до 4294967295). Именно поэтому критерий имеет такое название (u32 — unsigned 32-bit).

Другой не вполне очевидный момент — синтаксис операторов =, &, << и >>. Дело в том, что в качестве первого операнда указывается не само значение, а его *адрес* в пределах пакета, заданный смещением в байтах от текущего начального смещения. Например, `8 = 0x12345678` означает, что нужно взять четыре байта, начиная с восьмого (обратите внимание, что

нумерация байтов в пакете начинается нуля), т.е. байты 8, 9, 10 и 11, и сравнить полученное значение с шестнадцатеричным числом 0x12345678. Как уже говорилось, критерий u32 *всегда* манипулирует блоками по четыре байта, поэтому, если вас интересуют блоки меньшего размера, используйте оператор & и маску, например, `8 & 0xFF = 0x78` позволяет проверить только последний в блоке (11-й от начального смещения) байт. Также можно использовать и оператор смещения, например, `8 >> 24 = 0x12` возьмет первый в блоке (8-й от начального смещения) байт и сравнит его с числом 0x12. Более подробно о битовых операциях можно почитать в соответствующей статье.

Рассмотрим возможности критерия u32 на нескольких простых примерах из официальной документации:

- Проверить, превышает ли длина пакета величину 256 байт: `--u32 "0 & 0xFFFF = 0x100:0xFFFF"`.

Эта программа предписывает взять четыре байта, начиная с нулевого, затем занулить первые два байта (в них содержится информация о версии протокола, длине заголовка и типе обслуживания, которая нас сейчас не интересует), и проверить значение, формируемого оставшимися двумя байтами (это и есть полная длина пакета в байтах) на принадлежность диапазону от 0x100 (256 в десятичном счислении) до 0xFFFF (65535, максимально возможная длина пакета).

- Проверить, является ли данный пакет ICMP-пакетом с типом 0 (эхо-ответ). Учитывая переменную длину IP-заголовка, эта проверка требует несколько большего количества операций: `--u32 "6 & 0xFF = 1 && 4 & 0x3FFF = 0 && 0 >> 22 & 0x3C @ 0 >> 24 = 0"`

Рассмотрим эти операции последовательно:

- **6 & 0xFF = 1** — проверка кода протокола. Код протокола хранится в 9-м (считая с нуля) байте IPv4-заголовка. Чтобы получить это значение, мы берем 4-х байтовый блок с шестого по девятый байт, а затем при помощи маски выделяем последний, интересующий нас байт в это блоке, и сравниваем его с единицей (код протокола ICMP).
- **4 & 0x3FFF = 0** — проверка, не является ли наш пакет фрагментом. Фрагменты ICMP-пакетов не содержат ICMP-заголовка, в котором должен храниться тип ICMP, поэтому для них дальнейшая проверка не имеет смысла. Пользуясь описанным выше принципом, наша программа выделяет последние 14 бит в блоке с 4-го по 7-й байт, и сравнивает полученное значение с нулем. Строго говоря, для наших целей достаточно было бы и 13 бит (маска 0x1FFF), выделяющих только поле смещения фрагмента (fragment offset, оно должно быть равным нулю для первого пакета в цепочке фрагментов), а использование 14 бит, как это сделано в примере из официальной

документации, также захватит и флаг MF (more fragments), что отсечет вообще все фрагментированные пакеты, даже те, которые являются первыми в последовательности и содержат все нужные нам заголовки.
- **0 >> 22 & 0x3C @** — определение длины IP-заголовка в байтах. Последние четыре бита в первом байте IP-пакета показывают длину IP-заголовка в блоках по четыре байта. Смещая это значение на 22 позиции вправо, мы оставляем справа «зазор» в два бита, которые впоследствии зануляем при помощи маски ($3C_{16} = 111100_2$). Также эта маска отсекает лишние биты слева (версия протокола). Полученные два нулевых бита справа соответствуют умножению на четыре, так что в результате мы получаем длину заголовка уже в байтах. Например, если длина IP-заголовка составляет 5 четырехбайтовых блоков (т.е. 20 байт), то изначально первый блок заголовка имеет вид `xxxx0101 yyzzzzzz uuuuuuuu uuuuuuuu` (где биты `xxxx` задают версию протокола, $0101_2=5_{10}$ — собственно длину заголовка, `yyzzzzzz` — тип обслуживания, а два байта `uuuuuuuu uuuuuuuu` показывают полную длину пакета вместе с нагрузкой), то после смещения вправо на 22 бита мы получим в этом блоке следующую картину: `00000000 00000000 000000xx xx0101yy`. После применения маски она пример вид `00000000 00000000 00000000 00010100`. Полученное число — $10100_2=20_{10}$ и будет равно длине заголовка в байтах. Наконец, оператор @ устанавливает полученное значение в качестве начального смещения, поэтому вся дальнейшая адресация ведется уже начиная с начала ICMP-заголовка, который следует непосредственно после IP-заголовка.
- **0 >> 24 = 0** — выделяет блок с нулевого по третий байт ICMP-заголовка, а затем при помощи смещения (на три байта вправо) оставляет от него только один, нулевой, байт, в котором и хранится тип ICMP. Нас интересует тип 0, поэтому мы и сравниваем этот байт с нулем.

- Проверить, задают ли байты с восьмого по одиннадцатый полезной нагрузки TCP-пакета одно из значений: 1, 2, 5, 8. Достигается следующим кодом: `--u32 "6 & 0xFF = 6 && 4 & 0x3FFF = 0 && 0 >> 22 & 0x3C @ 12 >> 26 & 0x3C @ 8 = 1,2,5,8"`

  - **6 & 0xFF = 6** — выделение байта с кодом протокола и сравнение его с кодом 6 (соответствует TCP). Описание этой проверки уже приведено при рассмотрении предыдущего примера.
  - **4 & 0x3FFF = 0** — проверка, не является ли данный пакет фрагментом. Полностью аналогична описанной в предыдущем примере.
  - **0 >> 22 & 0x3C @** — определение длины IP-заголовка в байтах. Также полностью аналогично описанному в предыдущем примере. После данной операции, начальное смещение

указывает на нулевой байт TCP-заголовка.
- **12 >> 26 & 0x3C @** — определение длины TCP-заголовка в байтах. Принцип действия аналогичен предыдущей операции, с тем лишь отличием, что длина TCP-заголовка закодирована в первых четырех битах двенадцатого байта (а не в последних четырех битах нулевого байта, как в случае с IP-заголовком), поэтому необходимое нам смещение вправо составляет уже 26 бит. Как и в предыдущей операции, мы оставляем справа зазор в два бита, которые впоследствии зануляем маской, и таким образом сразу получаем длину заголовка в байтах. Итак, после второго оператора @, начальное смещение будет указывать на начало полезной нагрузки TCP-пакета (следует непосредственно после заголовка).
- **8 = 1,2,5,8** — эта операция сравнения вполне тривиальна: выделяется четырехбайтовый блок, начиная с восьмого байта (разумеется, считая от начала полезной нагрузки), и полученное значение сравнивается с одним из допустимых по условию задачи.

Также, в качестве полезных примеров использования критерия u32, можно упомянуть проверку портов источника и назначения в протоколе UDPLite (en): `--u32 "6 & 0xFF = 136 && 4 & 0x3FFF = 0 && 0 >> 22 & 0x3C @ 0 >> 16 = исходный_порт"` и `--u32 "6 & 0xFF = 136 && 4 & 0x3FFF = 0 && 0 >> 22 & 0x3C @ 0 & 0xFFFF = порт_назначения"` соответственно (после сказанного выше нетрудно догадаться, что исходный порт указывается в первой паре байт UDPLite-заголовка, а порт назначения — во второй, согласно RFC3828 (http://tools.ietf.org/html/rfc3828)).

Прочие критерии

Шаблон:В планах

Критерии из набора xtables-addons

Шаблон:В планах

Программы

Основные

iptables

Работа с цепочками

Работа с правилами

Вывод информации

iptables-save, iptables-restore и iptables-xml

/etc/init.d/iptables

iptables-apply

ipset

ipset работает с типами данных, которые представляют из себя:

- ipmap — список IP адресов из определенной подсети
- macipmap — связки IP+MAC
- portmap — список портов
- iphash — произвольный набор IP
- nethash — произвольный набор сетей
- iptree — временное хранилище IP адресов

Пример правила iptables с применением ipset

```
iptables -I INPUT 1 -i br0 -p tcp -m set --set test123 src -m tcp --dport 8080 -j ACCEPT
```

-где test123 это заданное имя сета

```
ipset -N test123 nethash
```

Вспомогательные

Полезные при конфигурировании фаервола

Полезные при тестировании фаервола

Фронтенды

С удобным интерфейсом

С графическим интерфейсом

С веб-интерфейсом

iptadmin (http://www.117.su) - это пользовательский web интерфейс для iptables, редактор настроек файрволла операционной системы Linux. Реализован в виде отдельного http(s) сервера.

Особенностью программы являются подробные сообщения об ошибках при редактировании правил файрволла.

iptadmin работает на уровне iptables. Настройки файрволла сохраняются в стандартных файлах. В системе не создаются никакие вспомогательные данные. Логика работы спроектирована таким образом, что нарушение целостности конфигурации iptables исключено. Iptadmin можно в любой момент установить или удалить без потери настроек файрволла.

Программные интерфейсы

Модули ядра

Модули conntrack и nat

Модули критериев и действий

Параметры sysctl/procfs

Псевдофайлы procfs

Базовые

Относящиеся к критериям и действиям

Основные параметры sysctl

ip_forward=1

Параметры конфигурации интерфейсов

Параметры, относящиеся к протоколам

TCP

UDP

ICMP

Параметры conntrack

Расширения

Userspace-компоненты

NuFW

l7-filter-userspace

iplist

Наборы дополнительных критериев и действий

Patch-o-Matic

Смотри на официальном сайте http://www.netfilter.org/projects/patch-o-matic/pom-external.html

Patch-o-Matic-NG

xtables-addons

Команды

Программа позволяет задать правила, которым должны соответствовать проходящие через брандмауэр IP-пакеты. Эти правила, как и все настройки Linux, записываются в текстовый файл, находящийся в папке /etc. Последовательность правил называется цепочкой (CHAIN). Для одного и того же сетевого интерфейса используются несколько цепочек. Если проходящий пакет не соответствует ни одному из правил, то выполняется действие по умолчанию.

При вызове, параметром указывается команда, которую нужно выполнить. Обычно можно указать только одну команду (но есть исключения). Команду можно указать одной большой буквой или словом. Если при вызове любой команды не указано название таблицы, то команда выполняется в таблице filter. Программа имеет подробную справку, вызываемую командой *man iptables*.

Вывести правила (`-L`, `--list`)

iptables [-t *таблица*] -L [*цепочка*] [*параметры*]

Вывести список правил для указанной таблицы и цепочки. Если цепочка не указана, то выводятся список правил для каждой цепочки. Например для вывода правил из таблицы nat: `iptables -t nat -n -L`

Часто используются параметры -n (во избежение медленных запросов DNS) и -v (для вывода более подробной информации).

Команду -L можно использовать с -Z (`iptables -L -Z`) для вывода значений счетчиков и одновременного их обнуления.

Удалить все правила из цепи (`-F`, `--flush`)

iptables [-t *таблица*] -[F] [*цепочка*] [*параметры*]

Если цепочка не указана, то удаляются все цепочки из таблицы.

Обнулить все счетчики (`-Z`, `--zero`)

iptables [-t *таблица*] -Z [*цепочка*] [*параметры*]

Присваивает счетчикам числа пакетов и объема данных нулевые значения. Если цепочка не указана, то обнуление выполняется для всех цепочек.

Команду -Z можно использовать с -L (`iptables -L -Z`) для вывода значений счетчиков и одновременного их обнуления.

Создать новую цепочку (`-N`, `--new-chain`)

iptables [-t *таблица*] -N *цепочка*

Создать новую цепочку в указанной таблице с указанным именем. Если в указанной таблице уже есть цепочка с указанным именем, то новая не создается.

Удалить созданную цепочку (`-X`, `--delete-chain`)

iptables [-t *таблица*] -X [*цепочка*]

Удалить цепочку ранее созданную с помощью команды **-N**. Перед удалением цепи необходимо удалить или заменить все правила, которые ссылаются на эту цепочку. Если цепочка не указана, то из таблицы будут удалены все цепи кроме стандартных (INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING). Стандартные цепочки не удаляются.

Переименование цепи (`-E`, `--rename-chain`)

iptables [-t *таблица*] -E *цепочка новое_название*

Присваивает указанной цепочке новое название. Косметическая функция, не влияющая на функциональность.

Установить политику для стандартной цепи (`-P`, `--policy`)

iptables [-t *таблица*] -P *цепочка действие* [*параметры*]

Над пакетами которые доходят до конца указанной цепочки будет выполняться указанное действие. В качестве действия нельзя указывать название какой-либо цепочки. Устанавливать политику можно только на встроенных цепочках. Например: `iptables -P INPUT DROP`

Команды модификации правил

Добавить новое правило (`-A, --append chain`)

iptables [-t *таблица*] -A *цепочка спецификация_правила* [*параметры*]

Правило добавляется в конец указанной цепочки. Если в спецификации правила указано имя отправителя или получателя, которое одновременно соответствует нескольким адресам, то в конец цепочки добавляются правила для всех возможных комбинаций.

Вставить новое правило (`-I, --insert`)

iptables [-t *таблица*] -I *цепочка* [*номер_правила*] спецификация_правила [*параметры*]

Указанное правило вставляется в указанное место указанной цепочки. Правила нумеруются с 1, поэтому если указать номер 1 (или не указать вообще), то правило будет вставлено в начало цепочки.

Удалить правило (`-D, --delete`)

iptables [-t *таблица*] -D *цепочка номер_правила* [*параметры*]

iptables [-t *таблица*] -D *цепочка спецификация_правила* [*параметры*]

Правило можно указывать при помощи его номера в цепочке (нумерация начинается с 1) или его спецификации.

Заменить правило (`-R, --replace`)

iptables [-t *таблица*] -R *цепочка номер_правила спецификация_правила* [*параметры*]

Заменить правило с указанным номером в указанной цепочке. Правила нумеруются с 1. Спецификация правила не может содержать имени отправителя или получателя, которое одновременно соответствует нескольким адресам.

Параметры определения правил

Важно. В последующих версиях описанный ниже синтаксис изменился. Например, старому синтаксису

- **-s**, --src, --source [!] *адрес*[*/маска*]

теперь соответствует новый

- [!] **-s**, --src, --source *адрес*[*/маска*]

Перечисленные ниже параметры используются при задании спецификации правил и указываются с командами модификации правил. Эти параметры ограничивают применение правил — если обрабатываемый пакет не соответствует указанным в спецификации критериям, то

указанное в правиле действие на этот пакет не распространяется.

- **-p**, `--protocol [!]` *протокол*

Ограничение протокола. Основные значения: `tcp`, `udp`, `icmp`, или `all`. Протокол также можно указать с помощью номера, или названия указанного в файле `/etc/protocols`. Знак «!» перед именем протокола изменяет критерий на противоположенный (например `!tcp` означает «любой протокол, кроме TCP»). Значение «Любой протокол» можно указать с помощью слова `all` или числа 0. Если протокол не указан, то подразумевается «Любой протокол».

- **-s**, `--src, --source [!]` *адрес*[*/маска*]

Ограничение отправителя. Адрес может быть IP-адресом (возможно с маской), именем хоста, или доменным именем. Маска может быть в стандартном формате (например 255.255.255.0) или же в виде числа, указывающего число единиц с «левой стороны» маски (например 24). Знак «!» перед адресом изменяет критерий на противоположенный.

Настоятельно не рекомендуется использовать имена, для разрешения которых требуется удаленный запрос, например по системе DNS.

- **-d**, `--dst, --destination [!]` *address*[*/mask*]

Ограничение получателя. Синтаксис такой же, как у --src.

- **-i**, `--in-interface [!]` *имя_интерфейса*

Ограничение входящего сетевого интерфейса. Знак «!» перед адресом изменяет критерий на противоположенный. Если указанное имя интерфейса заканчивается на «+», то критерию совпадают все интерфейсы, чьи имена начинаются на указанное имя. Если параметр --in-interface не указан, то критерию соответствуют пакеты из любого сетевого интерфейса.

- **-o**, `--out-interface [!]` *имя_интерфейса*

Ограничение выходящего сетевого интерфейса. Синтаксис такой же, как и для --in-interface.

- `[!]` **-f**, `--fragment`

Ограничение пр фрагментам: критерию соответствуют только фрагменты пакета, начиная со второго фрагмента. Знак «!» перед адресом изменяет критерий на противоположенный.

У таких фрагментов начиная со второго нет заголовка с портами отправителя и получателя (или с типом ICMP). Следовательно, такие фрагменты не соответствуют критериям, указывающим номера портов.

- **-j**, `--jump` *действие_или_цепочка*
- **-g**, `--goto` *цепочка*

Спецификация действий и переходов. Если указанно название цепочки ранее созданной командой -N, то пакеты соответствующие критериям правила переносится в начало указанной цепочки (запрещено указывать название цепочки, в котором это правило находится). Если указано действие, то оно выполняется над пакетами соответствующим критериям правила. Если в правиле нет параметров --jump и --goto, то правило не влияет на проверяемые пакеты (но счетчик правила продолжает работать).

--goto отличается от --jump поведением при действии RETURN. Действие RETURN переводит пакет в правило, следующее после того, которое вызвало предыдущий переход --jump. То есть, если пакет перешел из цепочки X в цепочку Y при помощи --jump, а потом в Z опять при помощи --jump, то действие RETURN из цепочки Z возвращает его в Y. Если же пакет перешел в Z при помощи --goto, то RETURN возвращает его в X.

- **-c**, `--set-counters` *пакеты байты*

Параметр позволяет при добавлении или изменении правил одновременно инициализировать счетчики числа пакетов и размера данных. Подстрочный текст

Модули

Кроме фильтров по умолчанию поддерживаются дополнительные модули, подключающиеся автоматически при выборе протокола (**-p**) или вручную опцией **-m** (*--match*), после которой следует имя подключаемого фильтра и его опции. Также как и для остальных правил фильтрации, здесь можно использовать **!** в качестве отрицания.

Некоторые из встроенных (входящие в стандартный пакет)

state

Соединением для NetFilter является закономерно оформленная последовательность обмена пакетами между хостами, с выделением роли каждого пакета в соединении. Эта особенность позволяет контролировать трафик с точки зрения транспортного уровня (модели OSI). Ядро NetFilter отслеживает соединения и ведёт активное наблюдение за открытием новых, протеканием и завершением установленных соединений. Каждый пакет может быть классифицирован по одному из следующих состояний: NEW, ESTABLISHED, RELATED и INVALID.

NEW — типов пакетов, участвующих в установке нового соединения.

ESTABLISHED — пакет уже установленного соединения

RELATED — Соединение получает статус RELATED если оно связано с другим соединением, имеющим признак ESTABLISHED. Это означает, что соединение получает признак RELATED тогда, когда оно инициировано из уже установленного соединения, имеющего признак ESTABLISHED.

Состояние RELATED относится к служебному трафику, протоколу ICMP стека TCP/IP.

INVALID — неправильный/ошибочный/испорченный пакет. Обычно для таких пакетов применяют действие DROP

ПРИМЕР:

```
iptables -t nat -A PREROUTING -s 10.10.10.10 -m conntrack --ctstate NEW,INVALID -j DROP
```

Все новые и ошибочные пакеты, идущие с адреса 10.10.10.10 отбрасываются

connlimit

Позволяет задавать возможное количество одновременных подключений к машине от заданного IP или блока адресов

`[!] --connlimit-above n` — пакет подойдет под описание, если количество одновременных подключений на данный момент больше (меньше), чем n

`--connlimit-mask bits` — позволяет задать маску блока адресов

iprange

Выделяет не один адрес, как `--src`, а все адреса от ip1 до ip2

Нужно учитывать тот факт что перед использованием данного критерия, необходимо подгрузить модуль его реализующий «-m iprange»

```
[!]--src-range ip1-ip2
```

```
[!]--dst-range ip1-ip2
```

multiport

Выделяет не один порт, как `--dport` или `--sport`, а несколько по списку (до 15 штук). Можно задавать диапазоны как первый_порт:последний_порт. Может быть использовано только вместе с `-p udp` или `-p tcp`.

```
[!]--source-ports port1,port2,port3:port4
```

`[!]--destination-ports port1,port2,port3:port4`

`[!]--ports port1,port2,port3:port4`

mac

Сравнение не IP источника пакета, а его ethernet-адреса

`[!] --mac-source address`. MAC-адрес должен быть указан в стандартном формате XX:XX:XX:XX:XX:XX.

Если нужна фильтрация по MAC-адресу в мостовых соединениях (например, pppoe), то можно воспользоваться утилитой ebtables, которая предназначена как раз для более низкоуровневой фильтрации пакетов.

Примечания

1. ↑ Разработчики Netfilter представили замену iptables (http://www.opennet.ru/opennews/art.shtml?num=20843)
2. ↑ Если кратко, EtherType указывает тип протокола, инкапсулированного в Ethernet-кадр. При работе с iptables это значение будет всегда равно 08:00 (протокол IPv4), при работе с ip6tables — 86:DD (IPv6). Полный список EtherTypes доступен на сайте IANA (http://www.iana.org/assignments/ethernet-numbers), список поддерживаемых netfilter'ом — в файле /etc/ethertypes, который в большинстве дистрибутивов находится в пакете ebtables.
3. ↑ Обычно проблемы с пониженным MTU в Ethernet-сетях возникают как раз из-за VPN-туннелей
4. ↑ На самом деле, для «чистого» проброса адреса, даже при использовании stateful NAT, необходимо обычно два или три правила, в частности, для проброса соединений с самого сервера в цепочке OUTPUT. Более подробно это описано при рассмотрении операции DNAT таблицы nat, и здесь, с целью упрощения понимания примера, эти аспекты опущены
5. ↑ Заметим, что для расчета масок подсетей и диапазонов адресов существует очень удобная утилита ipcalc.

Литература

- *Gregor N. Purdy* Linux iptables. Pocket Reference. — O'Reilly, 2004. — С. 97. — ISBN 0-596-00569-5

Ссылки

- Настройка межсетевого экрана в Linux
- http://www.netfilter.org/ — Сайт проекта Netfilter
- Man page of iptables (http://security.maruhn.com/iptables-manual.html) (англ.)
- Frozentux Iptables Tutorial (http://www.frozentux.net/documents/iptables-

tutorial/)* (англ.)
- Руководство по iptables (Iptables Tutorial 1.1.19) (http://www.opennet.ru/docs/RUS/iptables)* (рус.)
- Простой полноценный межсетевой экран (готовый шаблон для дектопа) (http://wiki.archlinux.org/index.php/Simple_stateful_firewall_HOWTO#Prerequisites) (англ.)
- Настройка межсетевого экрана Iptables (готовый шаблон для дектопа) (http://posix.ru/network/iptables)* (рус.)
- http://freecode.com/projects/flex-fw — гибкая надстройка над iptables для построения сервис-ориентированного statefull-фаервола с простым синтаксисом команд.
- iptables(8) (http://linux.die.net/man/8/iptables) (англ.)
- ufw(8) (http://linux.die.net/man/8/ufw) (англ.)
- gufw(8) (http://linux.die.net/man/8/gufw) (англ.)
- https://help.ubuntu.com/community/IptablesHowTo (англ.)
- https://help.ubuntu.com/community/UFW (англ.)
- https://help.ubuntu.com/community/Gufw (англ.)
- https://help.ubuntu.com/9.10/serverguide/C/firewall.html (англ.)
- https://wiki.ubuntu.com/UncomplicatedFirewall (англ.)
- https://help.ubuntu.com/community/Firewall (англ.)

Источник — «https://ru.wikibooks.org/w/index.php?title=Iptables&oldid=116729»

Категории: Linux | Компьютерные сети

- Последнее изменение этой страницы: 18:42, 14 мая 2015.
- Текст доступен по лицензии Creative Commons Attribution-ShareAlike, в отдельных случаях могут действовать дополнительные условия. Подробнее см. Условия использования.